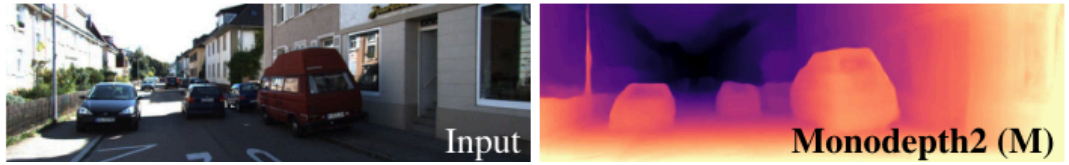
Self-Supervised Learning for Robot Perception

INF8422: Robot Perception and Spatial Intelligence

Prof. Pierre-Yves Lajoie · Polytechnique Montréal

POLYTECHNIQUE MONTRÉAL

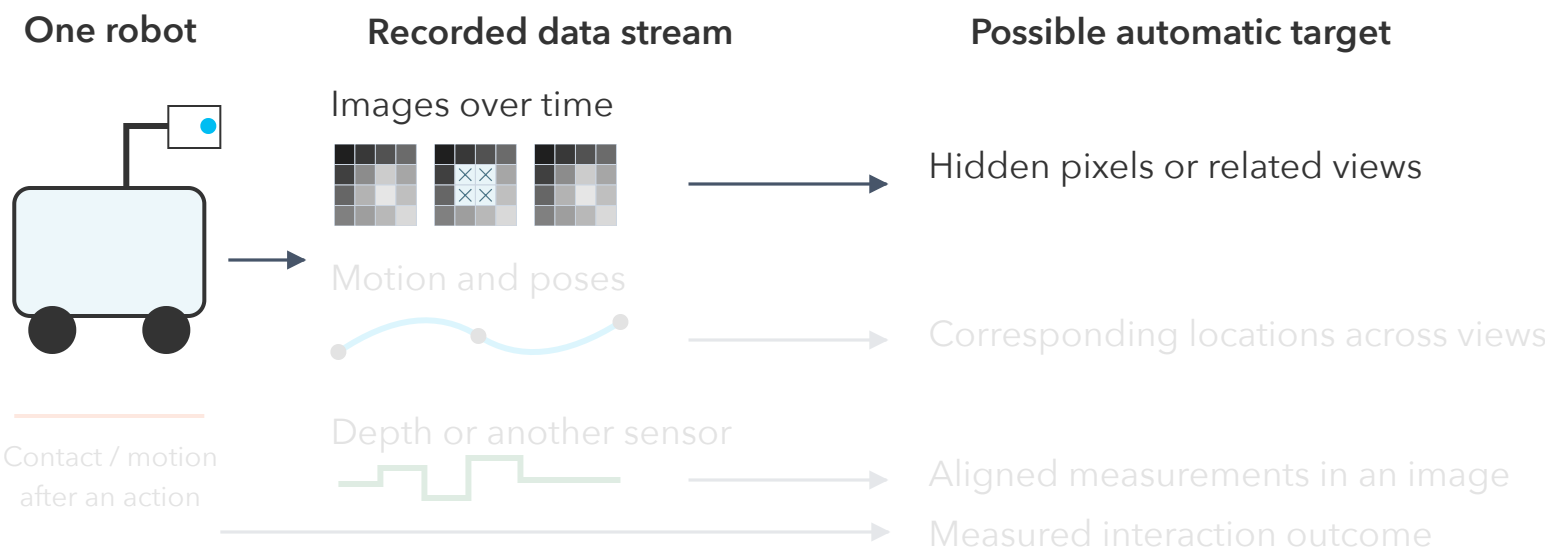


Camera observation and estimated depth
Godard et al., Monodepth2, ICCV 2019, Fig. 1 (top row)

SECTION 01

Self-supervised learning mechanics

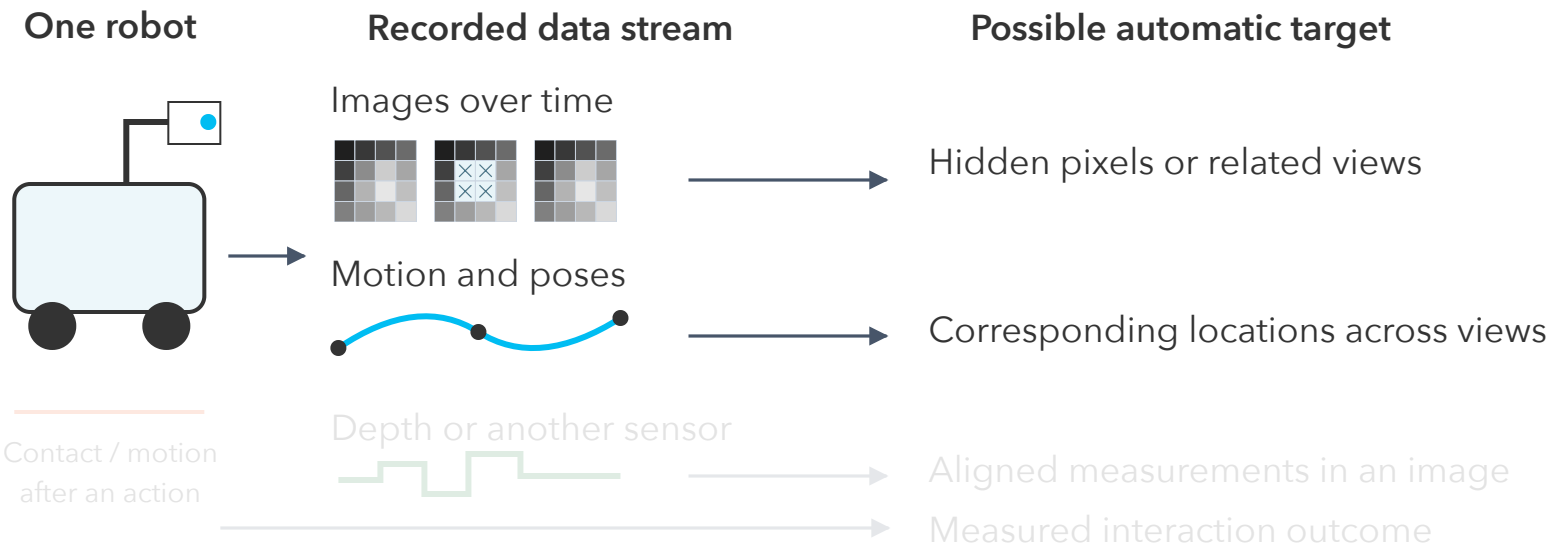
Supervision available to a robot



The target needs a construction rule

Recorded data can supply a training signal when we define a valid relationship between observations.

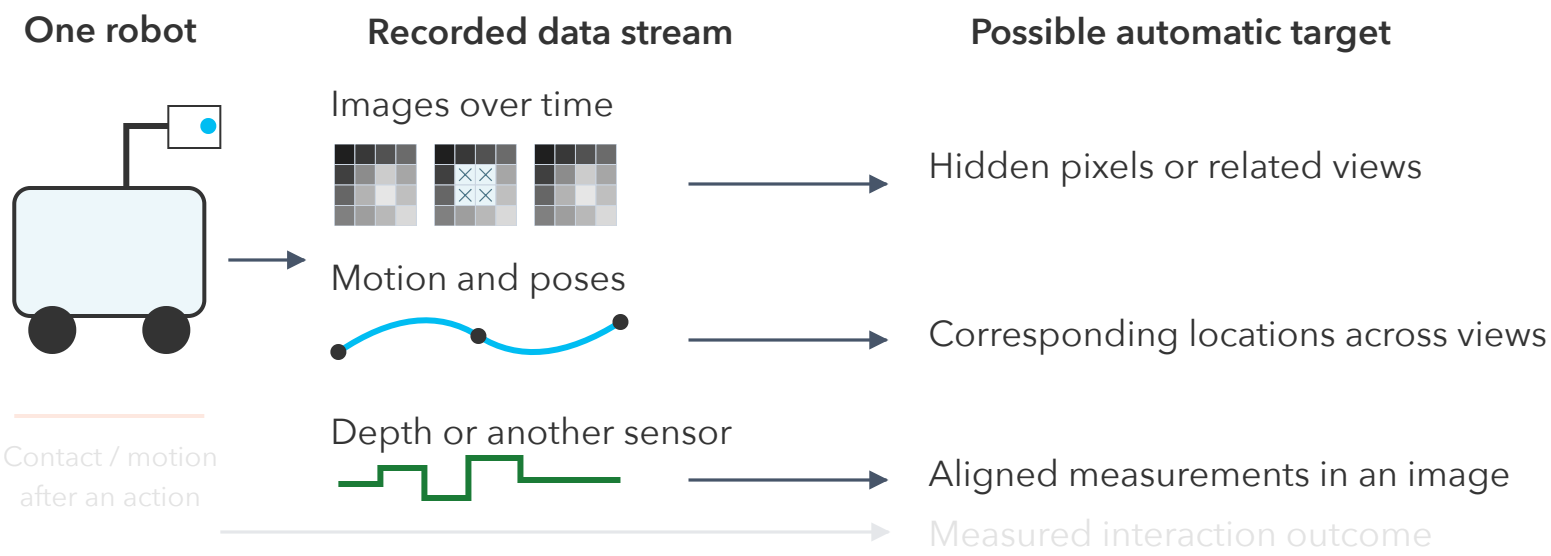
Supervision available to a robot



The target needs a construction rule

Recorded data can supply a training signal when we define a valid relationship between observations.

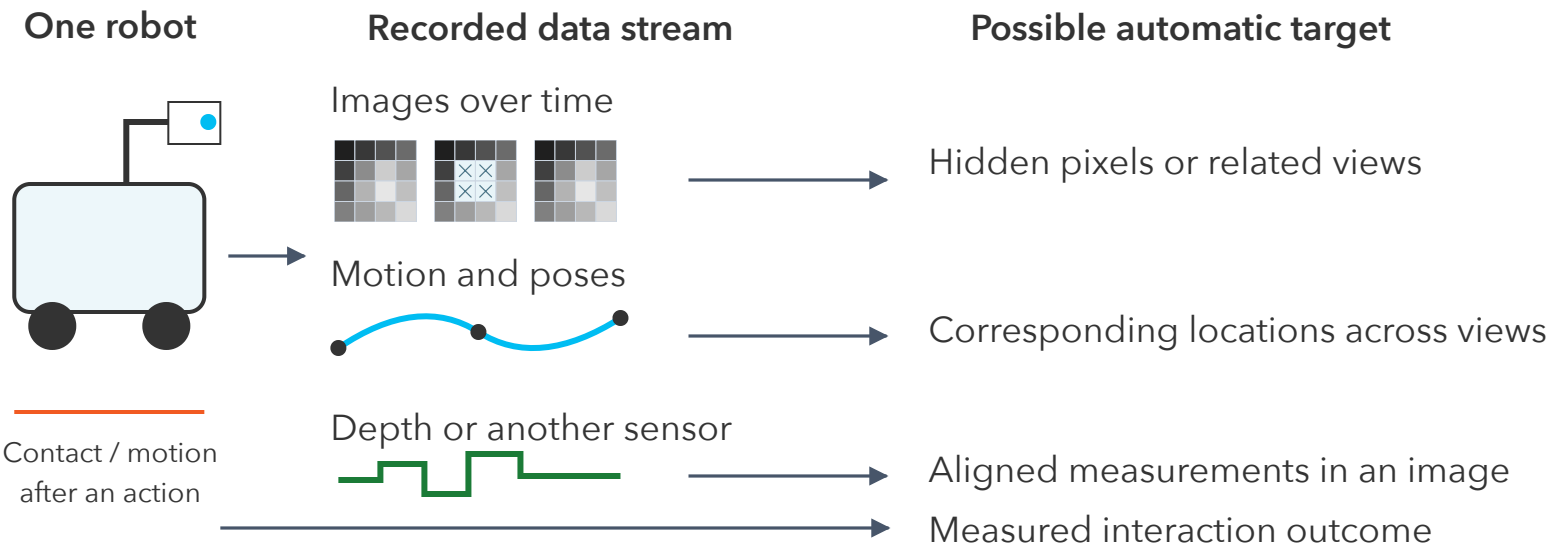
Supervision available to a robot



The target needs a construction rule

Recorded data can supply a training signal when we define a valid relationship between observations.

Supervision available to a robot

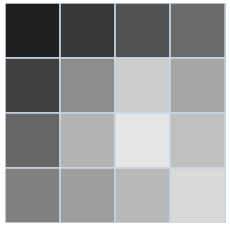


The target needs a construction rule

Recorded data can supply a training signal when we define a valid relationship between observations.

Definition of self-supervised learning

Self-supervised learning trains a model using supervisory signals derived automatically from the data itself, without requiring manually annotated labels for that learning stage.



Same observation

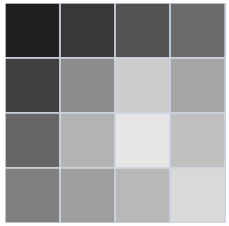
→ Predict an object label ← **"object"** Human annotation

→ Predict hidden pixels ← Copied from the observation

> The origin of the training target determines the supervision setting.

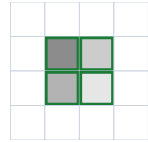
Definition of self-supervised learning

Self-supervised learning trains a model using supervisory signals derived automatically from the data itself, without requiring manually annotated labels for that learning stage.



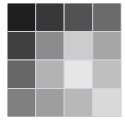
Same observation

→ Predict an object label ← **"object"** Human annotation

→ Predict hidden pixels ←  Copied from the observation

> The origin of the training target determines the supervision setting.

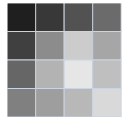
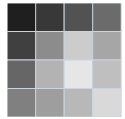
Supervised, self-supervised, and semi-supervised learning

Supervised		Manually labeled examples	→ Annotation-based loss
Self-supervised		Automatically constructed targets	→ Data-derived loss
Semi-supervised		Some labeled + unlabeled examples	→ Supervised + unlabeled-data terms

The loss (cost) function alone does not identify the setting

Squared error and cross-entropy can compare predictions to either manual or automatic targets.

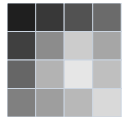
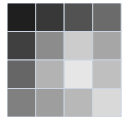
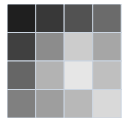
Supervised, self-supervised, and semi-supervised learning

Supervised		Manually labeled examples	→ Annotation-based loss
Self-supervised		Automatically constructed targets	→ Data-derived loss
Semi-supervised		Some labeled + unlabeled examples	→ Supervised + unlabeled-data terms

The loss (cost) function alone does not identify the setting

Squared error and cross-entropy can compare predictions to either manual or automatic targets.

Supervised, self-supervised, and semi-supervised learning

Supervised		Manually labeled examples	→ Annotation-based loss
Self-supervised		Automatically constructed targets	→ Data-derived loss
Semi-supervised		Some labeled + unlabeled examples	→ Supervised + unlabeled-data terms

The loss (cost) function alone does not identify the setting

Squared error and cross-entropy can compare predictions to either manual or automatic targets.

Pretext tasks and automatic targets



Hidden-pixel prediction

Input: visible information

→

Withheld observed pixels

Target: selected from the original image

Related-view matching

Input: two images with a known relationship

→

A known pair association

Target: known relation (e.g., rotation, translation, overlap)

Direct outcome prediction

Input: an observation before the action

→

A measured traversal cost: 0.4

Target: measurements after the action

A pretext task builds a representation through prediction or matching.

Pretext tasks and automatic targets



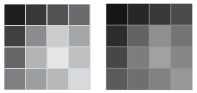
Hidden-pixel prediction

Input: visible information

→

Withheld observed pixels

Target: selected from the original image



Related-view matching

Input: two images with a known relationship

→

A known pair association

Target: known relation (e.g., rotation, translation, overlap)

Direct outcome prediction

Input: an observation before the action

→

A measured traversal cost: 0.4

Target: measurements after the action

A pretext task builds a representation through prediction or matching.

Pretext tasks and automatic targets



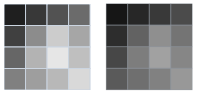
Hidden-pixel prediction

Input: visible information

→

Withheld observed pixels

Target: selected from the original image



Related-view matching

Input: two images with a known relationship

→

A known pair association

Target: known relation (e.g., rotation, translation, overlap)



Direct outcome prediction

Input: an observation before the action

→

A measured traversal cost: 0.4

Target: measurements after the action

A pretext task builds a representation through prediction or matching.

Pretraining and optional fine-tuning



Encoder fθ

Learn a representation

Pretext prediction

Target from the data

Reuse the learned encoder

Encoder fθ

Frozen or adapted

Task head hβ

Object category, for example

Manual task labels can enter here for adaptation or evaluation.

Two adaptation choices

Keep the encoder fixed and train a task head, or fine-tune the encoder together with the head.

Pretraining and optional fine-tuning



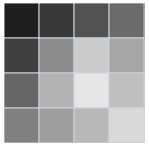
Encoder fθ

Learn a representation

Pretext prediction

Target from the data

Reuse the learned encoder



Encoder fθ

Frozen or adapted

Task head hβ

Object category, for example

Manual task labels can enter here for adaptation or evaluation.

Two adaptation choices

Keep the encoder fixed and train a task head, or fine-tune the encoder together with the head.

Direct learning from measured outcomes

See the terrain **Execute a known fixed motion** **Measure the outcome**



Automatic target Illustrative cost = 1 − measured / commanded = **0.4**

Earlier camera image → **Encoder + task head** → Predicted traversal cost

Direct learning from measured outcomes

See the terrain **Execute a known fixed motion** **Measure the outcome**

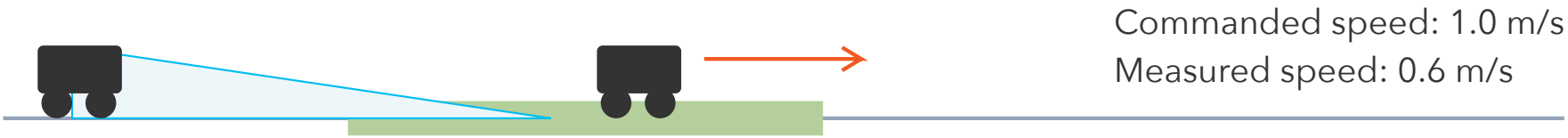


Automatic target Illustrative cost = 1 − measured / commanded = **0.4**

Earlier camera image → **Encoder + task head** → Predicted traversal cost

Direct learning from measured outcomes

See the terrain **Execute a known fixed motion** **Measure the outcome**

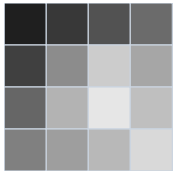


Automatic target Illustrative cost = 1 − measured / commanded = **0.4**

Earlier camera image → **Encoder + task head** → Predicted traversal cost

Masking: Input, prediction, and target

Observed x_i

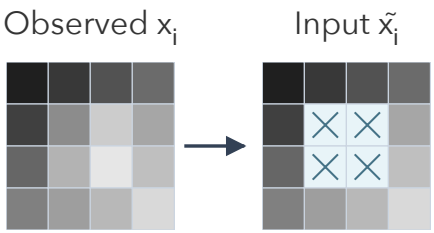


$$x_i \text{ is an observed image}$$

The complete observation is available to the training procedure.

i identifies an observation · Θ contains the encoder and decoder parameters · M selects hidden locations

Masking: Input, prediction, and target

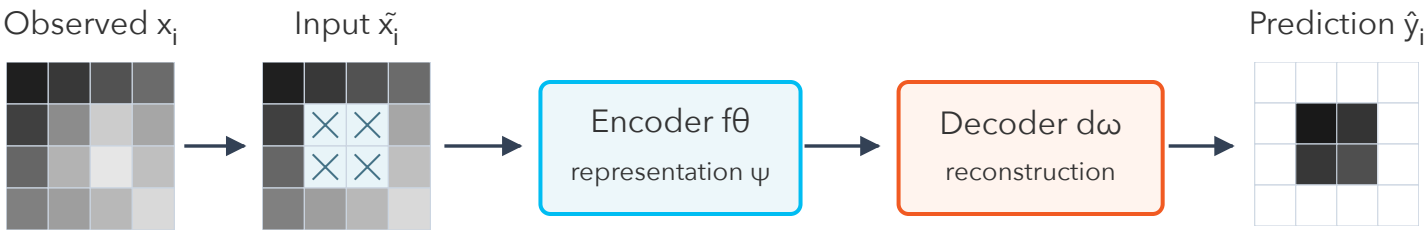


$$\tilde{x}_i = \mathbf{mask}(x_i)$$

The model receives only the visible information. The original is kept separately.

i identifies an observation · Θ contains the encoder and decoder parameters · M selects hidden locations

Masking: Input, prediction, and target

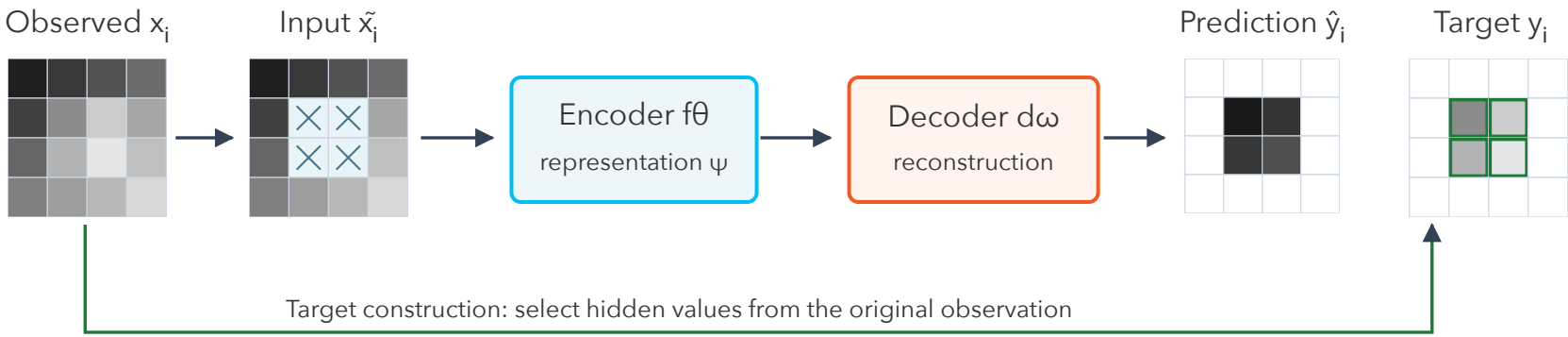


$$\hat{y}_i = P_\Theta(\tilde{x}_i)$$

The encoder and reconstruction decoder predict the withheld values.

i identifies an observation · Θ contains the encoder and decoder parameters · M selects hidden locations

Masking: Input, prediction, and target

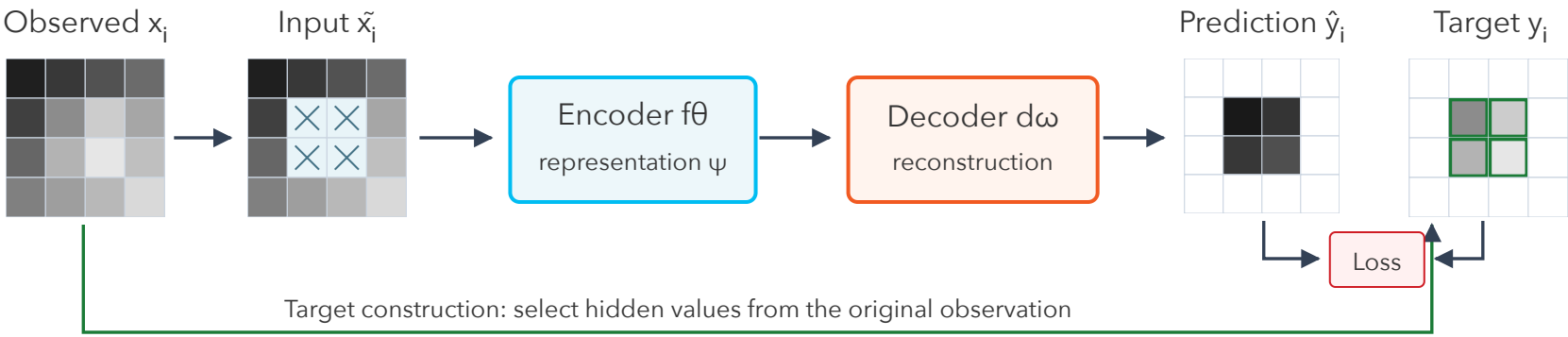


$$y_i = (x_{i,k})_{k \in \mathcal{M}}$$

Copy the withheld observed values into the target branch.

i identifies an observation · Θ contains the encoder and decoder parameters · M selects hidden locations

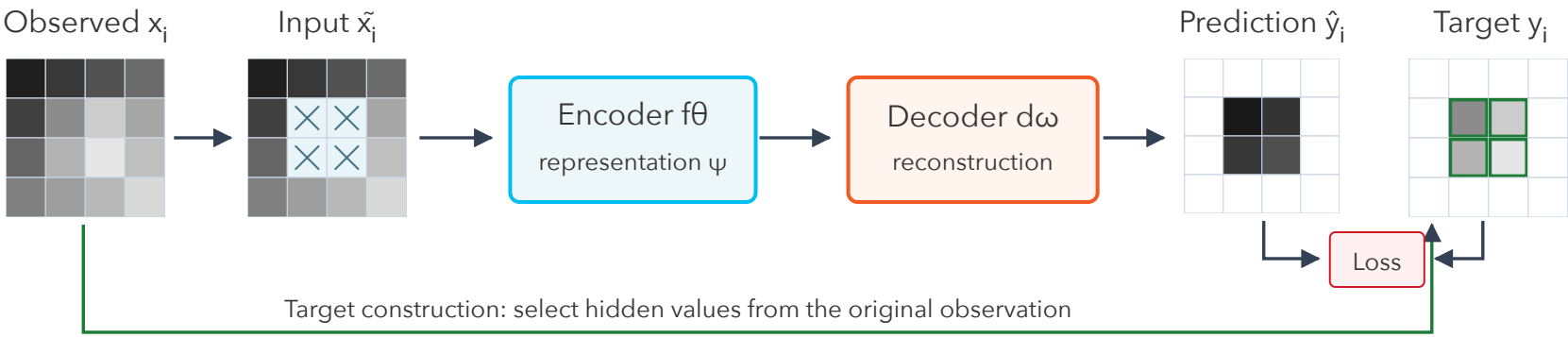
Masking: Loss and parameter update



$$\ell(\hat{y}_i, y_i) = \frac{1}{|\mathcal{M}|} \sum_{k \in \mathcal{M}} (\hat{y}_{i,k} - y_{i,k})^2$$

Evaluate the error only at the selected hidden locations.

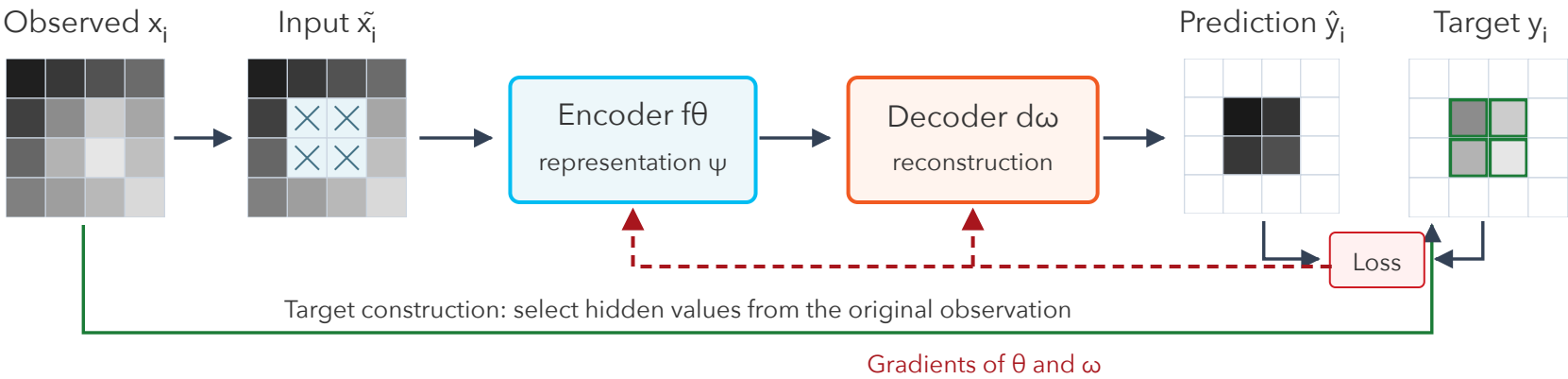
Masking: Loss and parameter update



$$\mathcal{L} = \frac{1}{B} \sum_{i=1}^{B} \ell(\hat{y}_i, y_i)$$

B is the number of observations. Spatial averaging happens inside each loss.

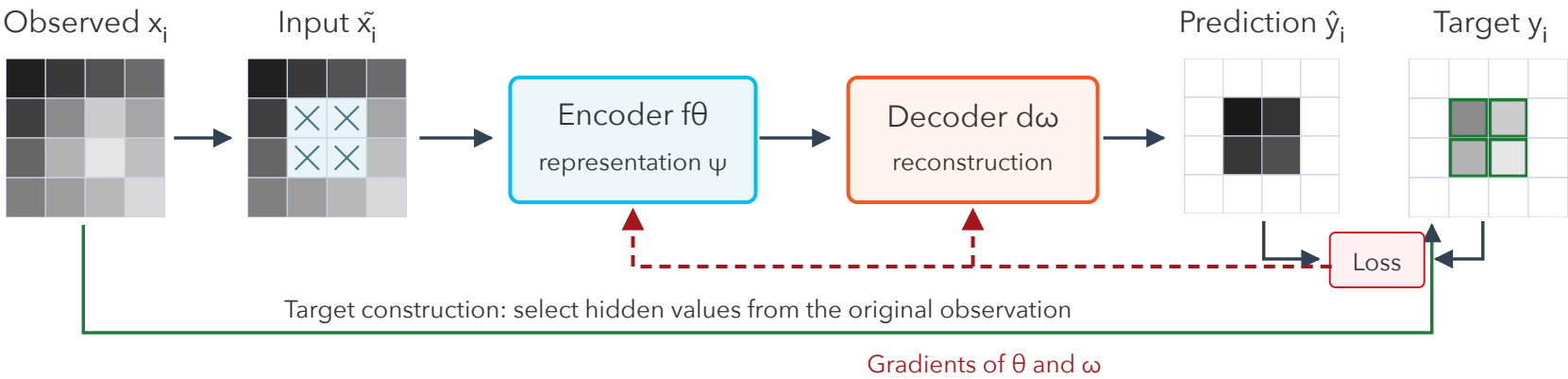
Masking: Loss and parameter update



$$\nabla_{\Theta}\mathcal{L} \qquad \Theta = (\theta, \omega)$$

Backpropagate to the encoder and decoder.

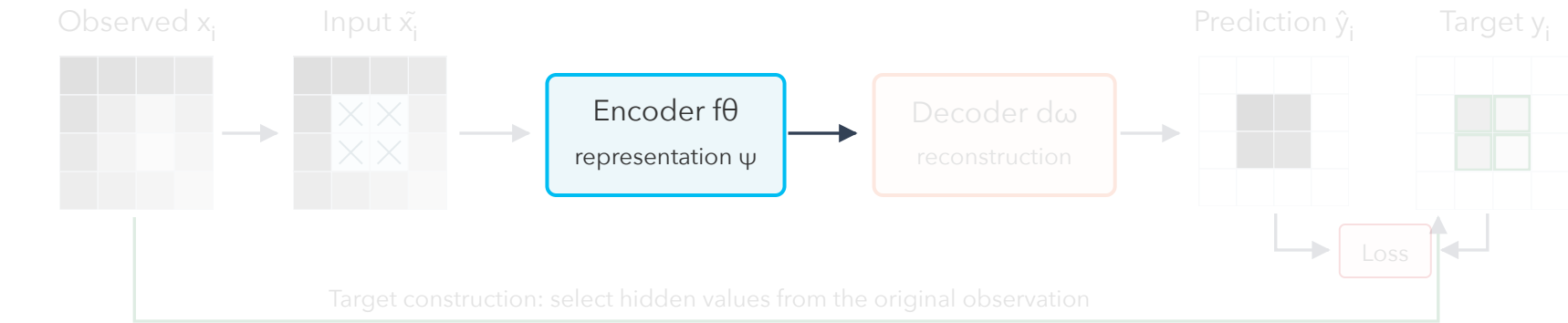
Masking: Loss and parameter update



$$\Theta_{n+1} = \Theta_n - \eta \nabla_\Theta \mathcal{L}$$

n counts optimization steps. η controls the size of each parameter update.

Masking: What remains after training



Keep fθ · replace dω with a task head when appropriate

$$\boldsymbol{\psi} = f_\theta(x) \qquad \hat{y}^{\text{task}} = h_\beta(\boldsymbol{\psi})$$

Use the encoder on a new observation. The task determines the output head.



→ **Encoder fθ** → **Task head hβ** → Task output

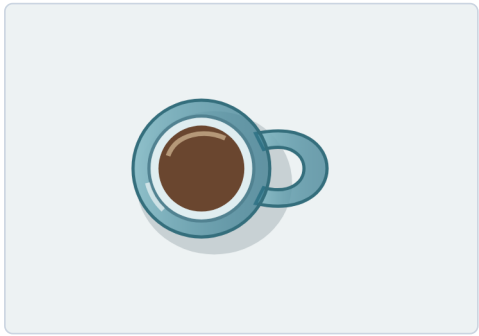
Evaluate the retained representation on the intended perception task.

SECTION 02

Representations for perception

Encoder representations

Observation x



Encoder fθ

learned parameters θ

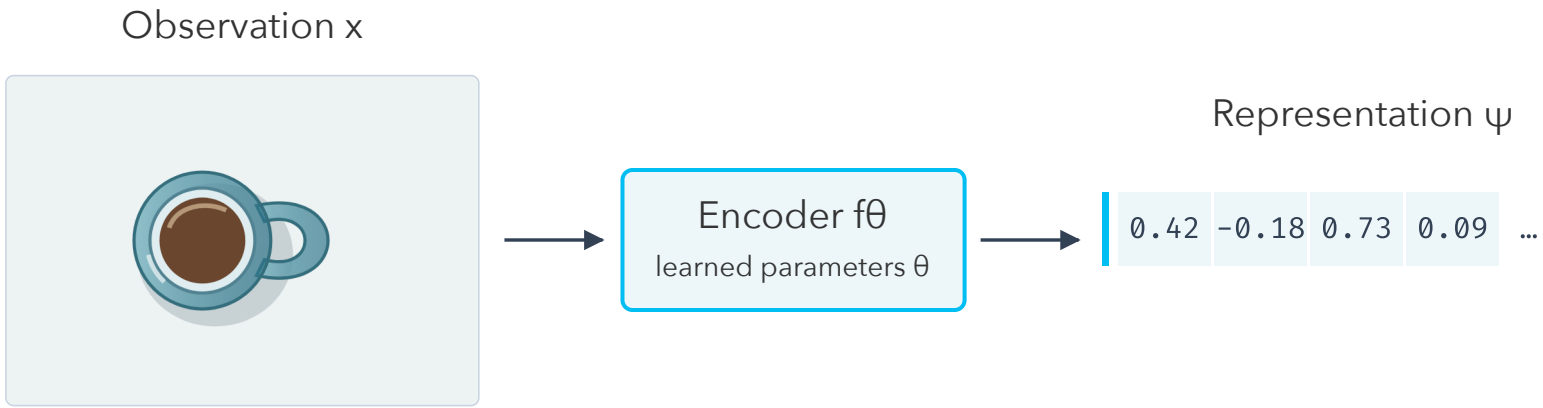
An internal description that a task can reuse

$$\psi = f_\theta(x)$$

A reusable description

Representation quality depends on the information available for the intended task.

Encoder representations



An internal description that a task can reuse

$$\psi = f_\theta(x)$$

A reusable description

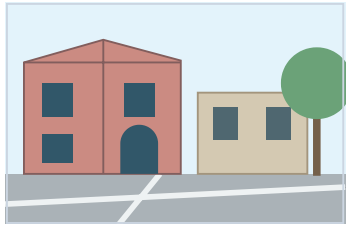
Representation quality depends on the information available for the intended task.

Global and dense features

Global features: one vector per image

Recognize a previously visited place

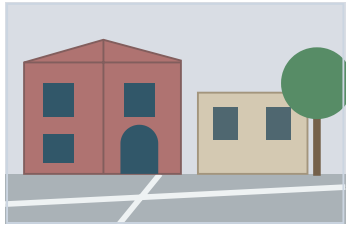
Query image



Encoder fθ ↓

`0.42` `-0.18` `0.73`

Stored image



Encoder fθ ↓

`0.40` `-0.20` `0.71`

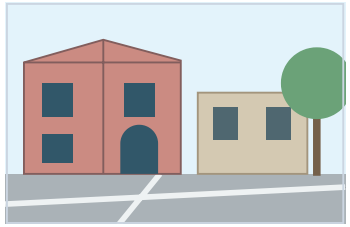
Compare whole-image descriptors

Global and dense features

Global features: one vector per image

Recognize a previously visited place

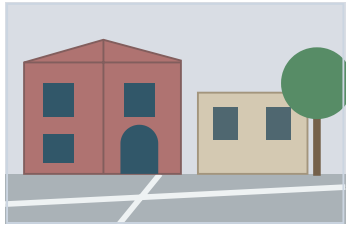
Query image



Encoder fθ ↓

`0.42` `-0.18` `0.73`

Stored image



Encoder fθ ↓

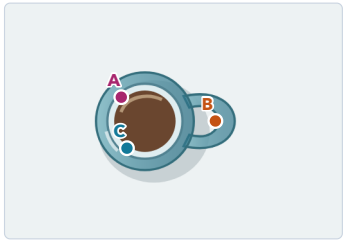
`0.40` `-0.20` `0.71`

Compare whole-image descriptors

Dense features: vectors at spatial locations

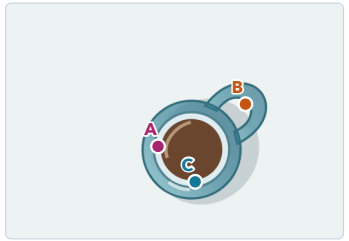
Find the corresponding handle point

Reference image



Encoder fθ ↓

New image



Encoder fθ ↓



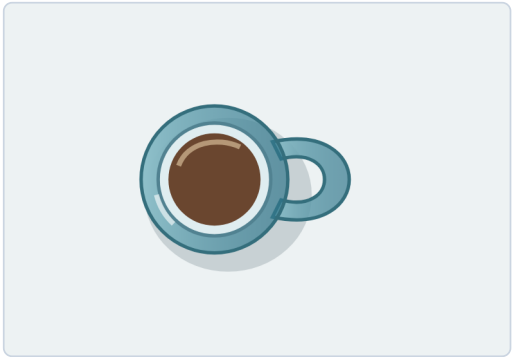
`-0.24` `0.81` `0.46`

Invariance to irrelevant changes

For **recognizing this mug**, lighting and background can be nuisance variables.

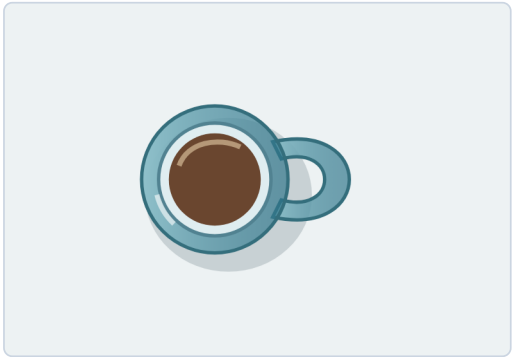
We therefore use **augmentations** during training.

Reference observation



change appearance

Changed observation



Desired identity output

same mug

Lighting: 1.00×

Background: plain

Pixels change; the recognition decision stays stable.

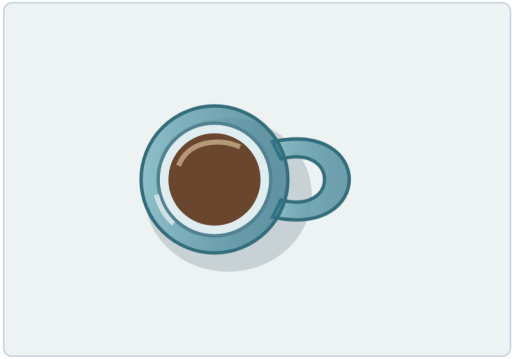
> Keep the information that identifies the object across the chosen changes.

Invariance to irrelevant changes

For **recognizing this mug**, lighting and background can be nuisance variables.

We therefore use **augmentations** during training.

Reference observation



change appearance

Changed observation



Desired identity output

same mug

Lighting: 0.55×

Background: plain

Pixels change; the recognition decision stays stable.

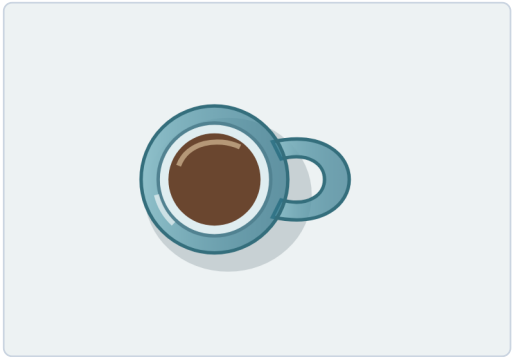
> Keep the information that identifies the object across the chosen changes.

Invariance to irrelevant changes

For **recognizing this mug**, lighting and background can be nuisance variables.

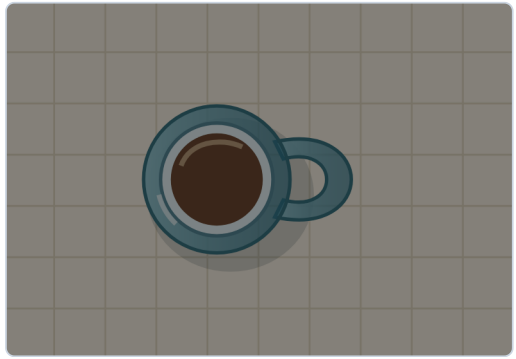
We therefore use **augmentations** during training.

Reference observation



change appearance

Changed observation



Desired identity output

same mug

Lighting: 0.55×

Background: pattern

Pixels change; the recognition decision stays stable.

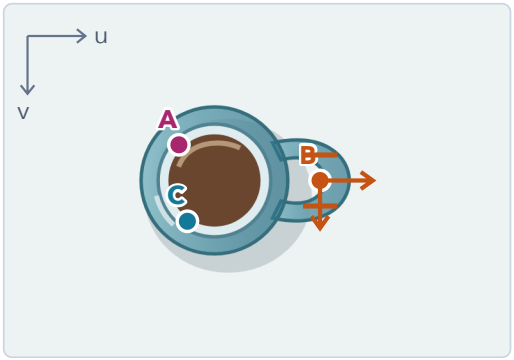
> Keep the information that identifies the object across the chosen changes.

Spatial information needed for action

For **grasping the handle**, its location and orientation must remain accessible.

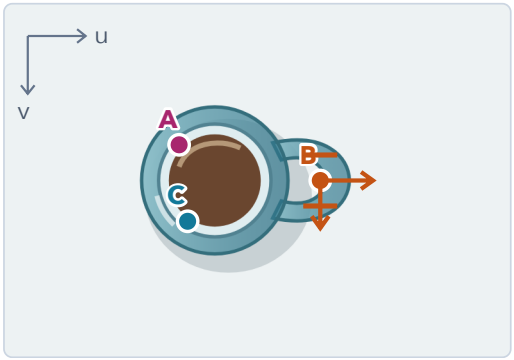
We therefore use **augmentations** during training.

Reference observation



move the mug

Changed observation



Grasp in image coordinates

(187.0, 105.0)

α = 0° · positive clockwise

B: same illustrative descriptor

```
-0.24  0.81  0.46
```

The handle descriptor stays associated with B; its image location and grasp orientation change.

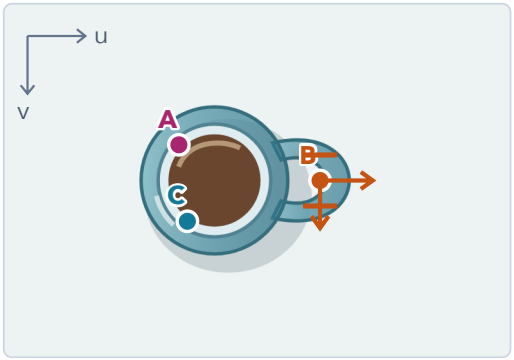
> Corresponding features can stay consistent while their spatial locations change.

Spatial information needed for action

For **grasping the handle**, its location and orientation must remain accessible.

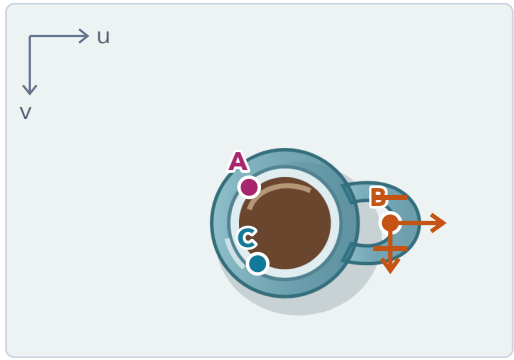
We therefore use **augmentations** during training.

Reference observation



move the mug

Changed observation



Grasp in image coordinates

(227.0, 130.0)

α = 0° · positive clockwise

B: same illustrative descriptor

```
-0.24  0.81  0.46
```

The handle descriptor stays associated with B; its image location and grasp orientation change.

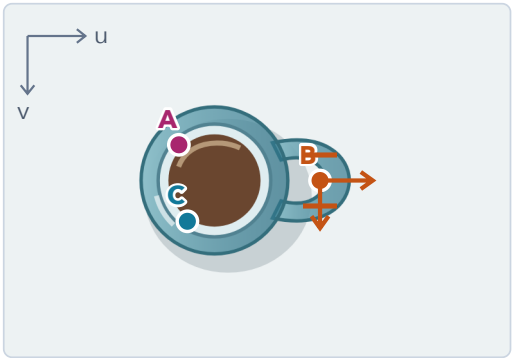
> Corresponding features can stay consistent while their spatial locations change.

Spatial information needed for action

For **grasping the handle**, its location and orientation must remain accessible.

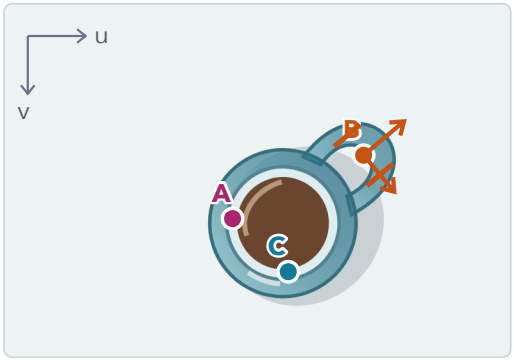
We therefore use **augmentations** during training.

Reference observation



move the mug

Changed observation



Grasp in image coordinates

(212.5, 90.1)

α = -40° · positive clockwise

B: same illustrative descriptor

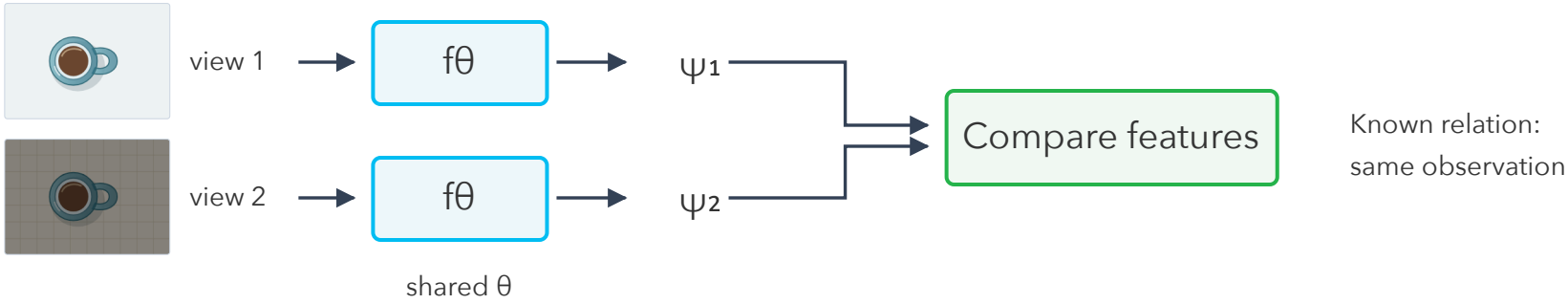
```
-0.24  0.81  0.46
```

The handle descriptor stays associated with B; its image location and grasp orientation change.

> Corresponding features can stay consistent while their spatial locations change.

Matching and prediction objectives

Match related observations



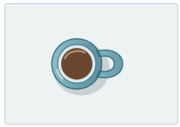
Two mechanisms to follow

Matching uses a known relation between observations.

Masking uses observed information withheld from the input.

Matching and prediction objectives

Match related observations



view 1 → fθ → ψ1



view 2 → fθ → ψ2

shared θ

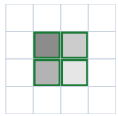
ψ1, ψ2 → Compare features

Known relation: same observation

Predict hidden information



input $\tilde{x}$ → fθ → ψ → dω → $\hat{y}$ → Loss ← Target y



Two mechanisms to follow

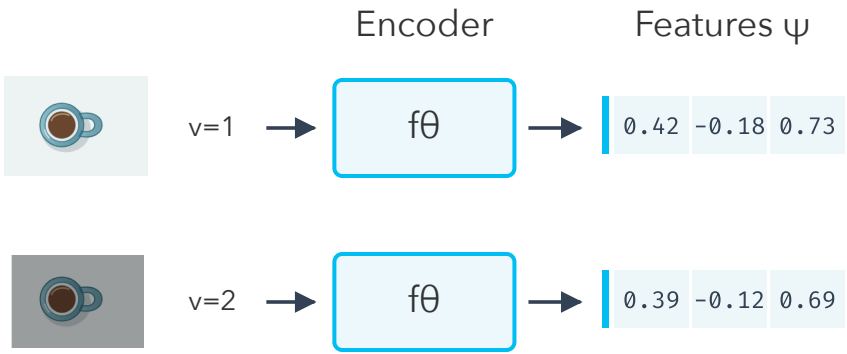
Matching uses a known relation between observations.

Masking uses observed information withheld from the input.

SECTION 03

Matching, collapse, and self-distillation

Encoder and projection head

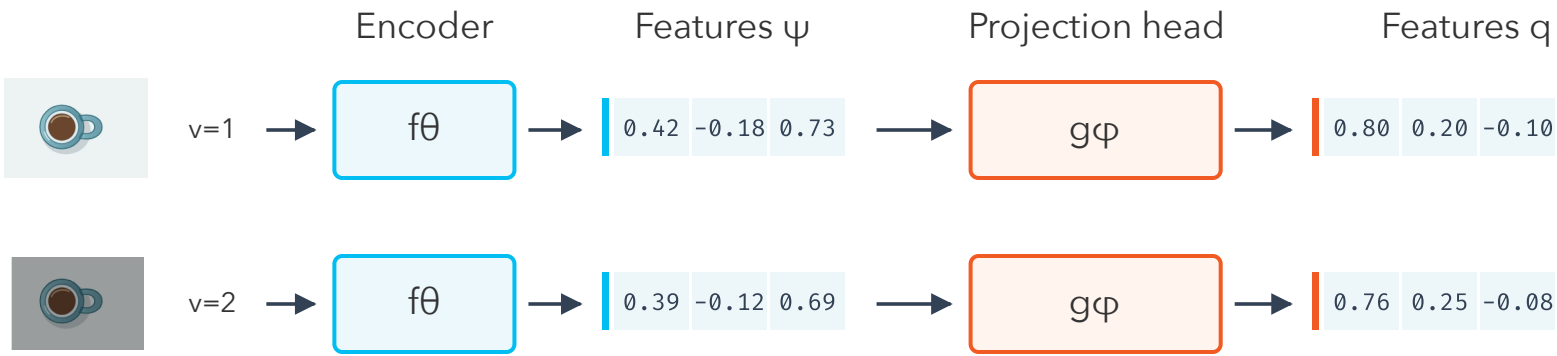


Reuse the encoder representation

$$\boldsymbol{\psi}_i^{(v)} = f_\theta(\tilde{\boldsymbol{x}}_i^{(v)})$$

The objective compares q. Many different downstream tasks can reuse encoder features ψ.

Encoder and projection head



Reuse the encoder representation

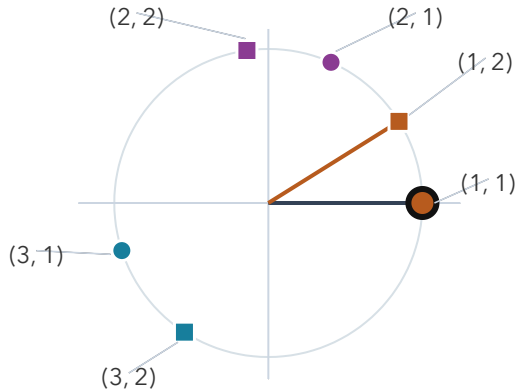
Compare projected features

$$\boldsymbol{\psi}_i^{(v)} = f_\theta(\tilde{\boldsymbol{x}}_i^{(v)}) \qquad \mathbf{q}_i^{(v)} = g_\phi(\boldsymbol{\psi}_i^{(v)})$$

The objective compares q. Many different downstream tasks can reuse encoder features ψ.

Similarity and temperature

Anchor (1, 1) · Positive (1, 2)

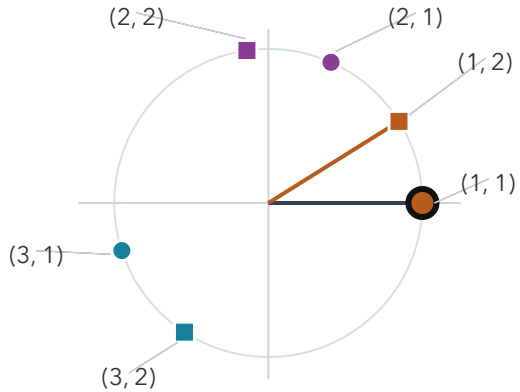


Candidate (i, v)	cosine	Normalized probability
(1, 2) · positive	0.848	60.9%
(2, 1) · negative	0.407	25.2%
(2, 2) · negative	-0.139	8.5%
(3, 1) · negative	-0.951	1.7%
(3, 2) · negative	-0.545	3.8%

$\tau = 0.50$ · positive probability = 60.9% · $\ell = 0.496$

Similarity and temperature

Anchor (1, 1) · Positive (1, 2)



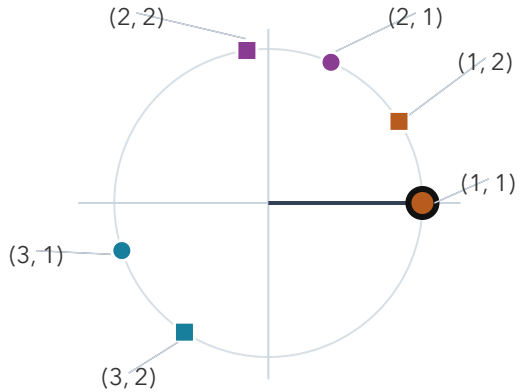
Candidate (i, v)	cosine	Normalized probability
(1, 2) · positive	0.848	60.9%
(2, 1) · negative	0.407	25.2%
(2, 2) · negative	-0.139	8.5%
(3, 1) · negative	-0.951	1.7%
(3, 2) · negative	-0.545	3.8%

τ = 0.50 · positive probability = 60.9% · ℓ = 0.496

$$\mathrm{sim}(\mathbf{q}_a, \mathbf{q}_b) = \frac{\mathbf{q}_a^\top \mathbf{q}_b}{\|\mathbf{q}_a\|_2 \|\mathbf{q}_b\|_2}$$

Contrastive loss

Anchor (1, 1) · Positive (1, 2)



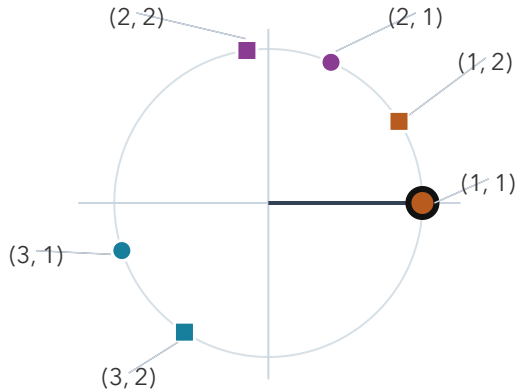
Candidate (i, v)	cosine	Normalized probability
(1, 2) · positive	0.848	60.9%
(2, 1) · negative	0.407	25.2%
(2, 2) · negative	-0.139	8.5%
(3, 1) · negative	-0.951	1.7%
(3, 2) · negative	-0.545	3.8%

τ = 0.50 · positive probability = 60.9% · ℓ = 0.496

$$w_+ = \exp(\mathrm{sim}(\mathbf{q}_i^{(1)}, \mathbf{q}_i^{(2)})/\tau)$$

Contrastive loss

Anchor (1, 1) · Positive (1, 2)



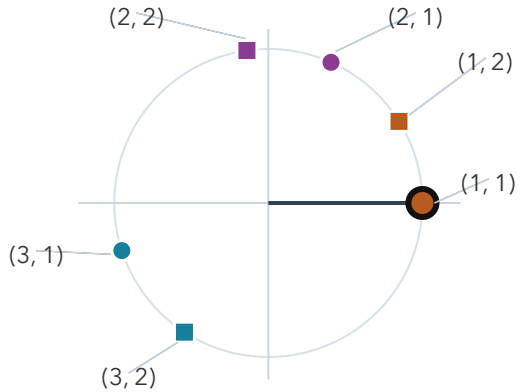
Candidate (i, v)	cosine	Normalized probability
(1, 2) · positive	0.848	60.9%
(2, 1) · negative	0.407	25.2%
(2, 2) · negative	-0.139	8.5%
(3, 1) · negative	-0.951	1.7%
(3, 2) · negative	-0.545	3.8%

τ = 0.50 · positive probability = 60.9% · ℓ = 0.496

$$L = \sum_{(j,v)\neq(i,1)} \exp(\mathrm{sim}(\mathbf{q}_i^{(1)}, \mathbf{q}_j^{(v)})/\tau)$$

Contrastive loss

Anchor (1, 1) · Positive (1, 2)



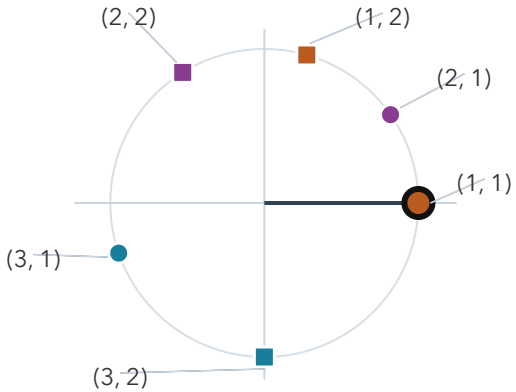
Candidate (i, v)	cosine	Normalized probability
(1, 2) · positive	0.848	60.9%
(2, 1) · negative	0.407	25.2%
(2, 2) · negative	-0.139	8.5%
(3, 1) · negative	-0.951	1.7%
(3, 2) · negative	-0.545	3.8%

τ = 0.50 · positive probability = 60.9% · ℓ = 0.496

$$\ell_i^{(1)} = -\log \frac{\exp(\mathrm{sim}(\mathbf{q}_i^{(1)}, \mathbf{q}_i^{(2)})/\tau)}{\sum_{(j,v)\neq(i,1)} \exp(\mathrm{sim}(\mathbf{q}_i^{(1)}, \mathbf{q}_j^{(v)})/\tau)}$$

How the objective changes representations

Augmented view → **fθ** → ψ → **gφ** → q



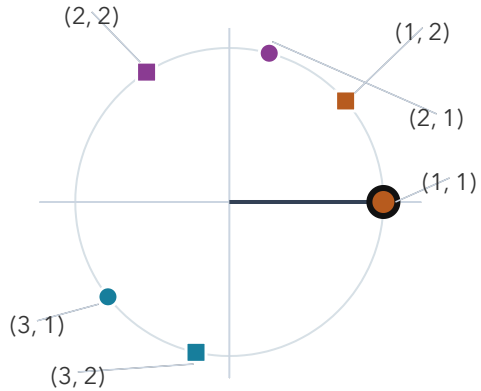
Candidate (i, v)	cosine	Normalized probability
(1, 2) · positive	0.276	20.7%
(2, 1) · negative	0.819	61.4%
(2, 2) · negative	-0.530	4.1%
(3, 1) · negative	-0.946	1.8%
(3, 2) · negative	-0.000	11.9%

τ = 0.50 · positive probability = 20.7% · ℓ = 1.574 · mean over six anchors = 1.493

Increase agreement with the positive relative to the competing views.

How the objective changes representations

Augmented view → **fθ** → ψ → **gφ** → q



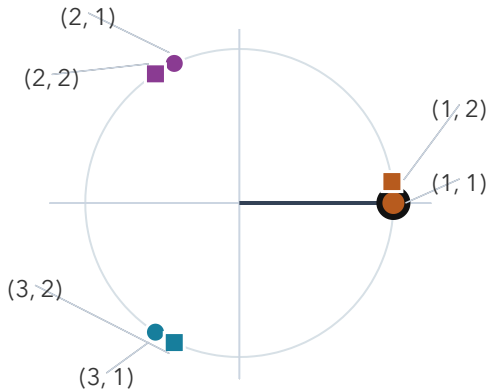
Candidate (i, v)	cosine	Normalized probability
(1, 2) · positive	0.755	61.1%
(2, 1) · negative	0.259	22.7%
(2, 2) · negative	-0.537	4.6%
(3, 1) · negative	-0.788	2.8%
(3, 2) · negative	-0.216	8.8%

τ = 0.50 · positive probability = 61.1% · ℓ = 0.492 · mean over six anchors = 0.585

Increase agreement with the positive relative to the competing views.

How the objective changes representations

Augmented view → **fθ** → ψ → **gφ** → q

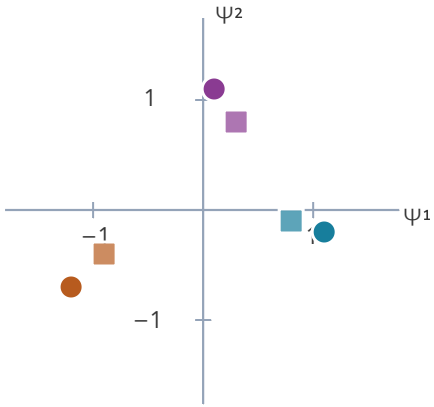


Candidate (i, v)	cosine	Normalized probability
(1, 2) · positive	0.990	82.6%
(2, 1) · negative	-0.423	4.9%
(2, 2) · negative	-0.545	3.8%
(3, 1) · negative	-0.545	3.8%
(3, 2) · negative	-0.423	4.9%

τ = 0.50 · positive probability = 82.6% · ℓ = 0.192 · mean over six anchors = 0.189

Increase agreement with the positive relative to the competing views.

Constant representation: Collapse



Three distinct pairs

Mean pair-agreement loss: **0.1367**

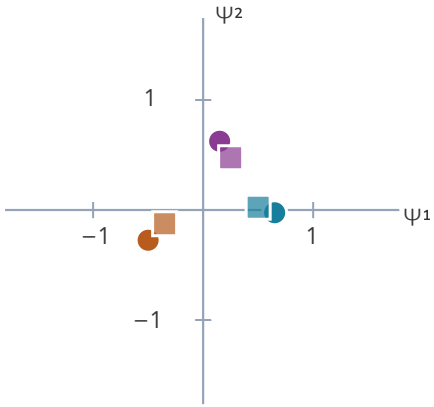
Feature spread: (0.916, 0.708)

Bringing views of the same pair closer does not necessarily preserve differences between pairs.

$$\ell_{\text{agree},i} = \|\boldsymbol{\psi}_i^{(1)} - \boldsymbol{\psi}_i^{(2)}\|_2^2$$

The learning objective must avoid collapse of the representation as a possible optimal solution.

Constant representation: Collapse



Three distinct pairs

Mean pair-agreement loss: **0.0342**

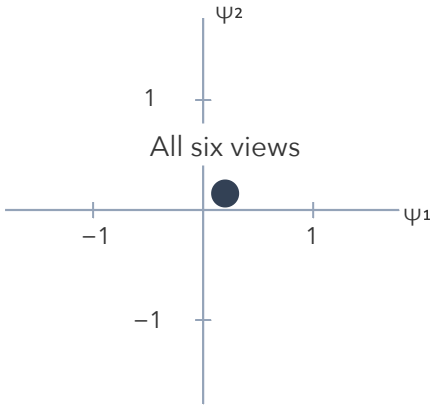
Feature spread: (0.458, 0.354)

Bringing views of the same pair closer does not necessarily preserve differences between pairs.

$$\ell_{\text{agree},i} = \|\boldsymbol{\psi}_i^{(1)} - \boldsymbol{\psi}_i^{(2)}\|_2^2$$

The learning objective must avoid collapse of the representation as a possible optimal solution.

Constant representation: Collapse



Three distinct pairs

Mean pair-agreement loss: **0.0000**

Feature spread: (0.000, 0.000)

One constant vector erases distinctions between observations.

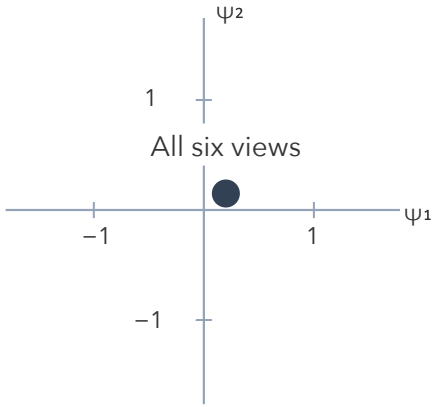
c = (0.20, 0.15)

$$f_\theta(x) = \mathbf{c} \text{ for every } x \quad \Longrightarrow \quad \ell_{\text{agree},i} = 0$$

The learning objective must avoid collapse of the representation as a possible optimal solution.

Loss of feature dimensions

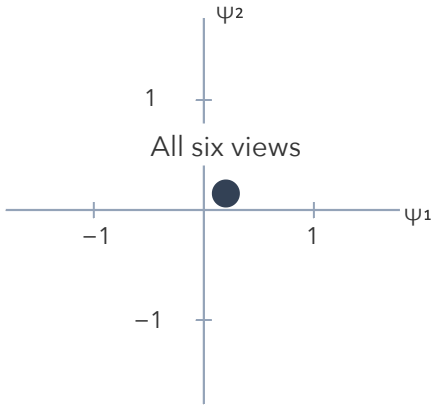
Point: complete collapse



Std: (0.00, 0.00)

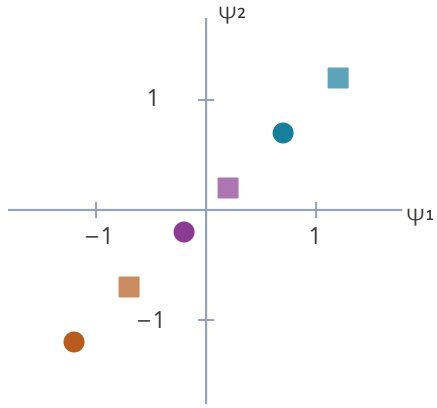
Loss of feature dimensions

Point: complete collapse



Std: (0.00, 0.00)

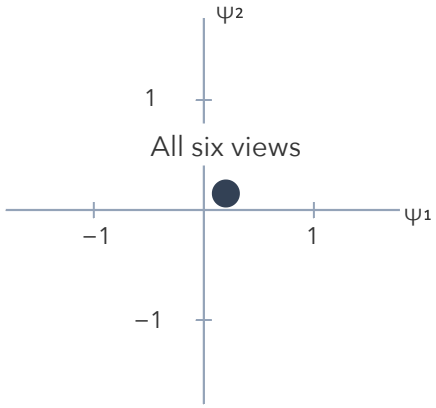
Line: one varying direction



Std: (0.89, 0.89)

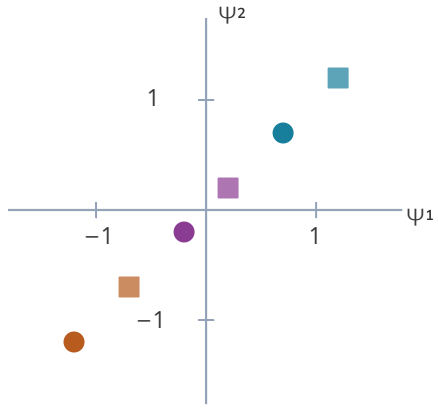
Loss of feature dimensions

Point: complete collapse



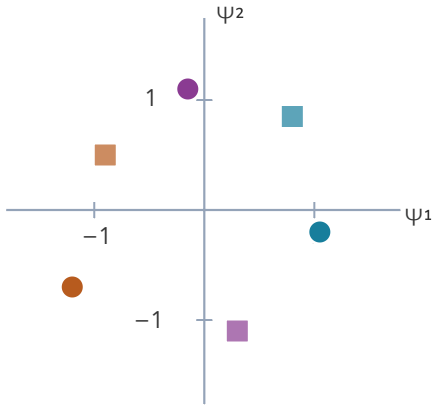
Std: (0.00, 0.00)

Line: one varying direction



Std: (0.89, 0.89)

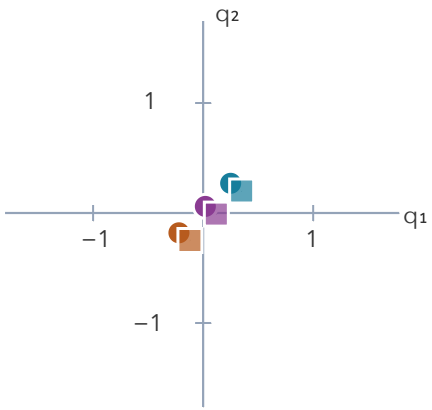
Two varying directions



Std: (0.91, 0.88)

Regularization: Variance-Invariance-Covariance (VICReg)

View → fθ → ψ → **gφ (expander)** → q · regularize q in both branches



Invariance · 0.0149

Distance between corresponding views

Variance · 1.5394

Per-dimension standard-deviation floor

Covariance · 0.0056

Squared off-diagonal covariance

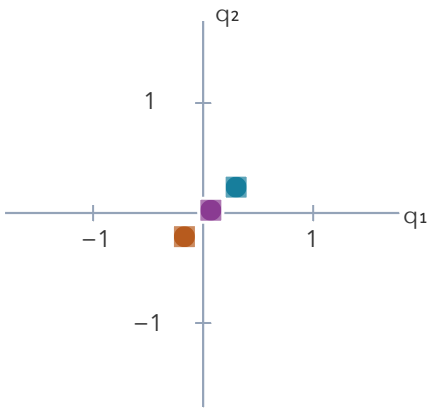
$$\mathcal{L}_{\text{VICReg}} = \lambda_{\text{inv}}\mathcal{L}_{\text{inv}} + \lambda_{\text{var}}\mathcal{L}_{\text{var}} + \lambda_{\text{cov}}\mathcal{L}_{\text{cov}}$$

Start with mismatched, nearly collapsed views.

Bardes et al. · VICReg · ICLR 2022

Regularization: Variance-Invariance-Covariance (VICReg)

View → fθ → ψ → **gφ (expander)** → q · regularize q in both branches



Invariance · 0.0000

Distance between corresponding views

Variance · 1.5394

Per-dimension standard-deviation floor

Covariance · 0.0056

Squared off-diagonal covariance

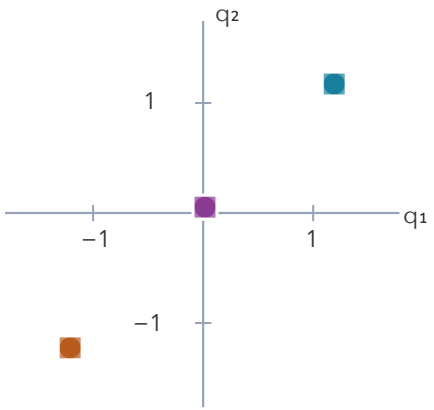
$$\mathcal{L}_{\text{VICReg}} = \lambda_{\text{inv}}\mathcal{L}_{\text{inv}} + \lambda_{\text{var}}\mathcal{L}_{\text{var}} + \lambda_{\text{cov}}\mathcal{L}_{\text{cov}}$$

Invariance brings corresponding views together.

Bardes et al. · VICReg · ICLR 2022

Regularization: Variance-Invariance-Covariance (VICReg)

View → fθ → ψ → **gφ (expander)** → q · regularize q in both branches



Invariance · 0.0000

Distance between corresponding views

Variance · 0.0000

Per-dimension standard-deviation floor

Covariance · 4.1444

Squared off-diagonal covariance

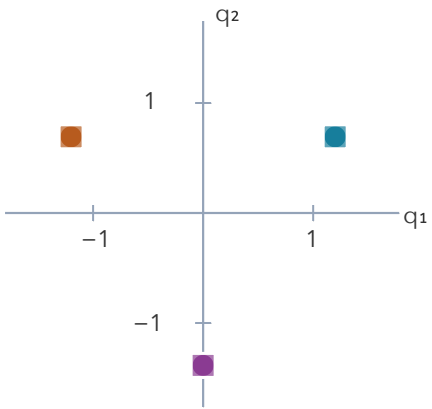
$$\mathcal{L}_{\text{VICReg}} = \lambda_{\text{inv}}\mathcal{L}_{\text{inv}} + \lambda_{\text{var}}\mathcal{L}_{\text{var}} + \lambda_{\text{cov}}\mathcal{L}_{\text{cov}}$$

Variance encourages a minimum spread in each dimension.

Bardes et al. · VICReg · ICLR 2022

Regularization: Variance-Invariance-Covariance (VICReg)

View → fθ → ψ → **gφ (expander)** → q · regularize q in both branches



Invariance · 0.0000

Distance between corresponding views

Variance · 0.0000

Per-dimension standard-deviation floor

Covariance · 0.0000

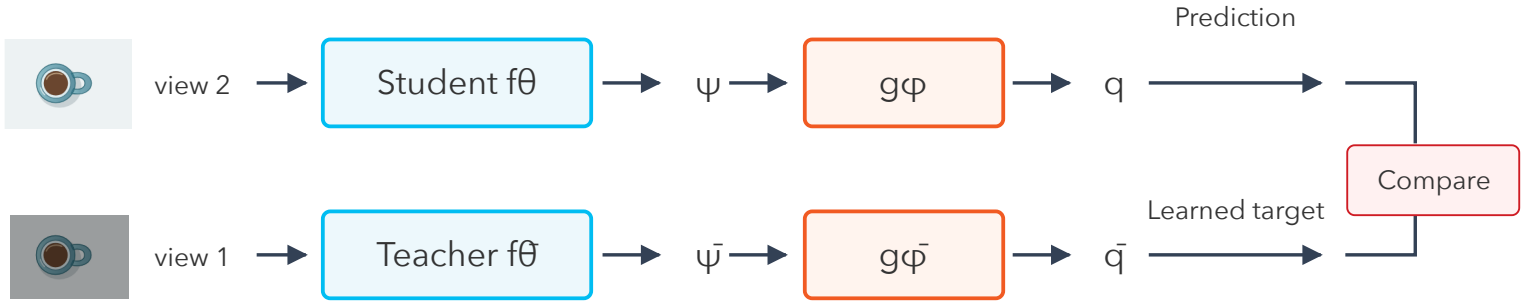
Squared off-diagonal covariance

$$\mathcal{L}_{\text{VICReg}} = \lambda_{\text{inv}}\mathcal{L}_{\text{inv}} + \lambda_{\text{var}}\mathcal{L}_{\text{var}} + \lambda_{\text{cov}}\mathcal{L}_{\text{cov}}$$

Covariance discourages redundant feature directions.

Bardes et al. · VICReg · ICLR 2022

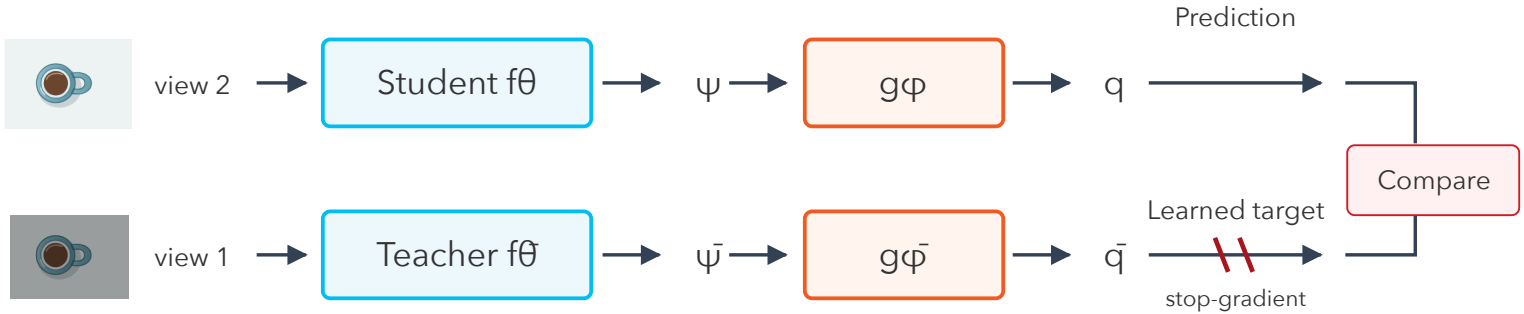
Self-distillation: Learning targets from a teacher



A target that evolves during learning

The student predicts a teacher output from a different view of the observation.

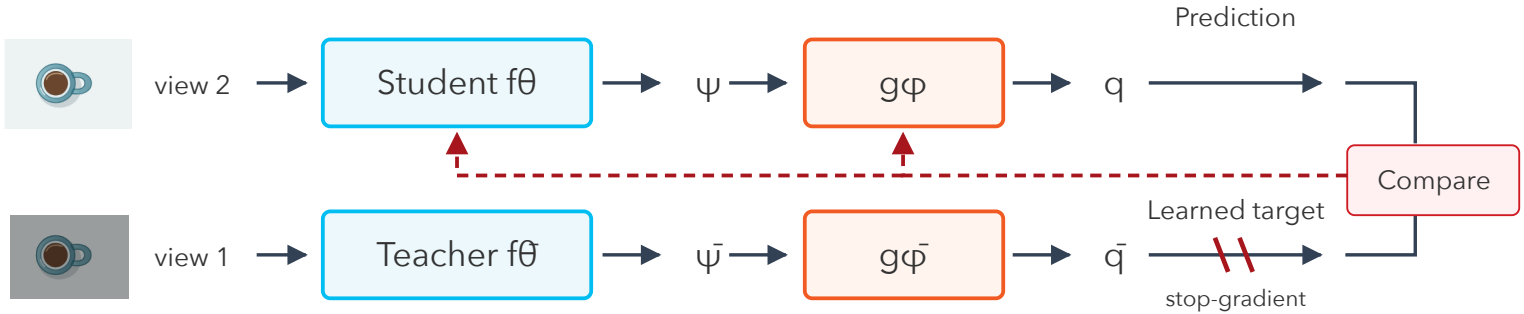
Self-distillation: Learning targets from a teacher



A target that evolves during learning

The student predicts a teacher output from a different view of the observation.

Self-distillation: Learning targets from a teacher

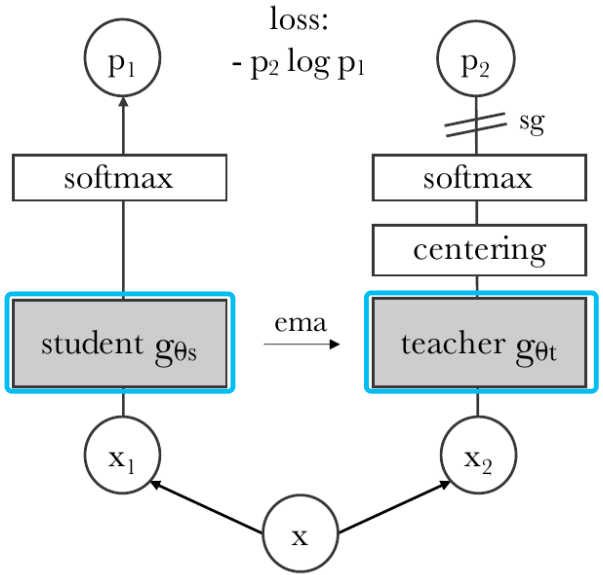


Dashed: gradients update θ and φ · no teacher gradient

A target that evolves during learning

The student predicts a teacher output from a different view of the observation.

DINO architecture



Caron et al. · DINO · ICCV 2021 · main-paper Fig. 2

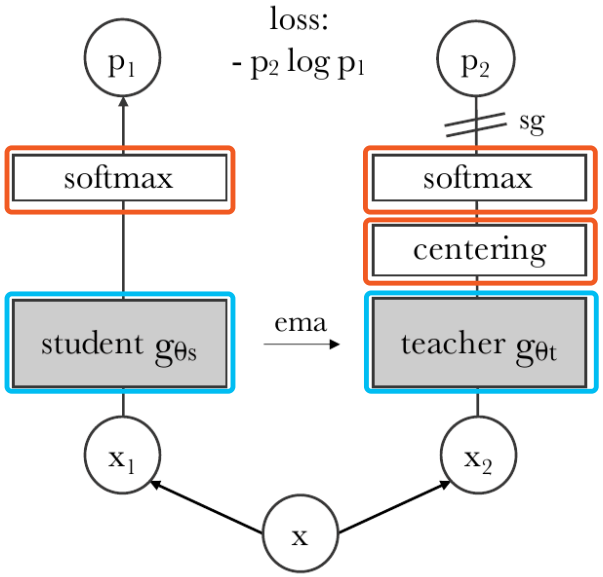
Student and teacher include encoder + head

$$g_{\theta_s} \leftrightarrow g_\phi \circ f_\theta$$

$$g_{\theta_t} \leftrightarrow g_{\bar{\phi}} \circ f_{\bar{\theta}}$$

Match teacher and student distributions; update their parameters by different rules.

DINO architecture



Caron et al. · DINO · ICCV 2021 · main-paper Fig. 2

Student and teacher include encoder + head

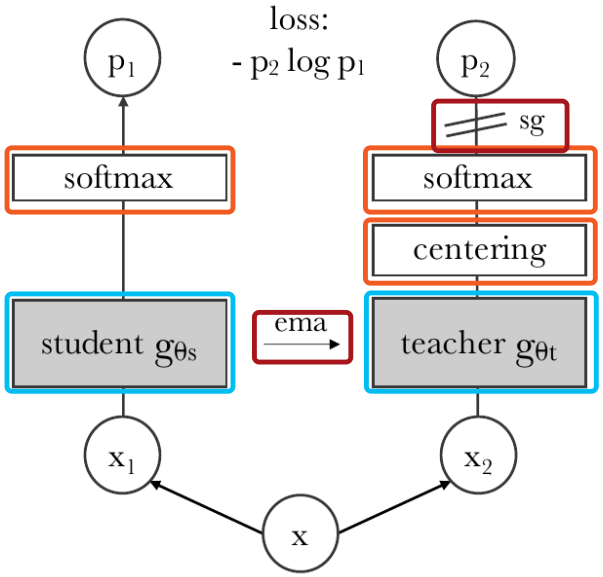
$$g_{\theta_s} \leftrightarrow g_\phi \circ f_\theta$$

$$g_{\theta_t} \leftrightarrow g_{\bar{\phi}} \circ f_{\bar{\theta}}$$

Center the teacher outputs, then apply separate softmax temperatures.

Match teacher and student distributions; update their parameters by different rules.

DINO architecture



Caron et al. · DINO · ICCV 2021 · main-paper Fig. 2

Student and teacher include encoder + head

$$g_{\theta_s} \leftrightarrow g_\phi \circ f_\theta$$

$$g_{\theta_t} \leftrightarrow g_{\bar{\phi}} \circ f_{\bar{\theta}}$$

Center the teacher outputs, then apply separate softmax temperatures.

Stop the teacher gradient; update its parameters by a moving average.

Match teacher and student distributions; update their parameters by different rules.

Distribution matching in DINO

Student · view 2

Head outputs (example)

```
0.18  0.03  -0.08
```

Teacher · view 1 · stop-gradient

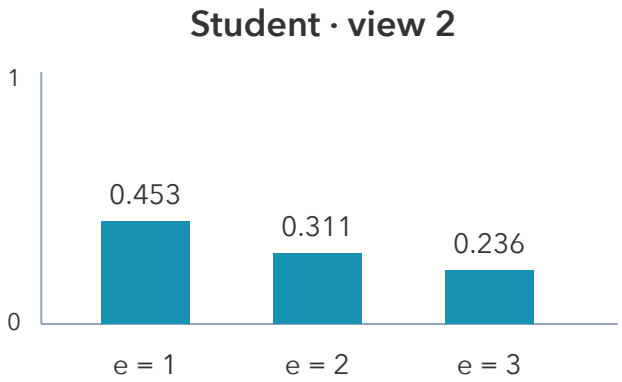
Head outputs (example)

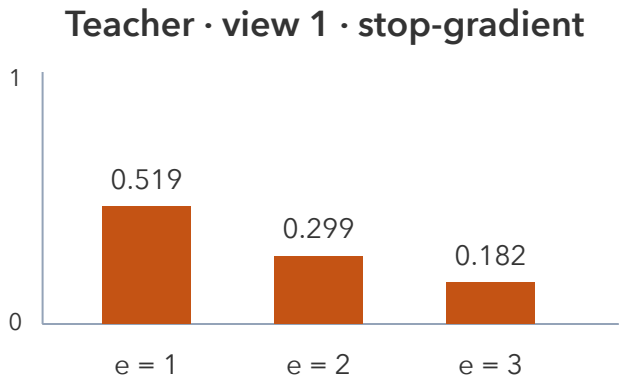
```
0.25  0.11  -0.02
```

Center m = (0.05, 0.02, −0.01)

$$\mathbf{q}_i^{(2)} = g_\phi(f_\theta(\tilde{x}_i^{(2)})) \qquad \bar{\mathbf{q}}_i^{(1)} = g_{\bar{\phi}}(f_{\bar{\theta}}(\tilde{x}_i^{(1)}))$$

Distribution matching in DINO

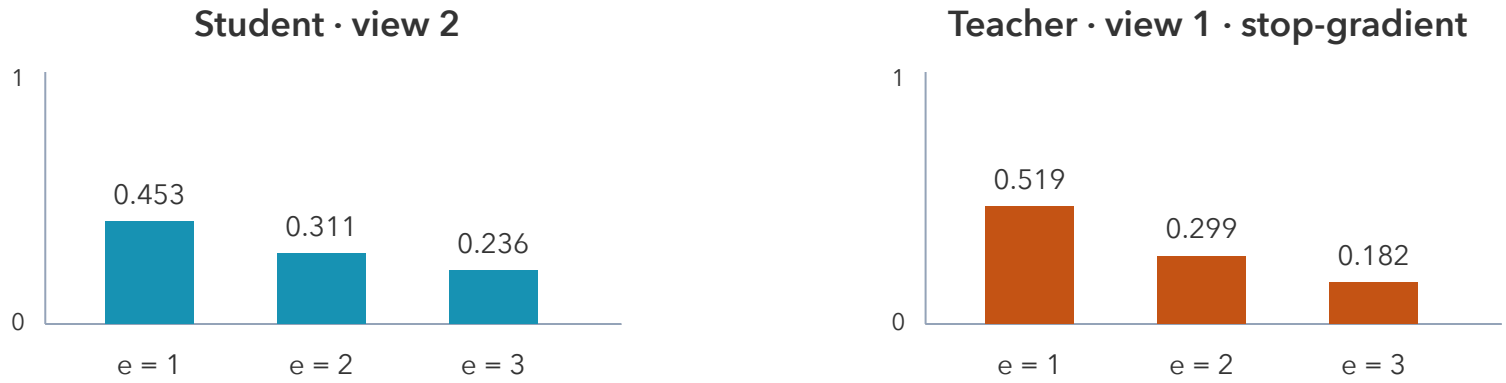




$$\boldsymbol{\rho}_{\text{stu}} = \text{softmax}(\mathbf{q}_i^{(2)} / \tau_{\text{stu}})$$

$$\boldsymbol{\rho}_{\text{teach}} = \text{sg}\Big(\text{softmax}((\bar{\mathbf{q}}_i^{(1)} - \mathbf{m}) / \tau_{\text{teach}})\Big)$$

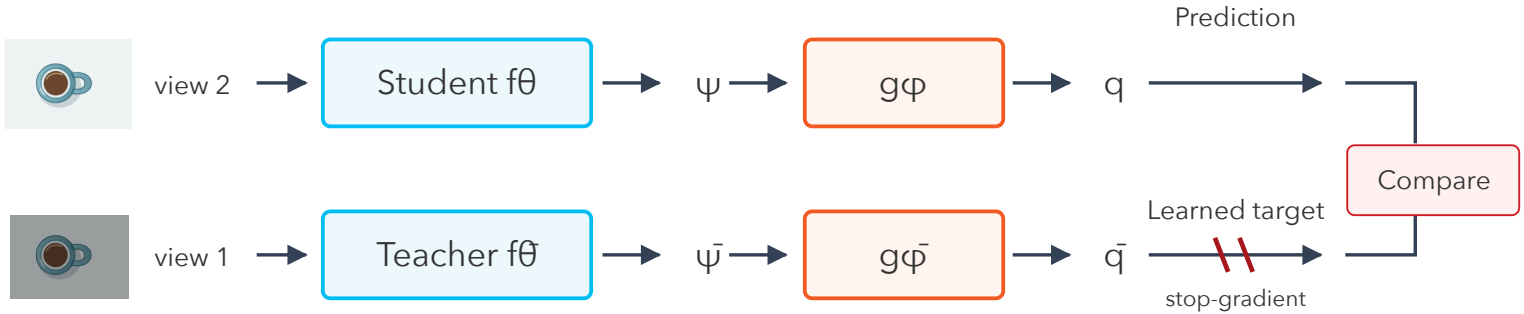
Distribution matching in DINO



$$\ell_{\text{DINO}} = -\sum_{e} \rho_{\text{teach},e} \log \rho_{\text{stu},e}$$

Computed one-view-pair cross-entropy: **1.0230**

Gradient updates and moving averages

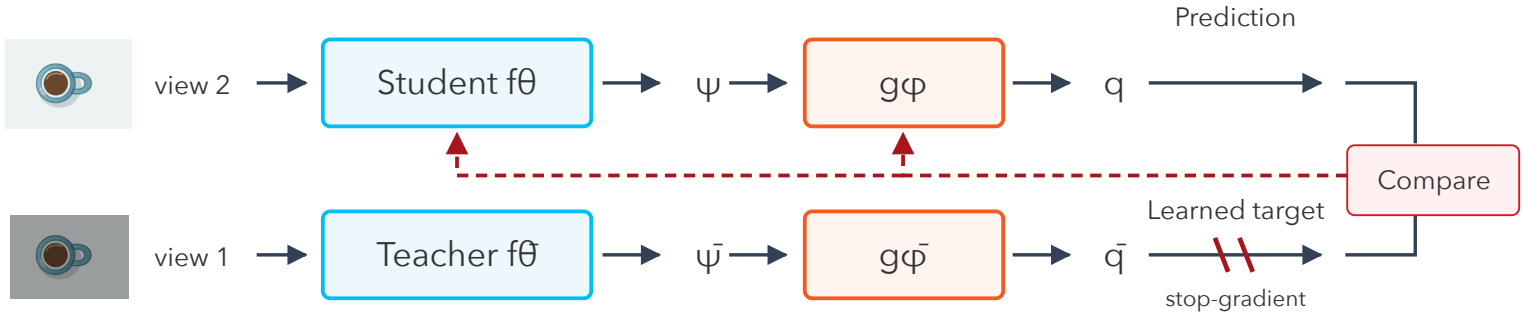


$$\Theta = (\theta, \phi) \qquad \bar{\Theta} = (\bar{\theta}, \bar{\phi})$$

Encoder: θ = 0.450 · θ̄ = 0.600 Head: φ = 0.800 · φ̄ = 1.000

Initially, the teacher parameters match the student's parameters.

Gradient updates and moving averages



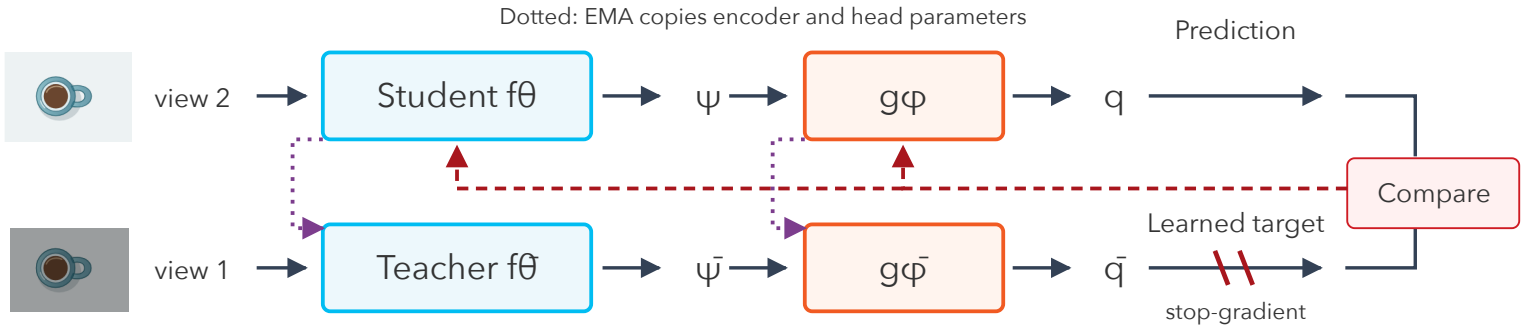
Dashed: gradients update θ and φ · no teacher gradient

$$\Theta_{n+1} = \Theta_n - \eta \nabla_\Theta \mathcal{L}$$

Encoder: θ = 0.488 · θ̄ = 0.600 Head: φ = 0.830 · φ̄ = 1.000

Computed student step: fixed-target cross-entropy 0.7281 → 0.6997.

Gradient updates and moving averages



Dashed: gradients update θ and φ · no teacher gradient

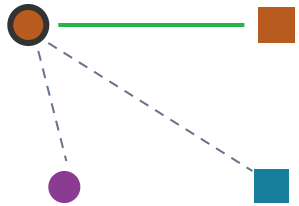
$$\bar{\theta}_{n+1} = \mu\bar{\theta}_n + (1-\mu)\theta_{n+1}$$

Encoder: θ = 0.488 · θ̄ = 0.578 Head: φ = 0.830 · φ̄ = 0.966

After a batch: update the teacher head φ̄ using a moving average.

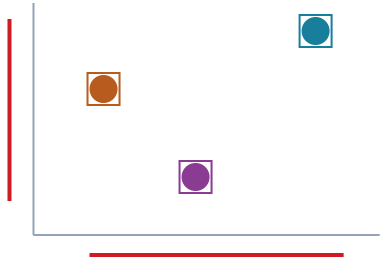
Summary of representation learning

SimCLR



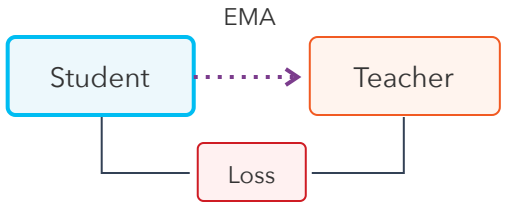
Competing views

VICReg



Spread + decorrelation

DINO



Centered, sharpened target

Teacher distribution

Observation → encoder → features → constructed relation or target → loss → parameter update

The goal of a good representation

Keep the information useful for the intended task.

Ignore the rest.

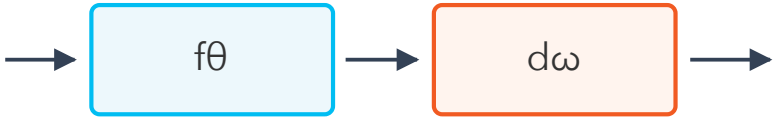
SECTION 04

Reconstruction and feature prediction

Reconstruction as a training signal

Observed image x





Predicted pixels

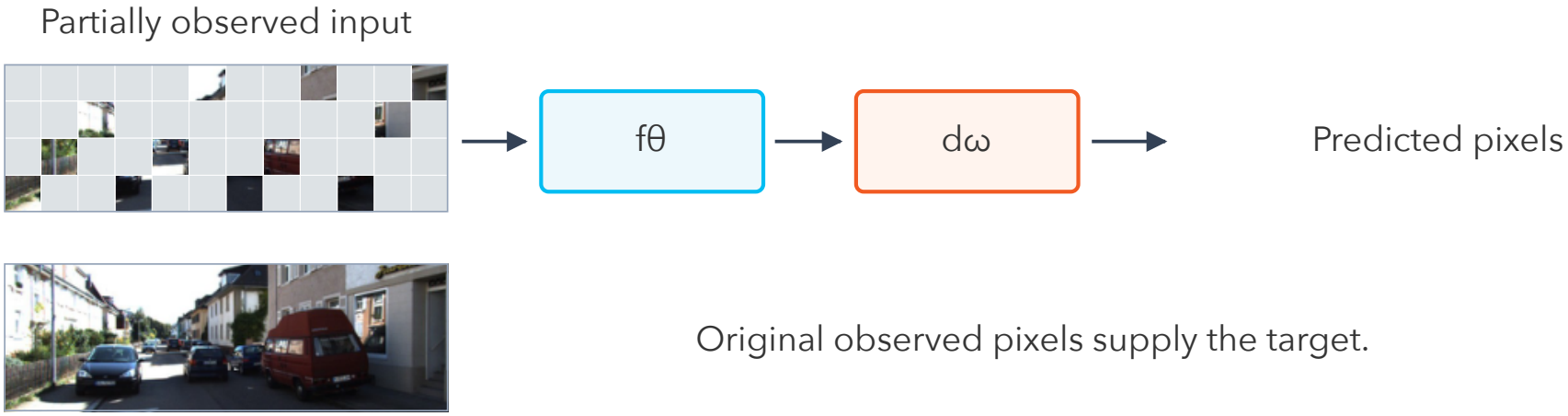


Original observed pixels supply the target.

A direct copy can solve reconstruction without learning useful features.

> The original observation supplies a target without a human annotation.

Reconstruction as a training signal

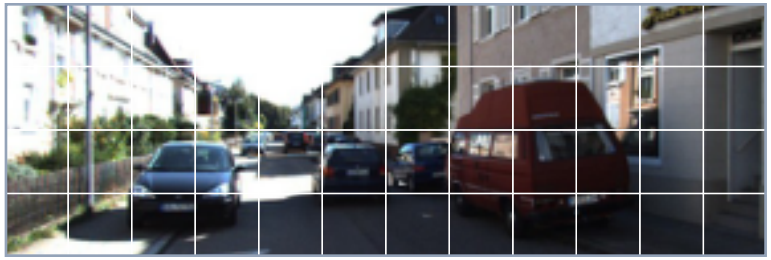


Withhold information from the prediction input; keep it for the target.

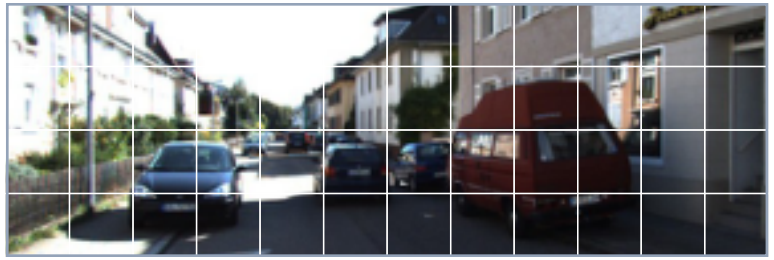
> The original observation supplies a target without a human annotation.

Visible and masked patches

Original observation x



Patchify: 48 square patches

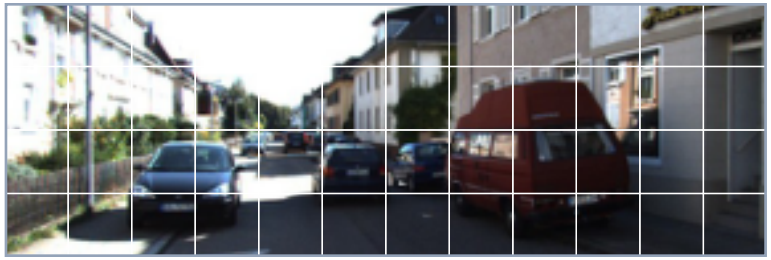


Patch index k identifies a location in the original image.

The training procedure still has every original patch value.

Visible and masked patches

Original observation x



Visible patches V · hidden patches M

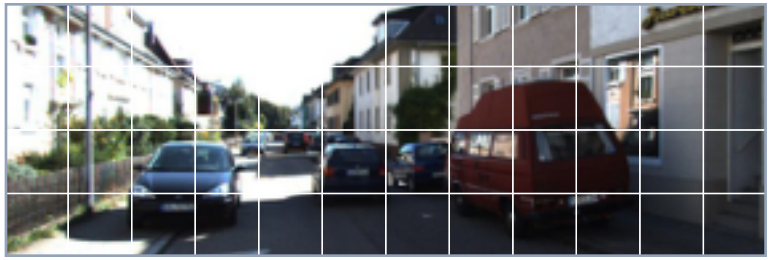


12 visible patches; 36 values withheld from the encoder.

The training procedure still has every original patch value.

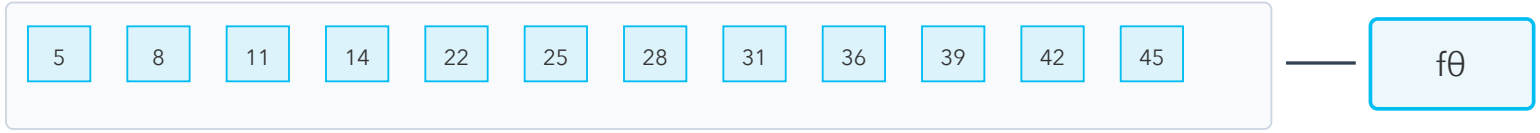
Visible and masked patches

Original observation x



Visible patches V · hidden patches M



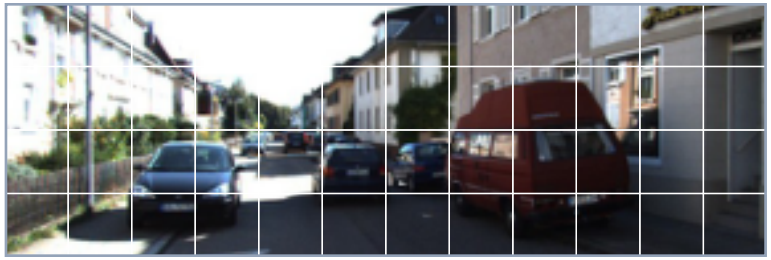


Visible patch tokens + their positions

The MAE (masked autoencoder) encoder receives visible tokens, with their original positions.

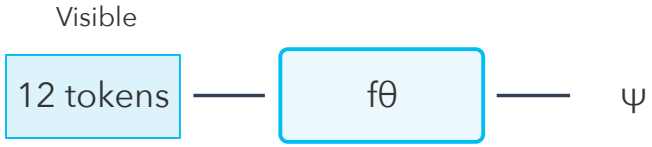
MAE architecture

Original observation



Encoder input: visible patches only

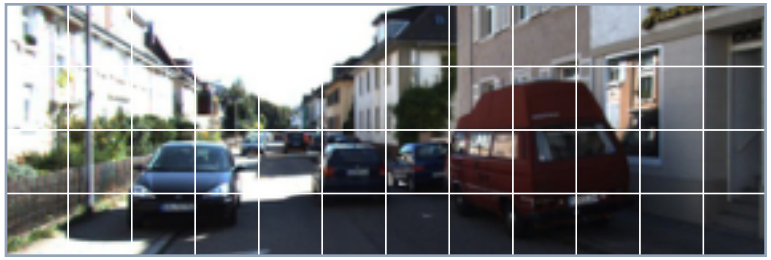




Encode only visible patches.

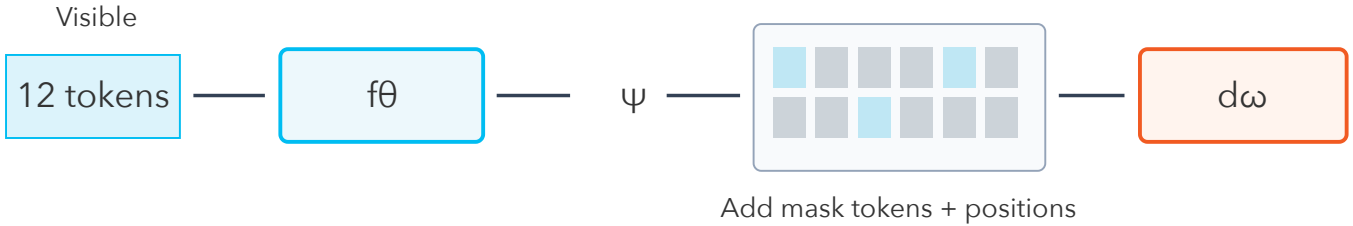
MAE architecture

Original observation



Encoder input: visible patches only

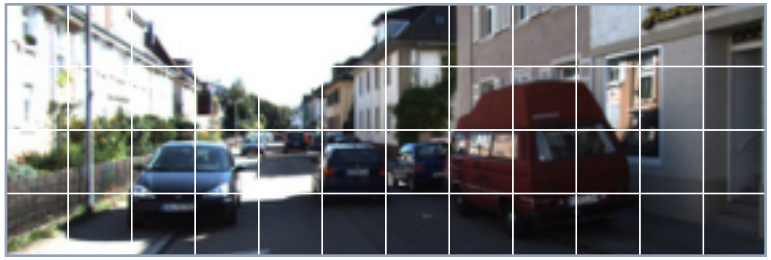




Reinsert locations using encoded visible tokens and learned mask tokens.

MAE architecture

Original observation



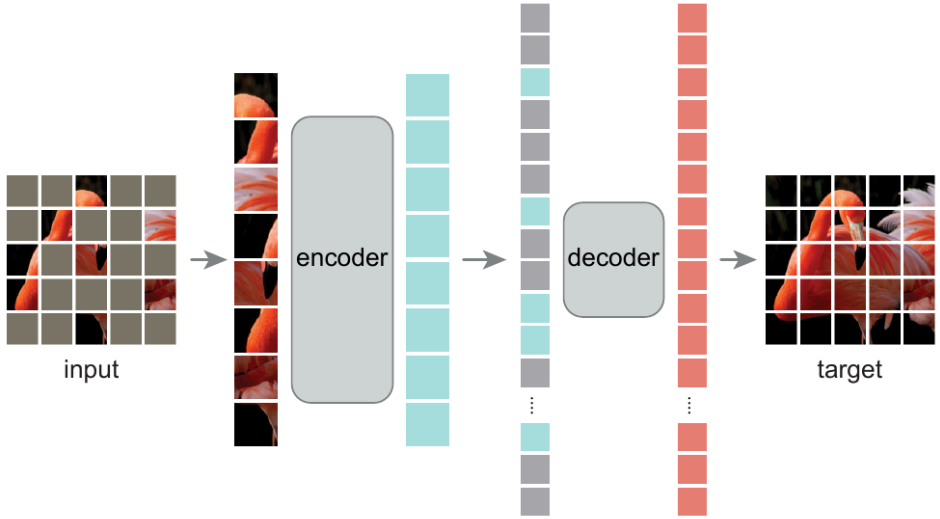
Encoder input: visible patches only





The decoder predicts pixel vectors at all patch positions.

MAE architecture

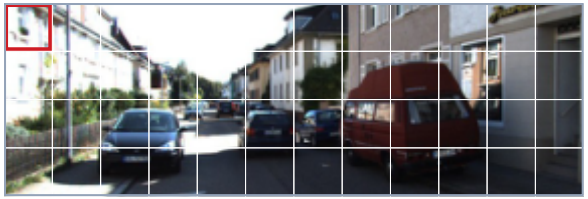


Mask tokens enter the decoder after visible-patch encoding.

He et al., MAE, CVPR 2022 · Fig. 1 · arXiv v3

Reconstruction loss on masked locations

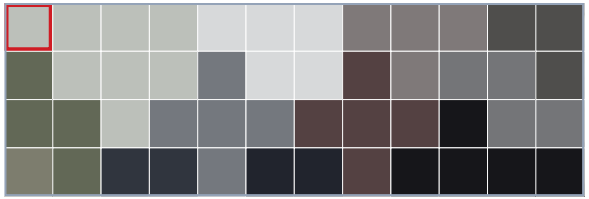
Original target pixels



Visible encoder input



Pixel prediction



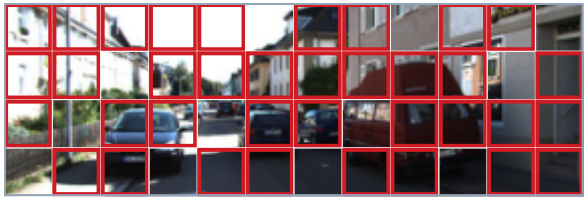
$$\|\hat{x}_k - x_k\|_2^2$$

Selected masked patch k = 0: squared error = 96.68

x_k is the original RGB pixel vector of patch k.

Reconstruction loss on masked locations

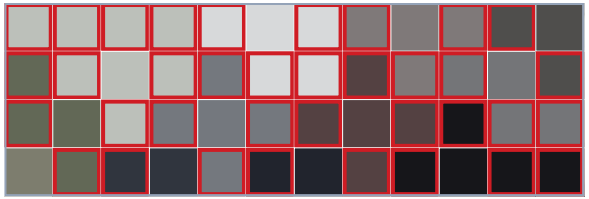
Original target pixels



Visible encoder input



Pixel prediction



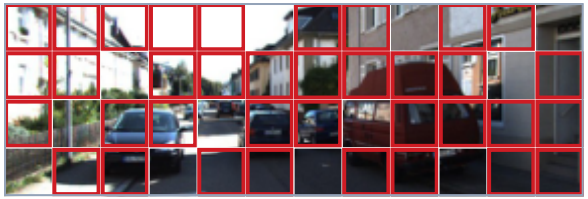
$$\mathcal{L}_{\text{pixel}} = \frac{1}{|\mathcal{M}|} \sum_{k \in \mathcal{M}} \|\hat{x}_k - x_k\|_2^2$$

Average over 36 masked patches: Lpixel = 148.32

x_k is the original RGB pixel vector of patch k.

Reconstruction loss on masked locations

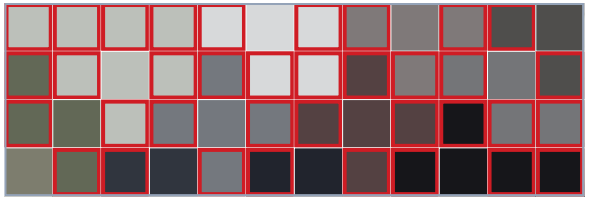
Original target pixels



Visible encoder input



Pixel prediction



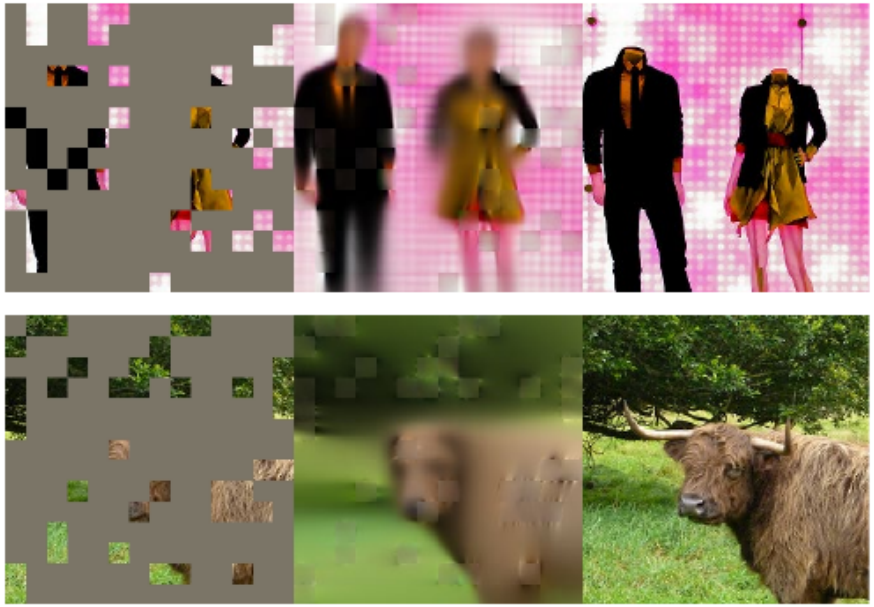
$$\mathcal{L}_{\text{pixel}} = \frac{1}{|\mathcal{M}|} \sum_{k \in \mathcal{M}} \|\hat{x}_k - x_k\|_2^2$$

> Average over 36 masked patches: Lpixel = 148.32

The basic loss acts on masked patches.

Plausible reconstruction and ambiguity

Masked input MAE reconstruction Original image



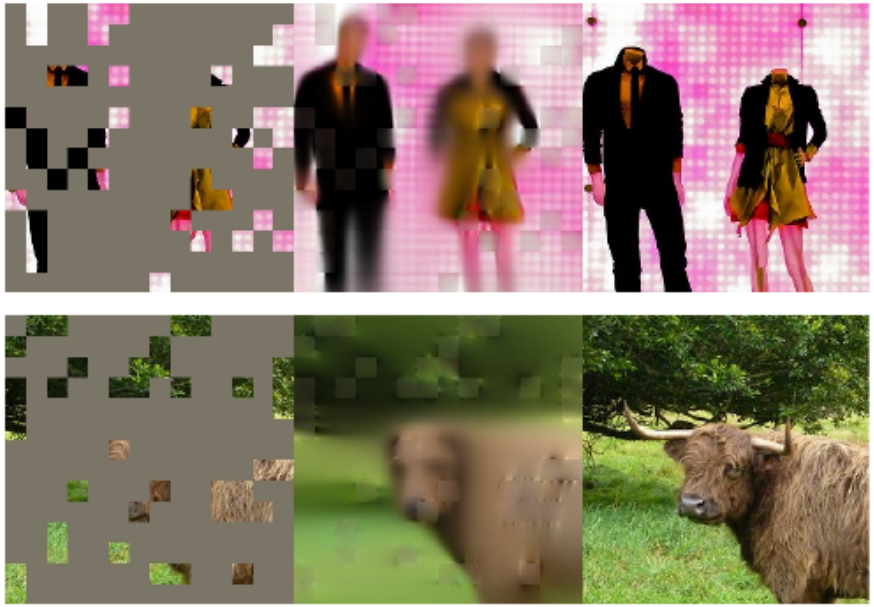
Multiple possible completions

Visible context does not determine every hidden detail.

He et al., MAE, CVPR 2022 · Fig. 3, two right-most examples · arXiv v3 · COCO validation images; model pretrained on ImageNet

Plausible reconstruction and ambiguity

Masked input MAE reconstruction Original image



Multiple possible completions

Visible context does not determine every hidden detail.

The outputs differ from the originals while retaining plausible scene structure.

Visual plausibility alone does not establish semantic understanding or robot-task accuracy.

He et al., MAE, CVPR 2022 · Fig. 3, two right-most examples · arXiv v3 · COCO validation images; model pretrained on ImageNet

What does a prediction target preserve?



Brick texture

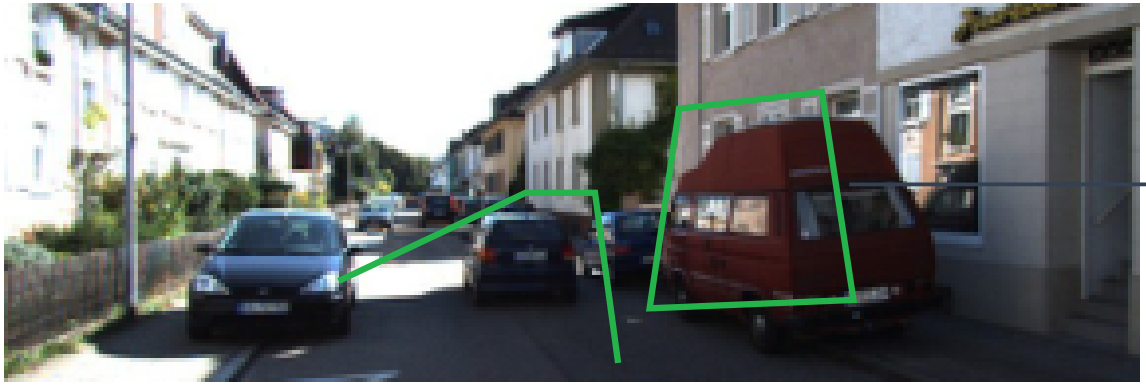
Appearance detail

Pixel targets ask for color, illumination, and texture.

A target defines what prediction errors the model must reduce.

> The features worth learning depend on the task.

What does a prediction target preserve?

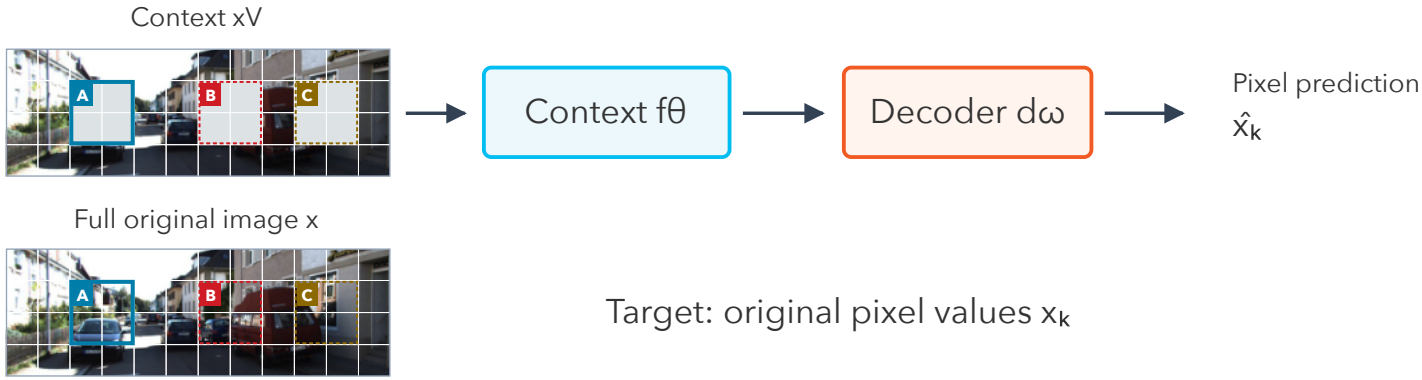


A robot task may place more value on object and road structure.

A target defines what prediction errors the model must reduce.

> The features worth learning depend on the task.

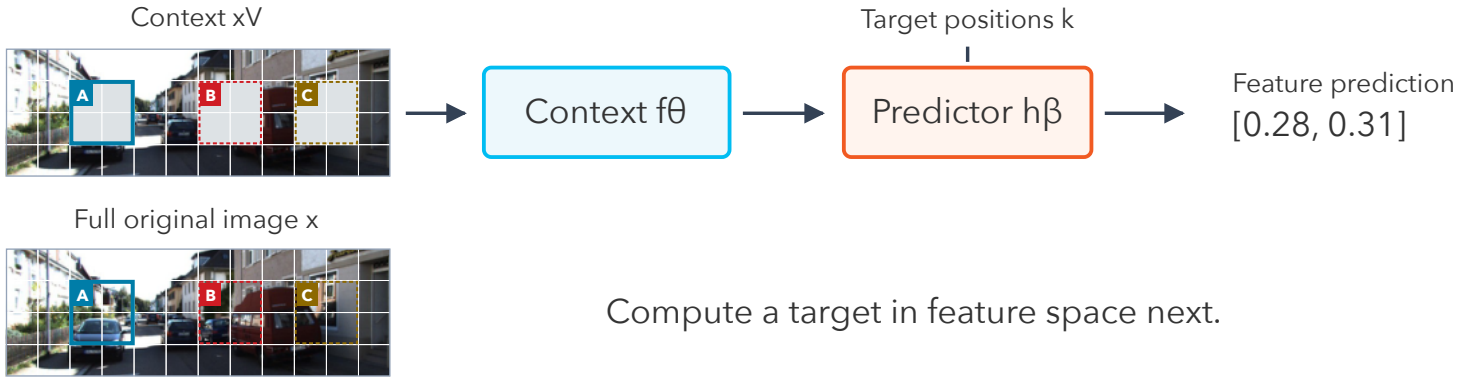
JEPA: Predicting in feature space



Pixel reconstruction compares output with the observed image.

> Compute the target in the intermediate feature space. No decoding into the output space (e.g., an image) is required. This supports learning more abstract concepts and more efficient training at scale.

JEPA: Predicting in feature space

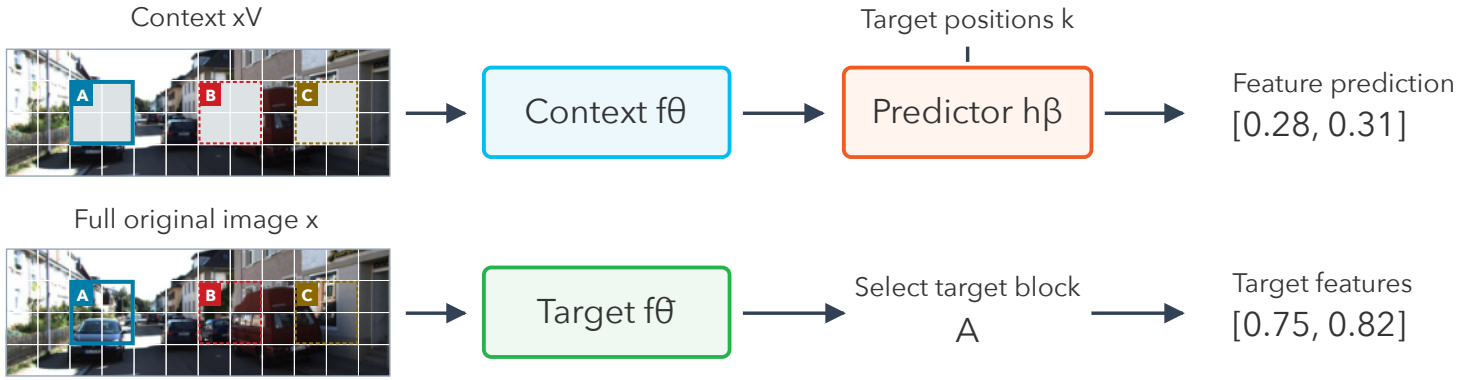


Compute a target in feature space next.

The predictor produces features for specified hidden locations.

> Compute the target in the intermediate feature space. No decoding into the output space (e.g., an image) is required. This supports learning more abstract concepts and more efficient training at scale.

JEPA: Predicting in feature space

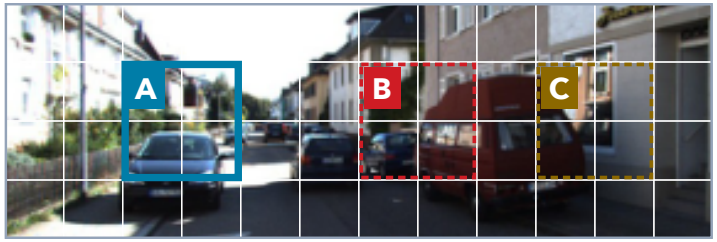


$$\psi_k^{\text{target}} = f_{\bar{\theta}}(x)_k$$

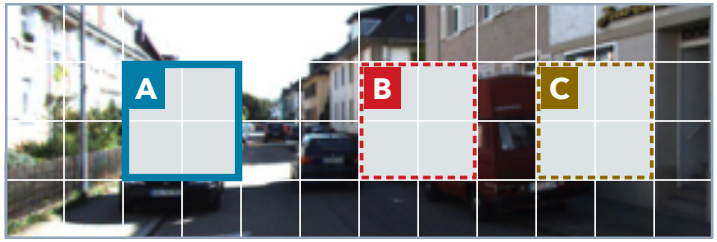
Compute the target in the intermediate feature space. No decoding into the output space (e.g., an image) is required. This supports learning more abstract concepts and more efficient training at scale.

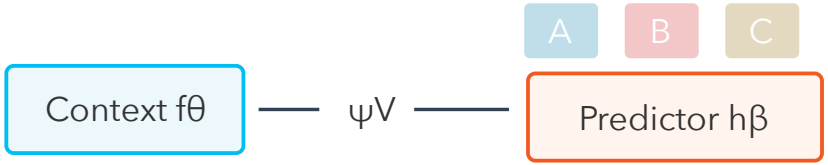
JEPA: Context and target positions

Original · spatial target blocks A, B, C



Context V · every target region removed



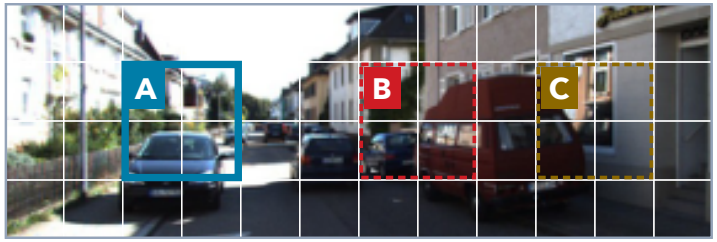


A position token identifies where to predict; it contains no hidden pixels.

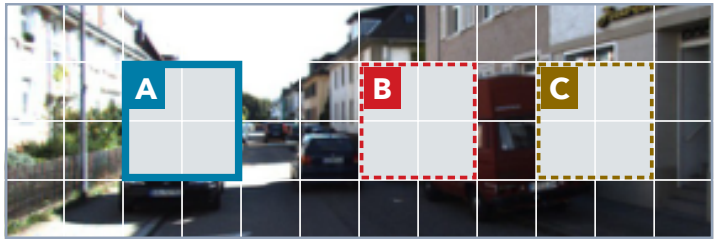
$$\widehat{\psi}_k = h_\beta(f_\theta(x_\mathcal{V}), k)$$

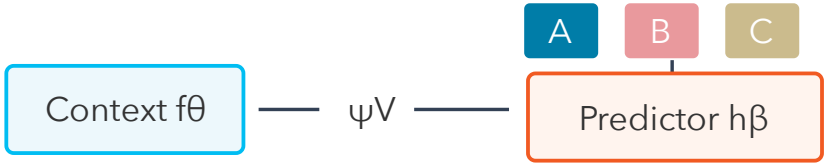
JEPA: Context and target positions

Original · spatial target blocks A, B, C



Context V · every target region removed



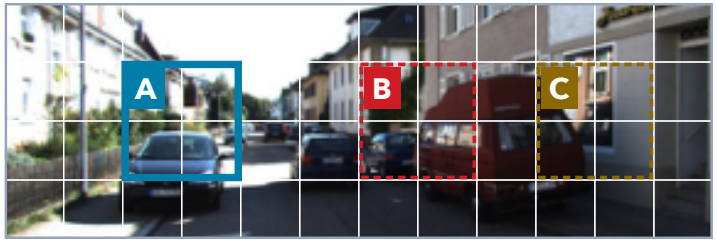


A position token identifies where to predict; it contains no hidden pixels.

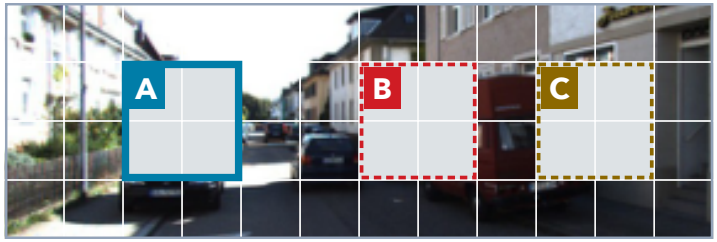
$$\widehat{\psi}_k = h_\beta(f_\theta(x_\mathcal{V}), k)$$

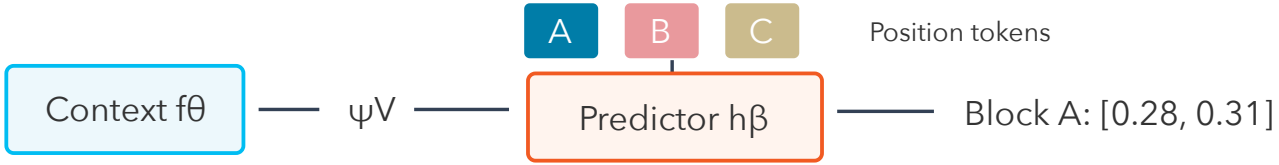
JEPA: Context and target positions

Original · spatial target blocks A, B, C



Context V · every target region removed

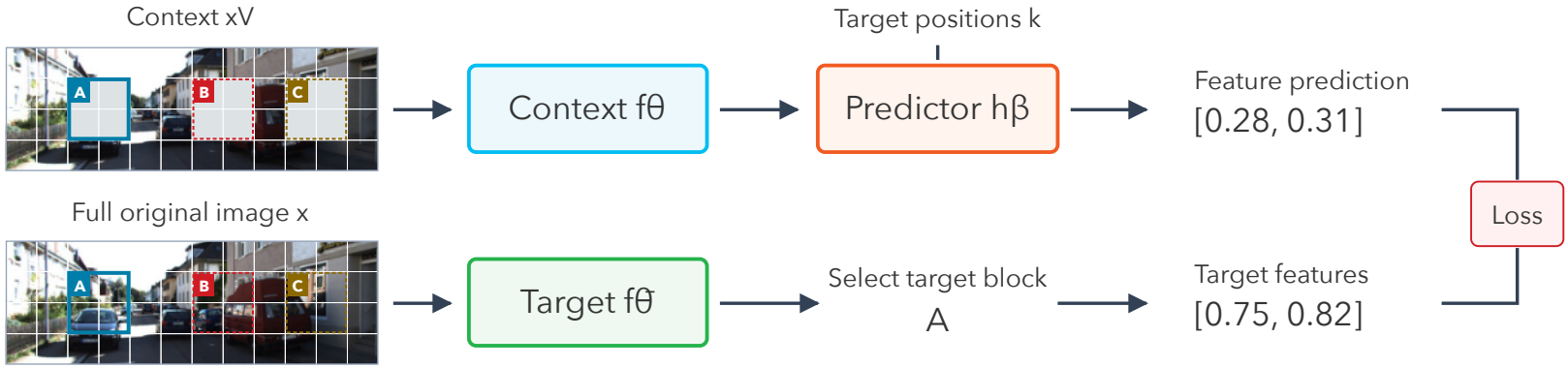




A position token identifies where to predict; it contains no hidden pixels.

$$\widehat{\psi}_k = h_\beta(f_\theta(x_\mathcal{V}), k)$$

JEPA: Feature loss and target updates

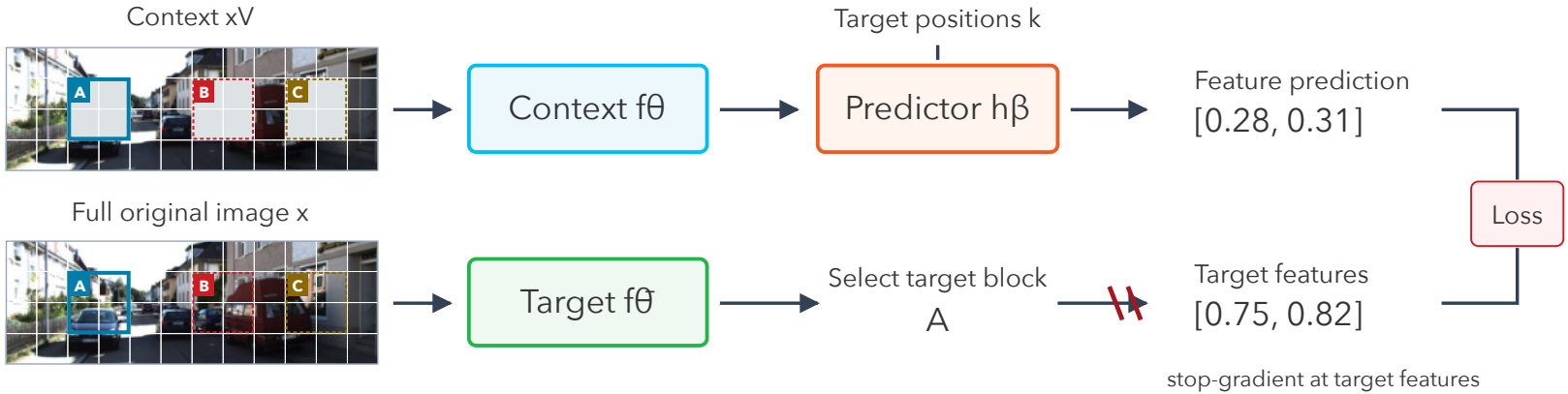


$$\mathcal{L}_{\text{feature}} = \frac{1}{|\mathcal{M}|} \sum_{k \in \mathcal{M}} \|\widehat{\boldsymbol{\psi}}_k - \boldsymbol{\psi}_k^{\text{target}}\|_2^2$$

Lfeature = 0.1978 θ = 0.800 · β = 0.700 · θ̄ = 1.000

Schematic squared distance, averaged over one target block.

JEPA: Feature loss and target updates



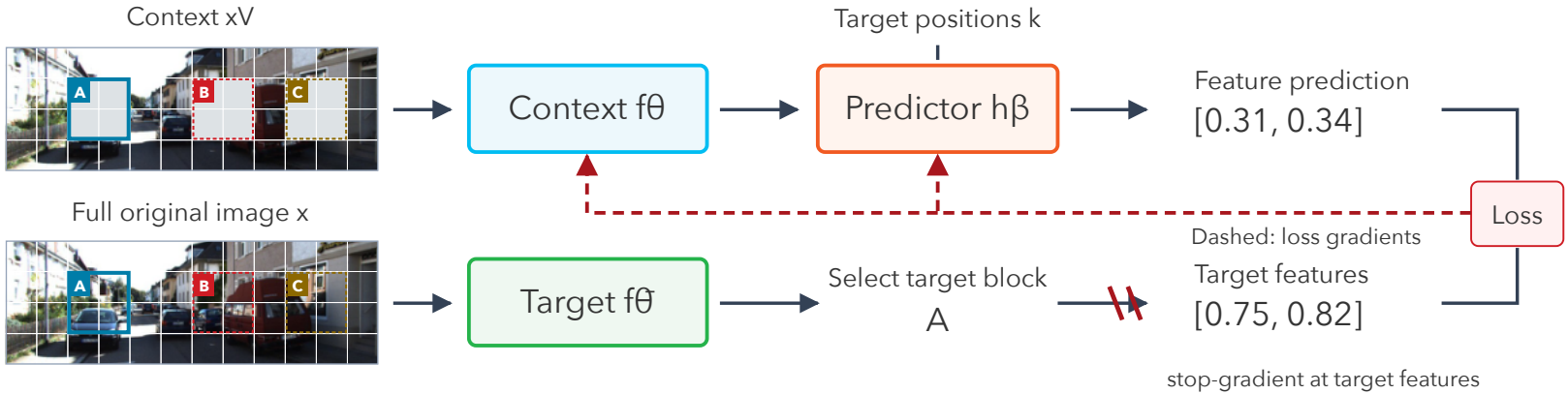
$$\psi_k^{\text{target}} = \text{sg}(f_{\bar{\theta}}(x)_k)$$

Lfeature = 0.1978

θ = 0.800 · β = 0.700 · θ̄ = 1.000

Treat target features as fixed when differentiating the loss.

JEPA: Feature loss and target updates



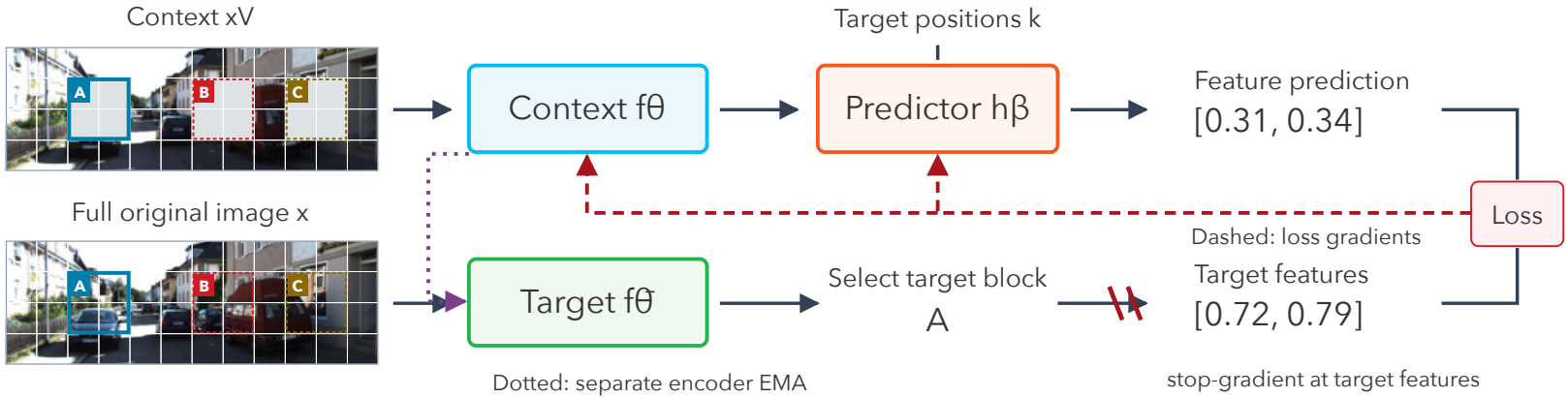
$$(\theta, \beta)_{n+1} = (\theta, \beta)_n - \eta \nabla_{(\theta,\beta)} \mathcal{L}_{\text{feature}}$$

Lfeature = 0.1731

θ = 0.833 · β = 0.738 · θ̄ = 1.000

Gradients update the context encoder and predictor.

JEPA: Feature loss and target updates



$$\bar{\theta}_{n+1} = \mu\bar{\theta}_n + (1-\mu)\theta_{n+1}$$

Lfeature = 0.1515

θ = 0.833 · β = 0.738 · θ̄ = 0.967

EMA transfers context-encoder parameters.

Pixel targets and feature targets

MAE · pixel target



Pixel targets and feature targets

MAE · pixel target



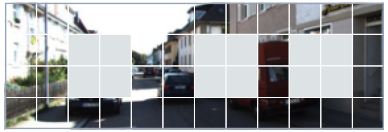
fθ → dω → $\hat{x}_k$ ↔

Mask + position tokens



Observed pixels x_k

I-JEPA · feature target



fθ → hβ → $\hat{\psi}_k$ ↔ $sg(f_{\bar{\theta}}(x)_k)$

Target-position tokens

Targets across all image features

Pixel targets and feature targets

MAE · pixel target



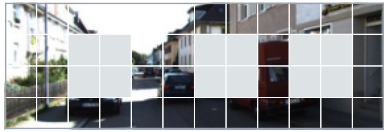
fθ → dω → $\hat{x}_k$ ↔

Mask + position tokens



Observed pixels x_k

I-JEPA · feature target



fθ → hβ → $\hat{\psi}_k$ ↔ $sg(f_{\bar{\theta}}(x)_k)$

Target-position tokens

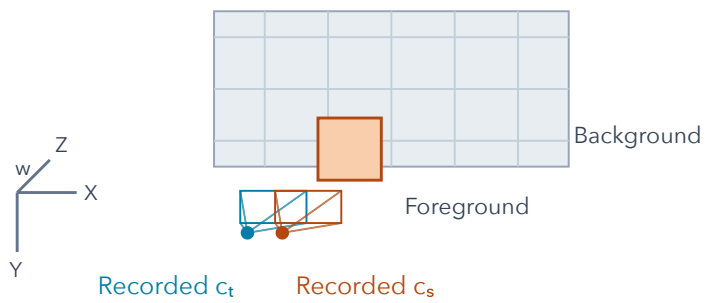
Targets across all image features

SECTION 05

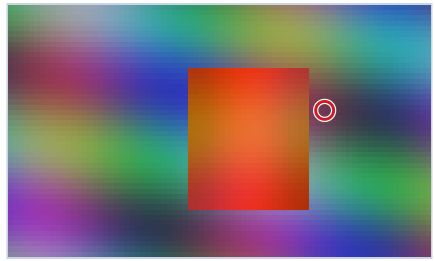
Geometric supervision

Learning depth from multiple observations

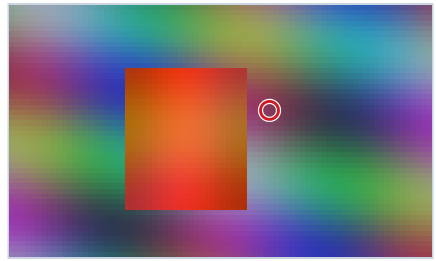
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Source marker follows the computed projection.

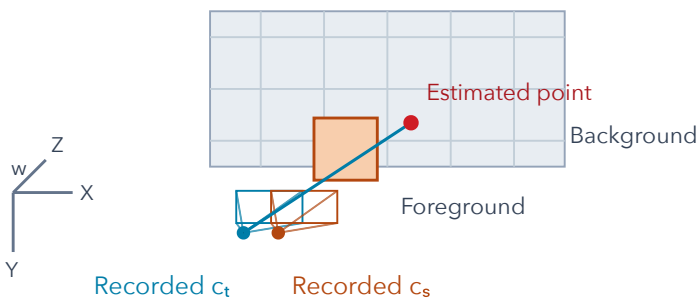
$\hat{D} = 3.50 \cdot \hat{p}$ in c_t: [1.06, -0.18, 3.50] · u_s = [49.31, 20.00]

Recorded image values provide the target; depth and relative pose are predicted.

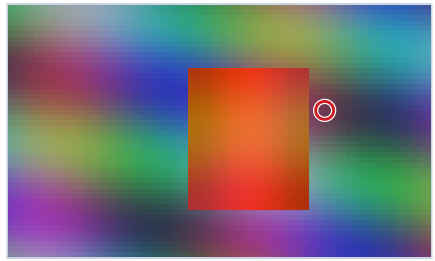
Scene structure and camera motion can turn recorded images into a learning signal.

Learning depth from multiple observations

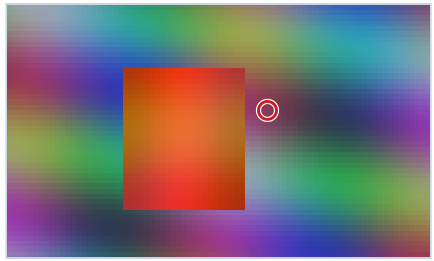
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Source marker follows the computed projection.

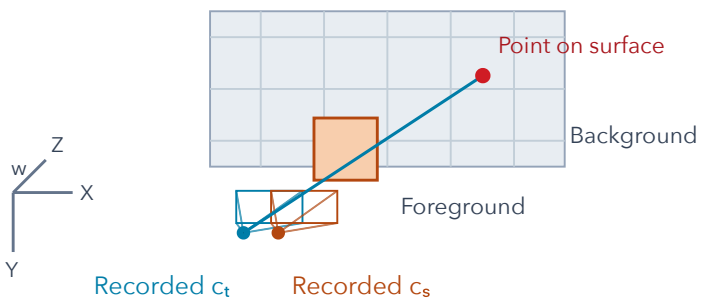
$\hat{D} = 3.50 \cdot \hat{p}$ in c_t: [1.06, -0.18, 3.50] · u_s = [49.31, 20.00]

Recorded image values provide the target; depth and relative pose are predicted.

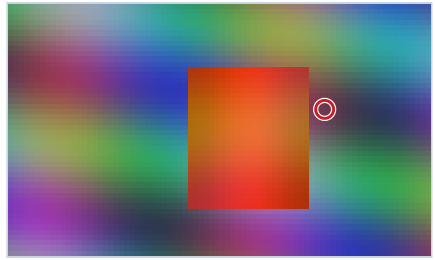
Scene structure and camera motion can turn recorded images into a learning signal.

Camera projection and intrinsics

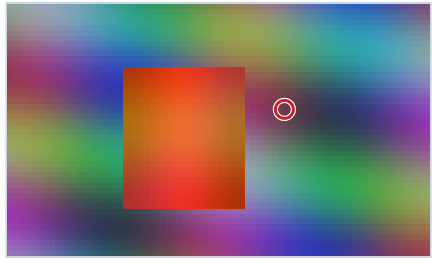
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Source marker follows the computed projection.

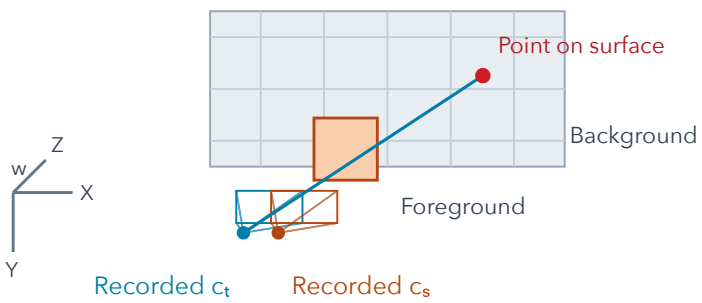
$$\mathbf{u} = \pi\left(K_t p^{c_t}\right)$$

fx = fy = 68 · cx = 39.5 · cy = 23.5 · X right, Y down, Z forward

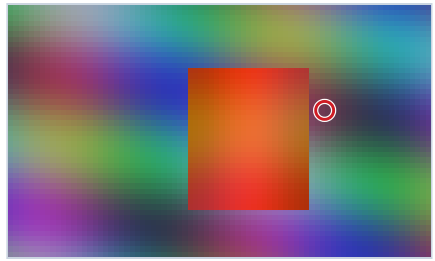
Image coordinates: u increases right; v increases down.

Camera projection and intrinsics

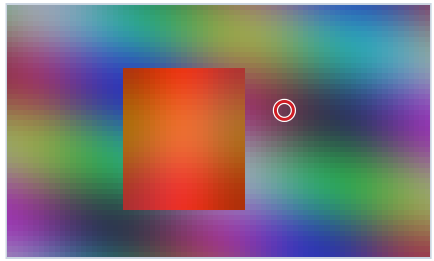
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Source marker follows the computed projection.

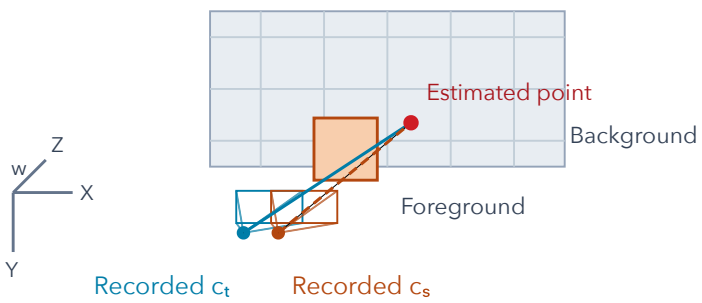
$$K_t = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

fx = fy = 68 · cx = 39.5 · cy = 23.5 · X right, Y down, Z forward

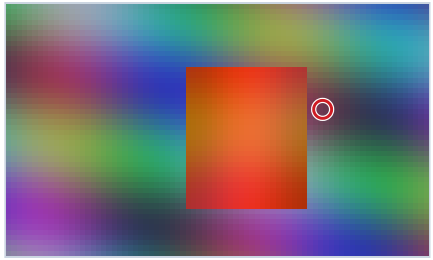
Optical-axis Z is depth; distance along a unit ray is a different quantity.

Back-projecting a target pixel

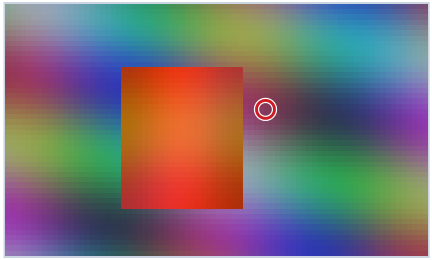
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Source marker follows the computed projection.

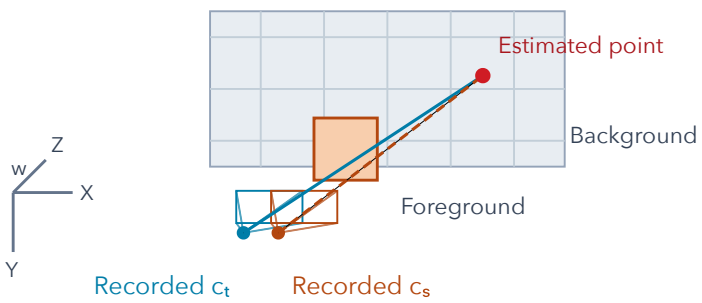
$$\hat{p}^{c_t} = \widehat{D}_t(\mathbf{u}) K_t^{-1} \tilde{\mathbf{u}}$$

$\hat{D}$ = 3.50 · $\hat{p}$ in c_t: [1.06, -0.18, 3.50] · u_s = [49.31, 20.00]

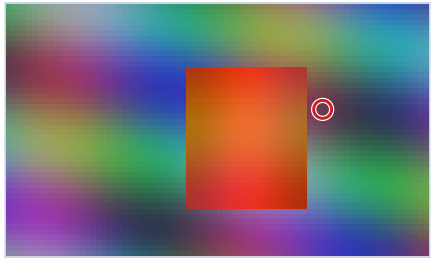
Changing estimated depth moves the hypothesis along a fixed target ray.

Back-projecting a target pixel

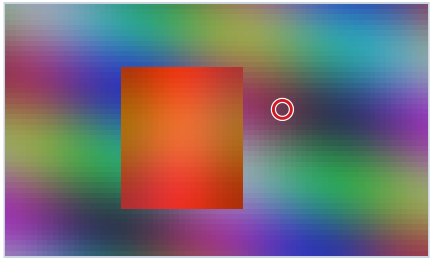
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Source marker follows the computed projection.

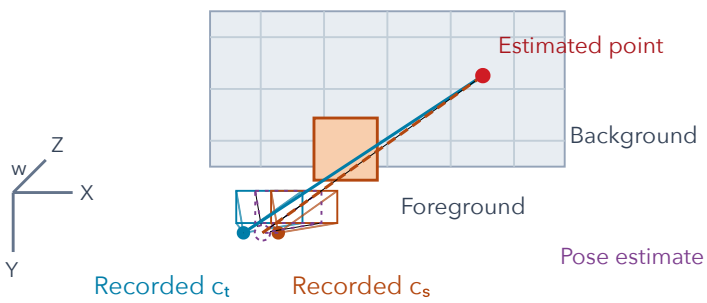
$$\hat{p}^{c_t} = \widehat{D}_t(\mathbf{u}) K_t^{-1} \tilde{\mathbf{u}}$$

$\hat{D}$ = 5.00 · $\hat{p}$ in c_t: [1.51, -0.26, 5.00] · u_s = [52.52, 20.00]

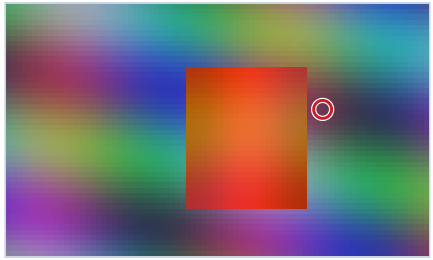
Changing estimated depth moves the hypothesis along a fixed target ray.

Effect of camera pose

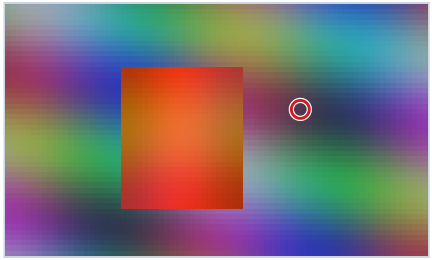
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Source marker follows the computed projection.

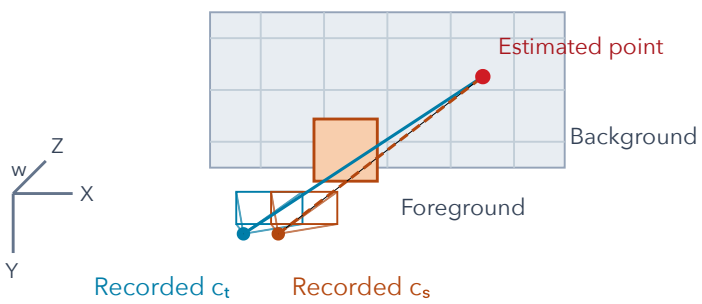
$$\hat{p}^{c_s} = \hat{R}^{c_s}_{c_t}\hat{p}^{c_t} + \hat{t}^{c_s}_{c_t}$$

$\hat{p}$ in c_t: [1.51, -0.26, 5.00] · $\hat{p}$ in c_s: [1.21, -0.26, 5.00]

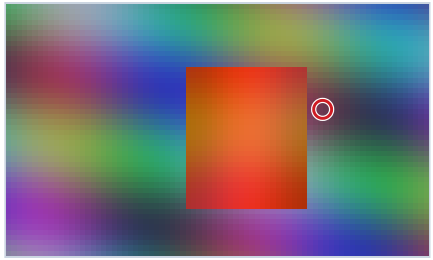
Changing the pose estimate changes coordinates and sampling; recorded images stay fixed.

Effect of camera pose

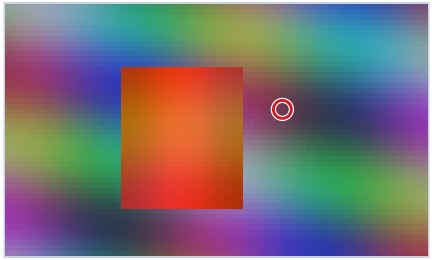
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Source marker follows the computed projection.

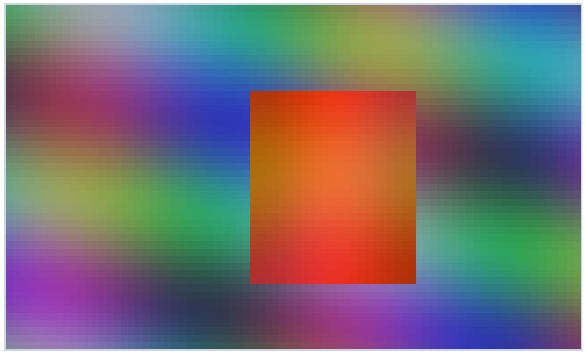
$$\hat{p}^{c_s} = \widehat{R}^{c_s}_{c_t}\hat{p}^{c_t} + \hat{t}^{c_s}_{c_t}$$

$\hat{p}$ in c_t: [1.51, -0.26, 5.00] · $\hat{p}$ in c_s: [0.96, -0.26, 5.00]

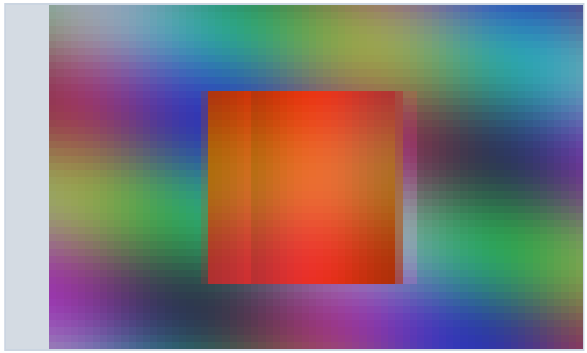
Changing the pose estimate changes coordinates and sampling; recorded images stay fixed.

Photometric reconstruction loss

Recorded target I_t



Reconstructed target $\hat{I}_t$



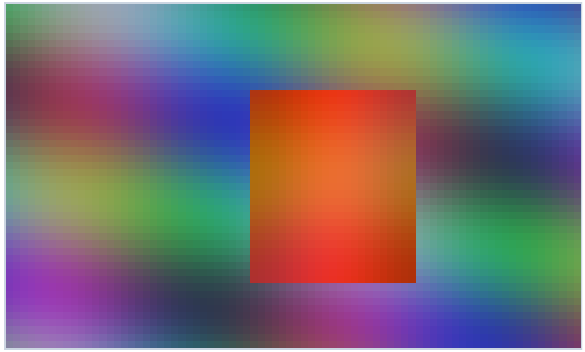
Predict where to sample in I_s for each target pixel.

$$\mathcal{L}_{\text{photo}} = \frac{1}{|\Omega|} \sum_{\mathbf{u} \in \Omega} \| I_t(\mathbf{u}) - \hat{I}_t(\mathbf{u}) \|_1$$

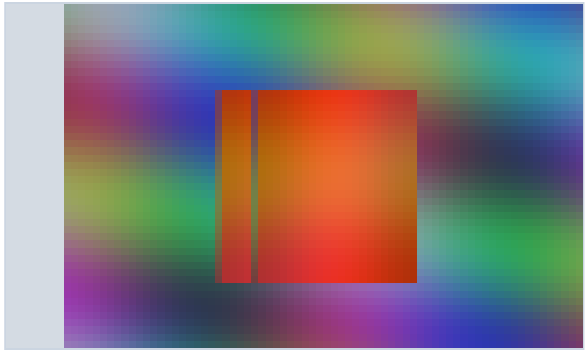
Mean RGB L1 = 0.1343 · 3552 / 3840 pixels in Ω

Photometric reconstruction loss

Recorded target I_t



Reconstructed target $\hat{I}_t$



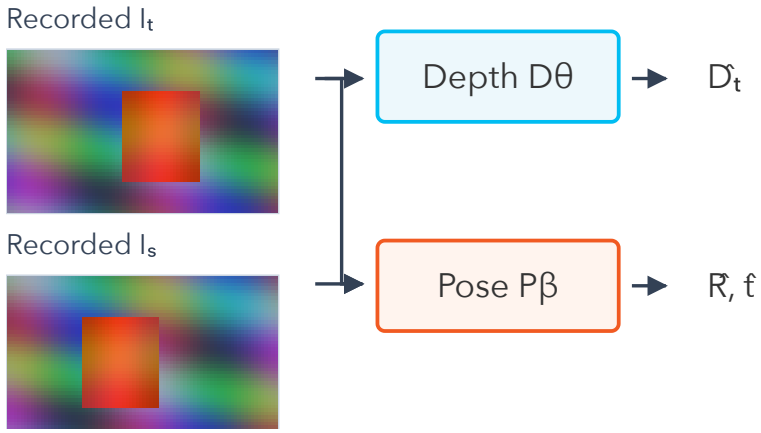
RGB L1 error · brighter = larger



$$\mathcal{L}_{\text{photo}} = \frac{1}{|\Omega|} \sum_{\mathbf{u} \in \Omega} \| I_t(\mathbf{u}) - \hat{I}_t(\mathbf{u}) \|_1$$

Mean RGB L1 = 0.0448 · 3456 / 3840 pixels in Ω

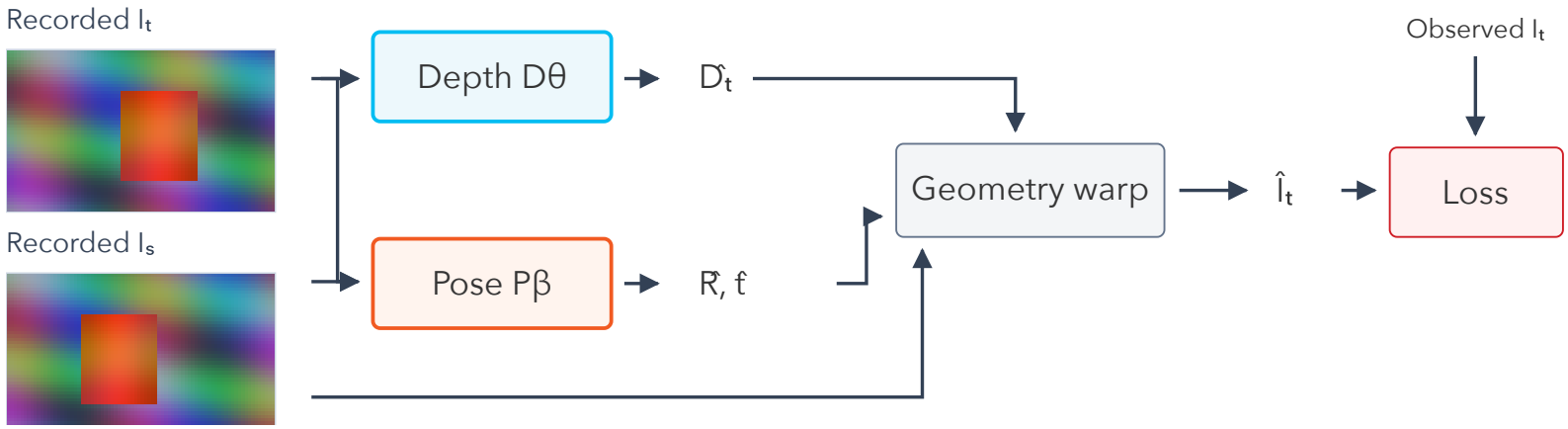
Updating depth and pose networks



$$\widehat{D}_t = D_\theta(I_t), \qquad \widehat{T}_{c_t}^{c_s} = P_\beta(I_t, I_s)$$

The predicted quantities are depth and relative pose.

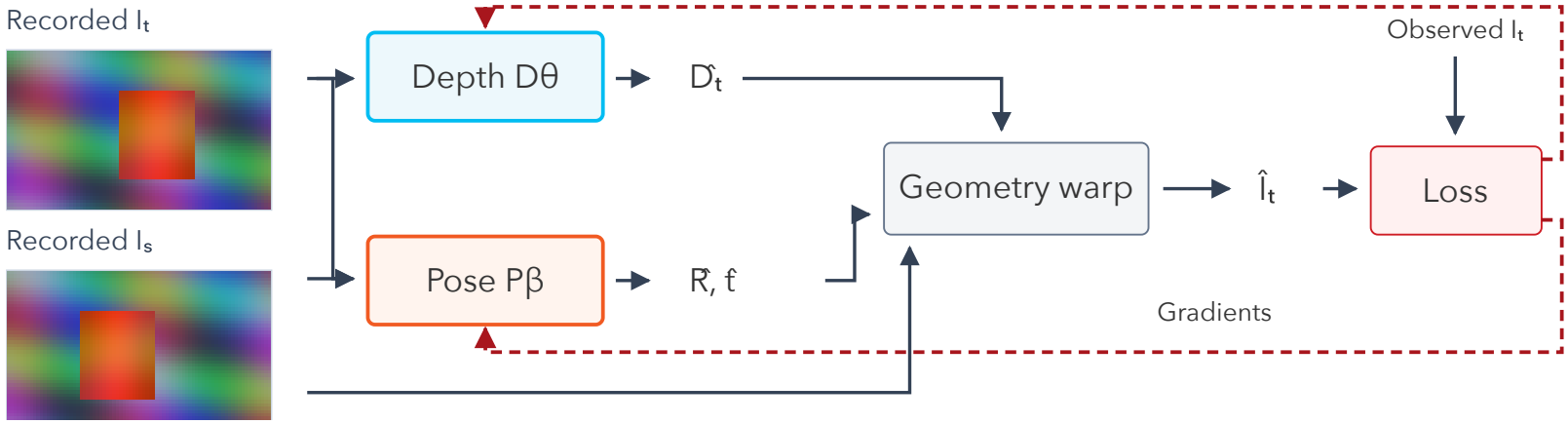
Updating depth and pose networks



$$\widehat{D}_t = D_\theta(I_t), \qquad \widehat{T}_{c_t}^{c_s} = P_\beta(I_t, I_s)$$

The predicted quantities are depth and relative pose.

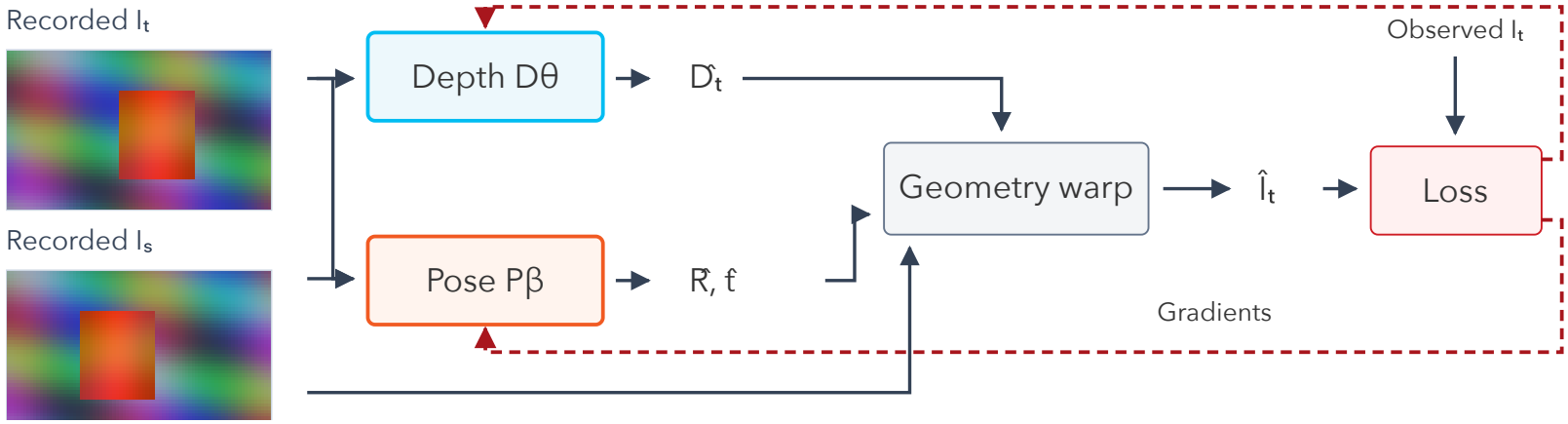
Updating depth and pose networks



$$\Theta_{n+1} = \Theta_n - \eta \nabla_\Theta \mathcal{L}_{\text{photo}}, \qquad \Theta = (\theta, \beta)$$

Update network parameters; recorded images and physical camera poses stay fixed.

Updating depth and pose networks

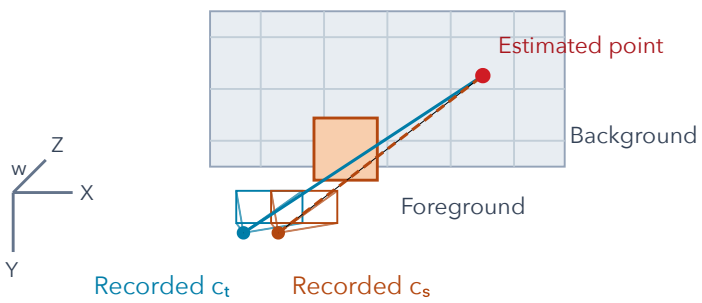


$$\Theta_{n+1} = \Theta_n - \eta \nabla_\Theta \mathcal{L}_{\text{photo}}, \qquad \Theta = (\theta, \beta)$$

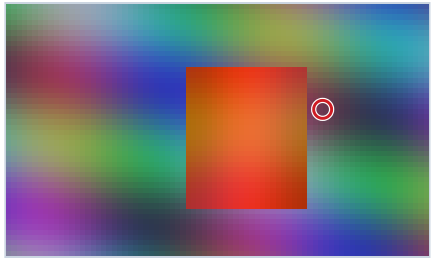
Computed scalar-model update: lower pixel error does not determine metric scale.

Monocular scale ambiguity

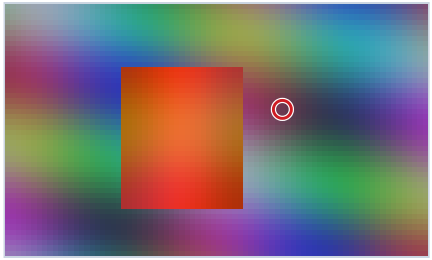
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Target Sampled **RGB L1 = 0.0004**

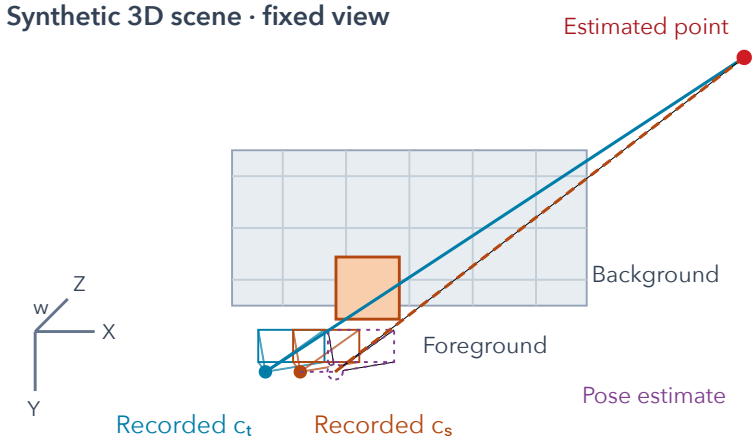
Source marker follows the computed projection.

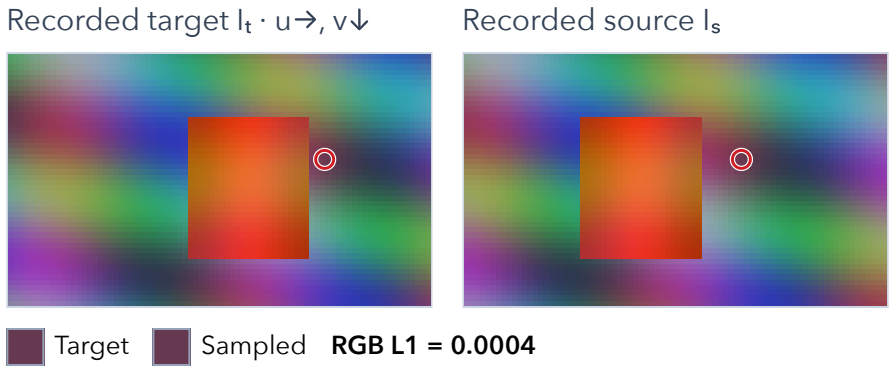
$$\pi(K_s\,\kappa\hat{p}^{c_s}) = \pi(K_s\hat{p}^{c_s}), \qquad \kappa > 0$$

$\hat{D}$ = 5.00 · $\hat{p}$ in c_t: [1.51, -0.26, 5.00] · u_s = [52.52, 20.00]

Scale the depth and every relative-translation component together; pixels stay fixed.

Monocular scale ambiguity





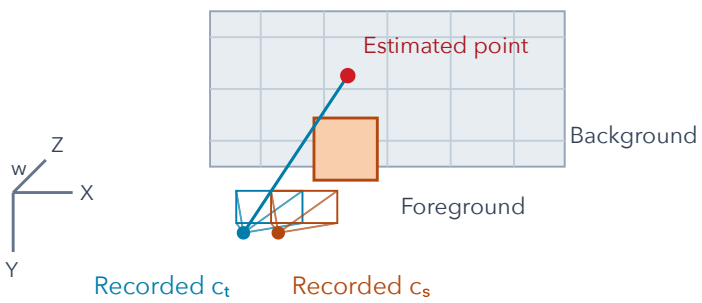
$$\pi\left(K_s\, \kappa \hat{p}^{c_s}\right) = \pi\left(K_s \hat{p}^{c_s}\right), \qquad \kappa > 0$$

$\hat{D}$ = 10.00 · $\hat{p}$ in c_t: [3.01, -0.51, 10.00] · u_s = [52.52, 20.00]

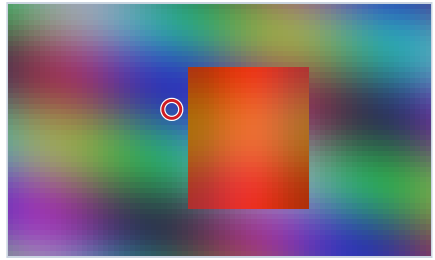
Scale the depth and every relative-translation component together; pixels stay fixed.

Occlusion limitations

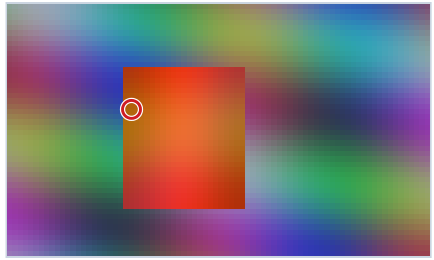
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Target Sampled **RGB L1 = 1.2332**

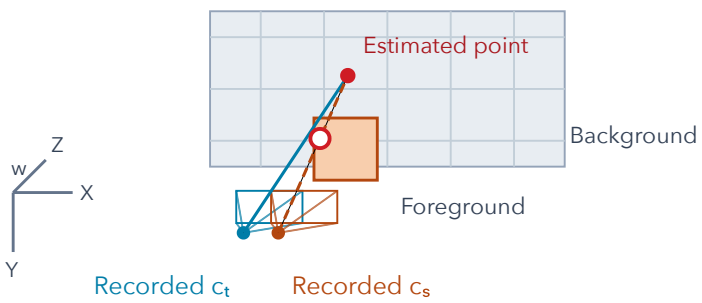
Source marker follows the computed projection.

$Z_s = 5.00 > 0$ · $u_s = [23.52, 20.00]$ · source ray hits the foreground first

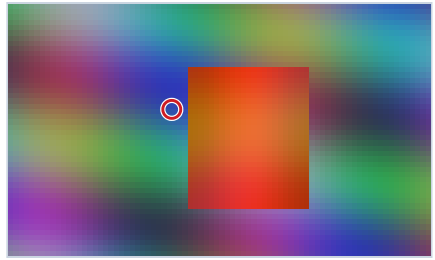
The target camera sees a background surface.

Occlusion limitations

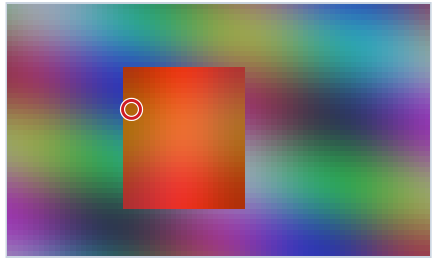
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



Target Sampled **RGB L1 = 1.2332**

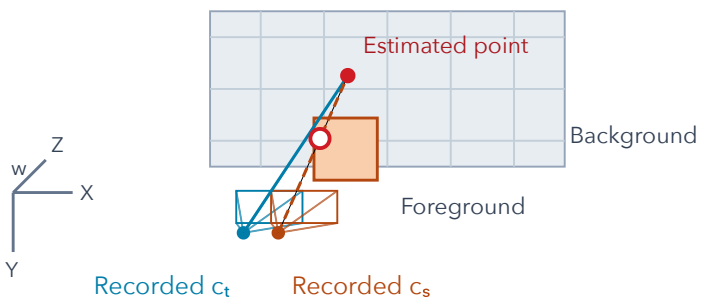
Source marker follows the computed projection.

$Z_s = 5.00 > 0$ · $u_s = [23.52, 20.00]$ · source ray hits the foreground first

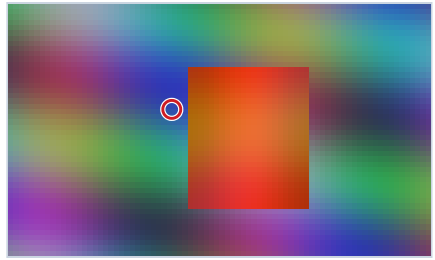
A positive, in-bounds projection can still hit an occluded surface.

Occlusion limitations

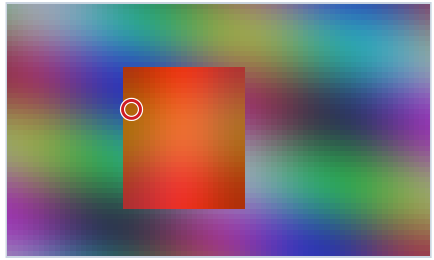
Synthetic 3D scene · fixed view



Recorded target I_t · u→, v↓



Recorded source I_s



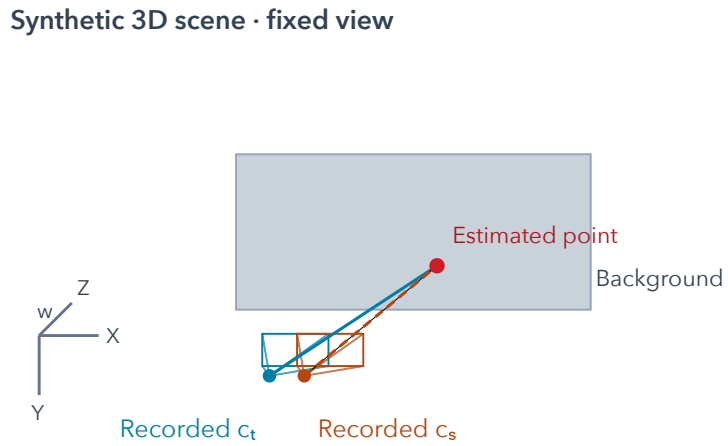
Target Sampled **RGB L1 = 1.2332**

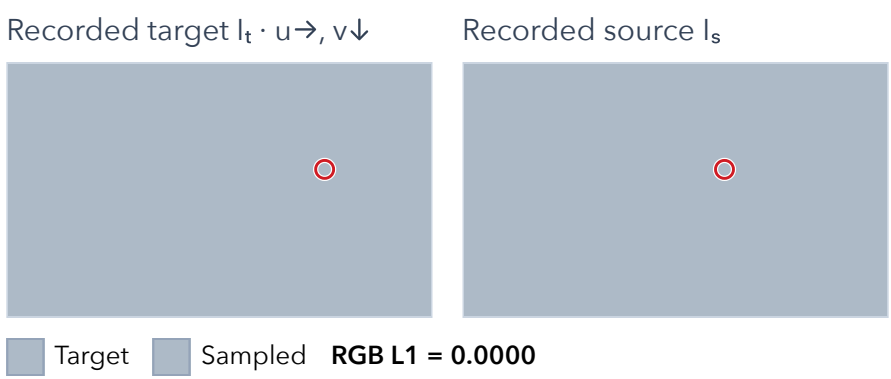
Source marker follows the computed projection.

$Z_s = 5.00 > 0$ · $u_s = [23.52, 20.00]$ · source ray hits the foreground first

A positive, in-bounds projection can still hit an occluded surface.

Texture and reflectance failures





$$I_t(\mathbf{u}) = \hat{I}_t(\mathbf{u}) \quad \text{for many depth hypotheses}$$

$\hat{D}$ = 3.50 · $\hat{p}$ in c_t: [1.06, -0.18, 3.50] · u_s = [49.31, 20.00]

Uniform appearance gives the same color at different projected locations.

Texture and reflectance failures



Godard et al., Monodepth2, ICCV 2019 · Fig. 8 · arXiv v4

Appearance consistency can fail

Top: distorted, reflective, and color-saturated regions.

These are the paper's examples, with its original method labels and highlighted regions.

Texture and reflectance failures



Godard et al., Monodepth2, ICCV 2019 · Fig. 8 · arXiv v4

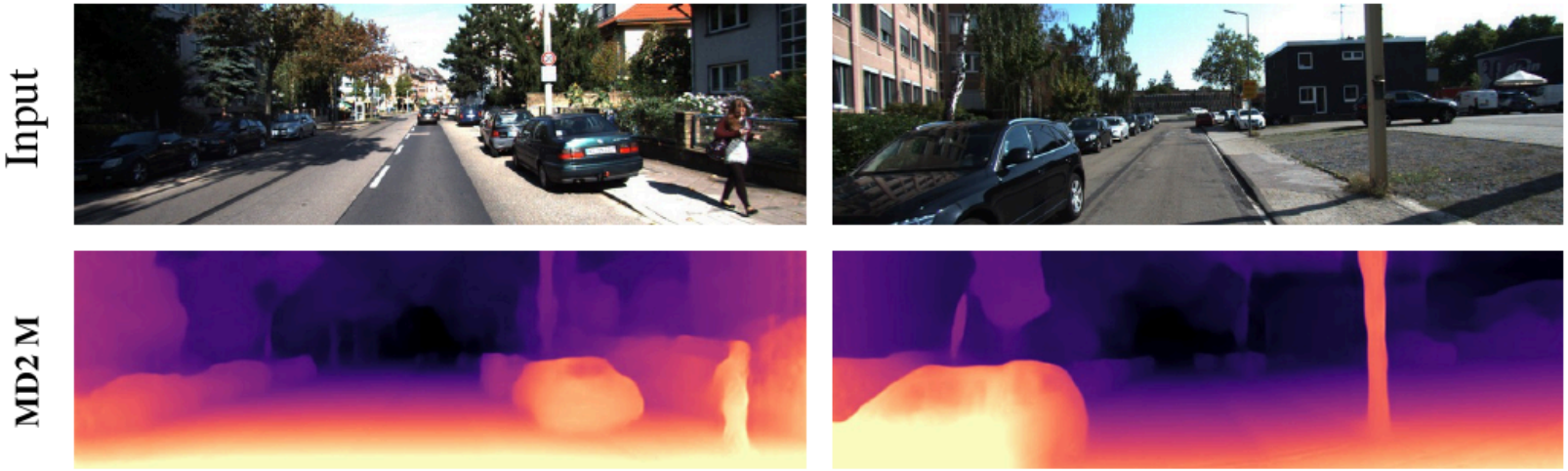
Appearance consistency can fail

Top: distorted, reflective, and color-saturated regions.

Bottom: ambiguous boundaries and intricate shapes can also be difficult.

These are the paper's examples, with its original method labels and highlighted regions.

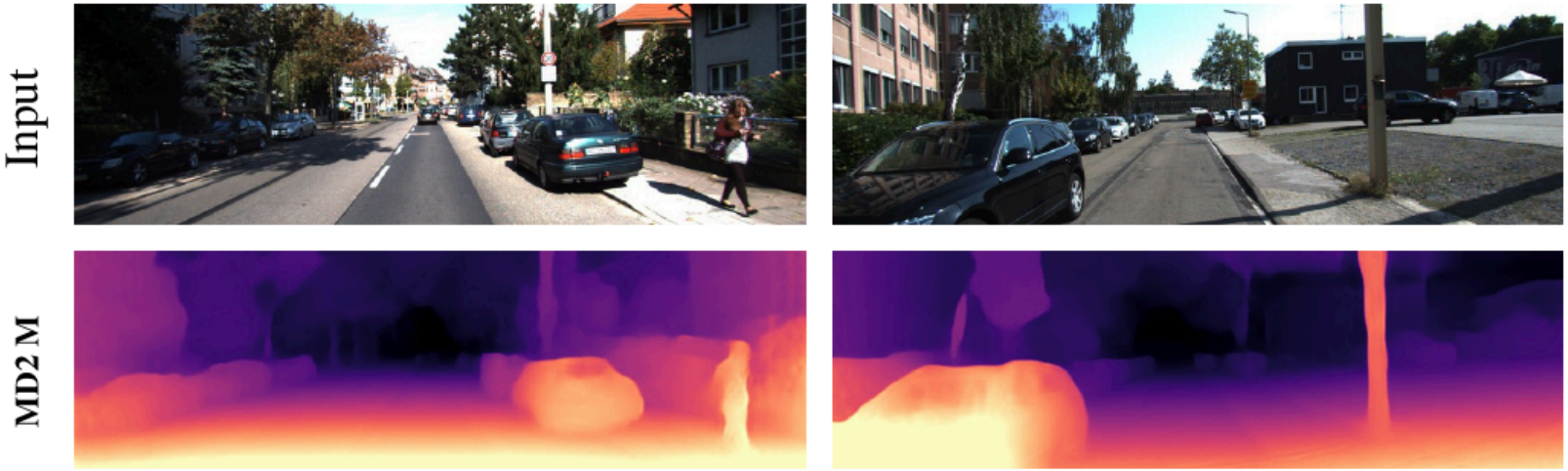
Learned depth on real images



Recorded camera images above; Monodepth2 monocular depth predictions below.

Godard et al., Monodepth2, ICCV 2019 · Fig. 7, first two examples, Input and MD2 M rows · KITTI Eigen split · arXiv v4

Learned depth on real images



Recorded camera images above; Monodepth2 monocular depth predictions below.

> Metric depth requires additional supervised fine-tuning against 3D data (e.g., LiDAR).

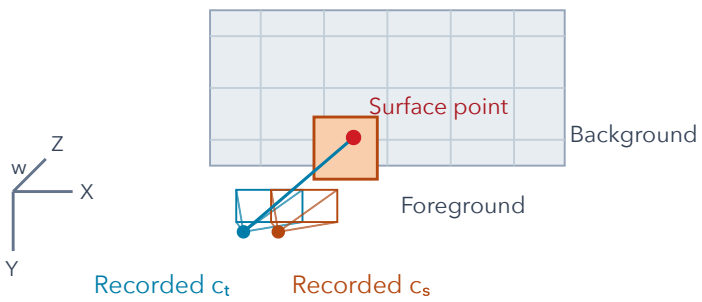
Godard et al., Monodepth2, ICCV 2019 · Fig. 7, first two examples, Input and MD2 M rows · KITTI Eigen split · arXiv v4

SECTION 06

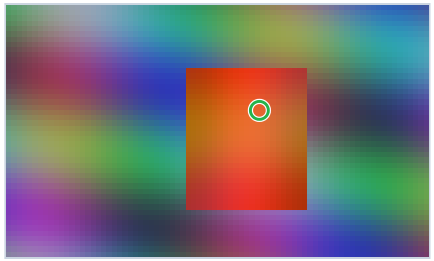
Dense correspondence

Geometrically generated pixel matches

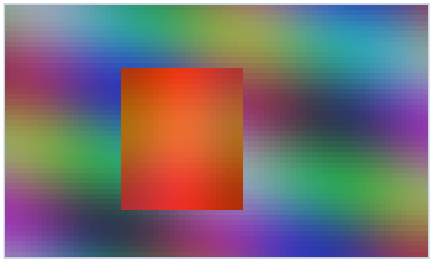
Synthetic 3D scene · fixed view



Target I_t



Source I_s

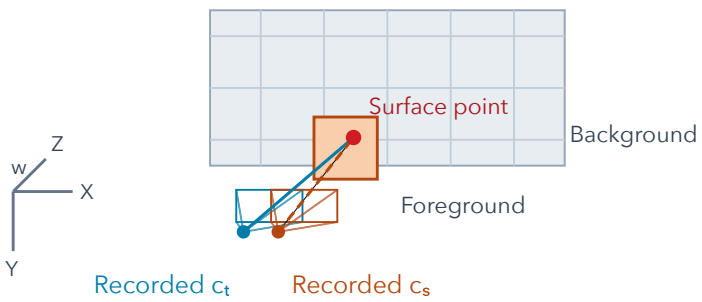


Green: same surface point

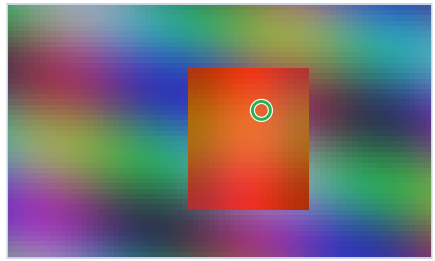
$$p^{c_t} = D_t(\mathbf{u}_t) K_t^{-1} \widetilde{\mathbf{u}}_t, \qquad \mathbf{u}_s = \pi(K_s(R_{c_t}^{c_s} p^{c_t} + t_{c_t}^{c_s}))$$

Geometrically generated pixel matches

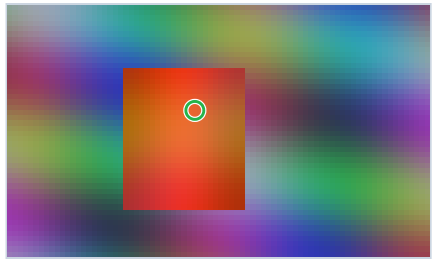
Synthetic 3D scene · fixed view



Target I_t



Source I_s

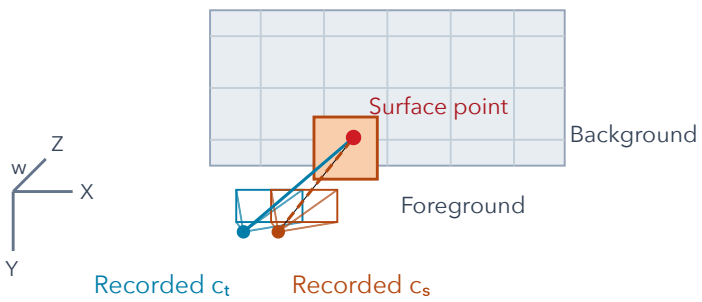


Green: same surface point

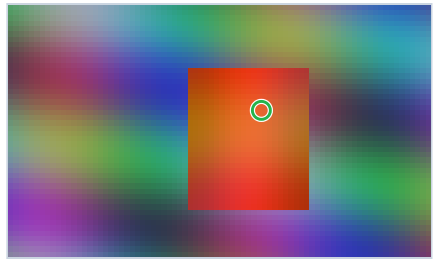
$$p^{c_t} = D_t(\mathbf{u}_t) K_t^{-1} \widetilde{\mathbf{u}}_t, \qquad \mathbf{u}_s = \pi(K_s(R_{c_t}^{c_s} p^{c_t} + t_{c_t}^{c_s}))$$

Geometrically generated pixel matches

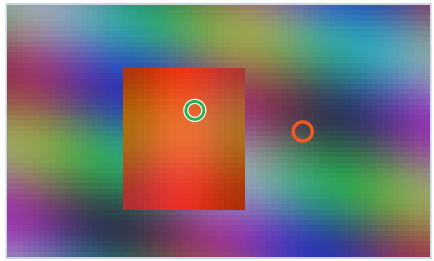
Synthetic 3D scene · fixed view



Target I_t



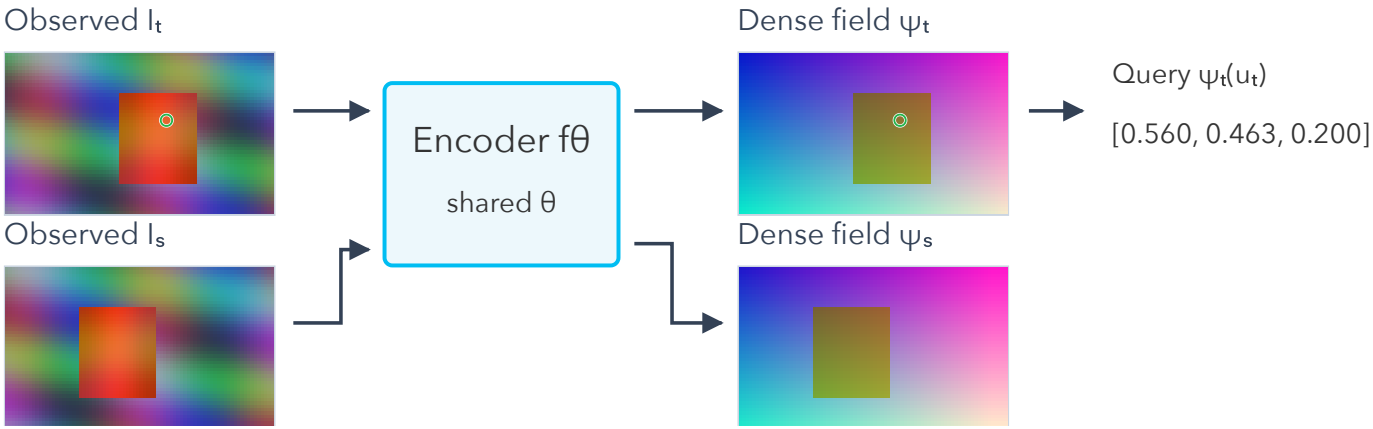
Source I_s



Green: same surface point · orange: different surface point

$$p^{c_t} = D_t(\mathbf{u}_t) K_t^{-1} \widetilde{\mathbf{u}}_t, \qquad \mathbf{u}_s = \pi(K_s(R_{c_t}^{c_s} p^{c_t} + t_{c_t}^{c_s}))$$

Learning dense descriptors

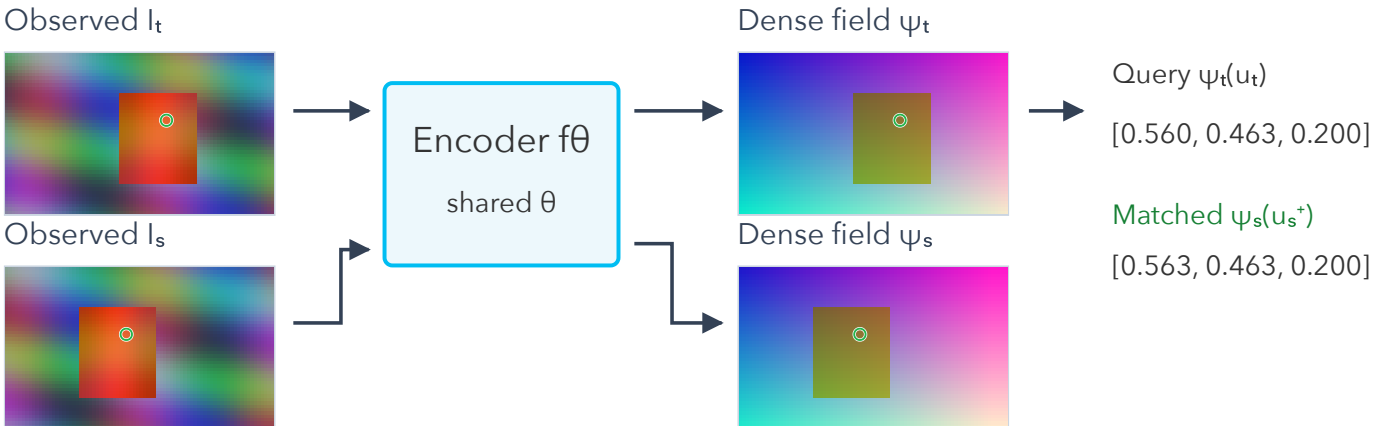


$$\boldsymbol{\psi}_t = f_\theta(I_t), \qquad \boldsymbol{\psi}_s = f_\theta(I_s)$$

Each image location has a feature vector.

Pixel pairs index local vectors in the dense encoder output.

Learning dense descriptors

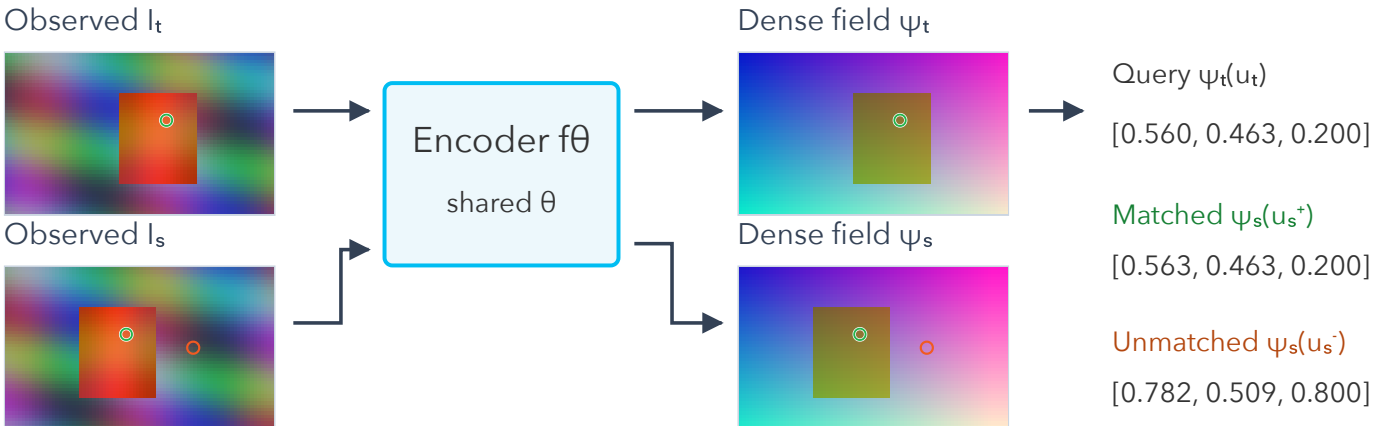


$$d_+ = \|\boldsymbol{\psi}_t(\mathbf{u}_t) - \boldsymbol{\psi}_s(\mathbf{u}_s^+)\|_2, \quad d_- = \|\boldsymbol{\psi}_t(\mathbf{u}_t) - \boldsymbol{\psi}_s(\mathbf{u}_s^-)\|_2$$

Matched distance $d_+ = 0.0033$

Matched pairs train their vectors to be close.

Learning dense descriptors



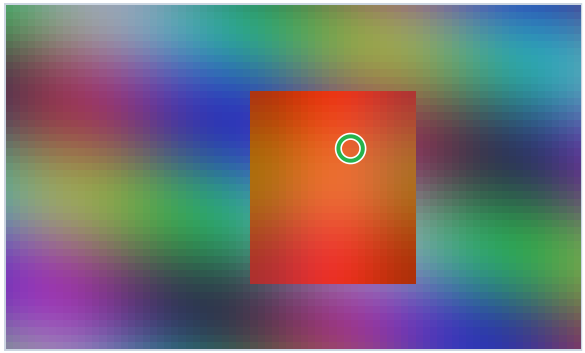
$$d_+ = \|\boldsymbol{\psi}_t(\mathbf{u}_t) - \boldsymbol{\psi}_s(\mathbf{u}_s^+)\|_2, \quad d_- = \|\boldsymbol{\psi}_t(\mathbf{u}_t) - \boldsymbol{\psi}_s(\mathbf{u}_s^-)\|_2$$

Matched distance $d_+ = 0.0033$ Unmatched distance $d_- = 0.6414$

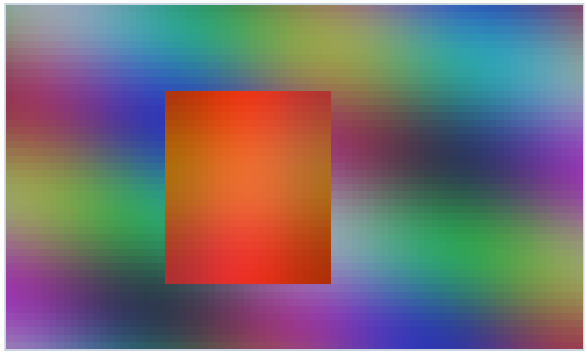
Nonmatching pairs provide a separation constraint.

Descriptor matching (searching for matches)

Query image I_t



Search image I_s

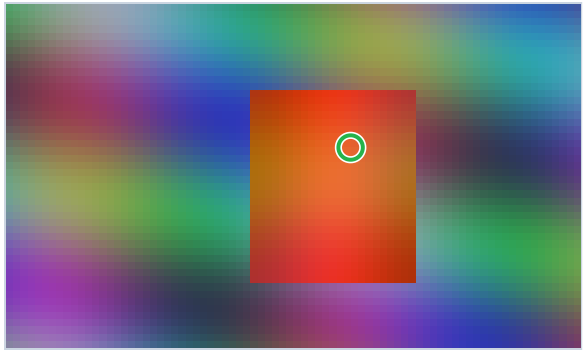


Choose a query descriptor $\psi_t(u_t)$ and a candidate field ψ_s.

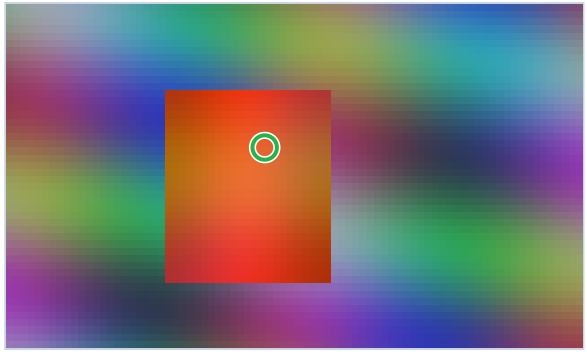
$$\mathbf{u}_s^* = \arg\min_{\mathbf{u}_s} \|\boldsymbol{\psi}_t(\mathbf{u}_t) - \boldsymbol{\psi}_s(\mathbf{u}_s)\|_2$$

Descriptor matching (searching for matches)

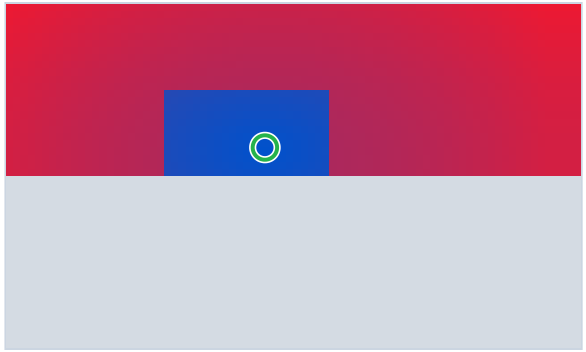
Query image I_t



Search image I_s



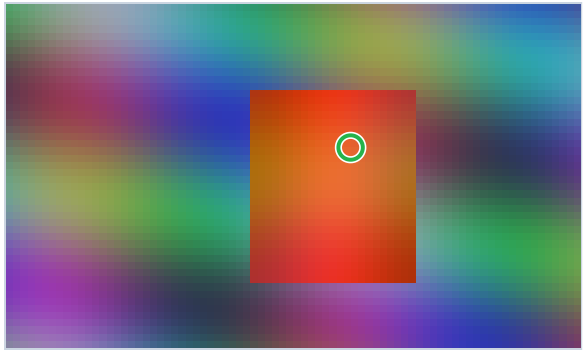
Distance · blue is closer



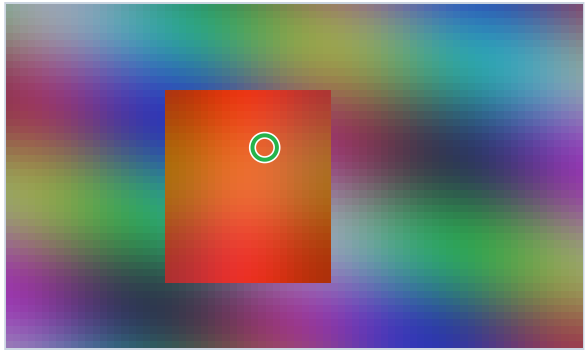
$$\mathbf{u}_s^* = \arg\min_{\mathbf{u}_s} \|\boldsymbol{\psi}_t(\mathbf{u}_t) - \boldsymbol{\psi}_s(\mathbf{u}_s)\|_2$$

Descriptor matching (searching for matches)

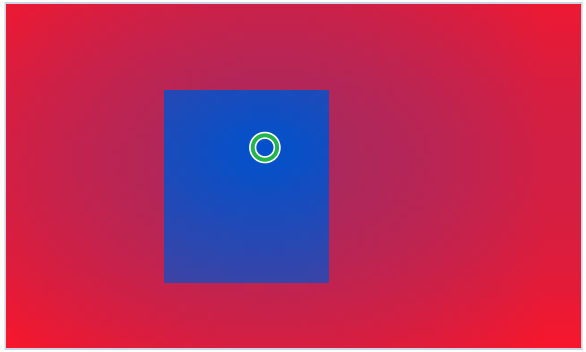
Query image I_t



Search image I_s



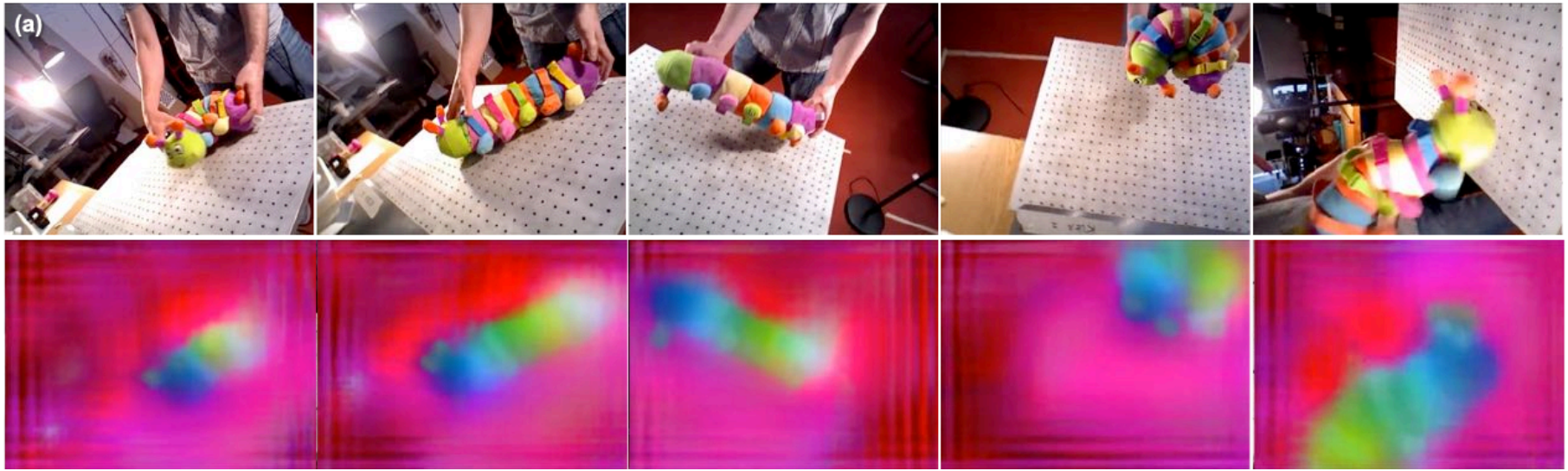
Distance · blue is closer



$$\mathbf{u}_s^* = \arg\min_{\mathbf{u}_s} \|\boldsymbol{\psi}_t(\mathbf{u}_t) - \boldsymbol{\psi}_s(\mathbf{u}_s)\|_2$$

Descriptor consistency across configurations

RGB observations above · dense descriptor colors below

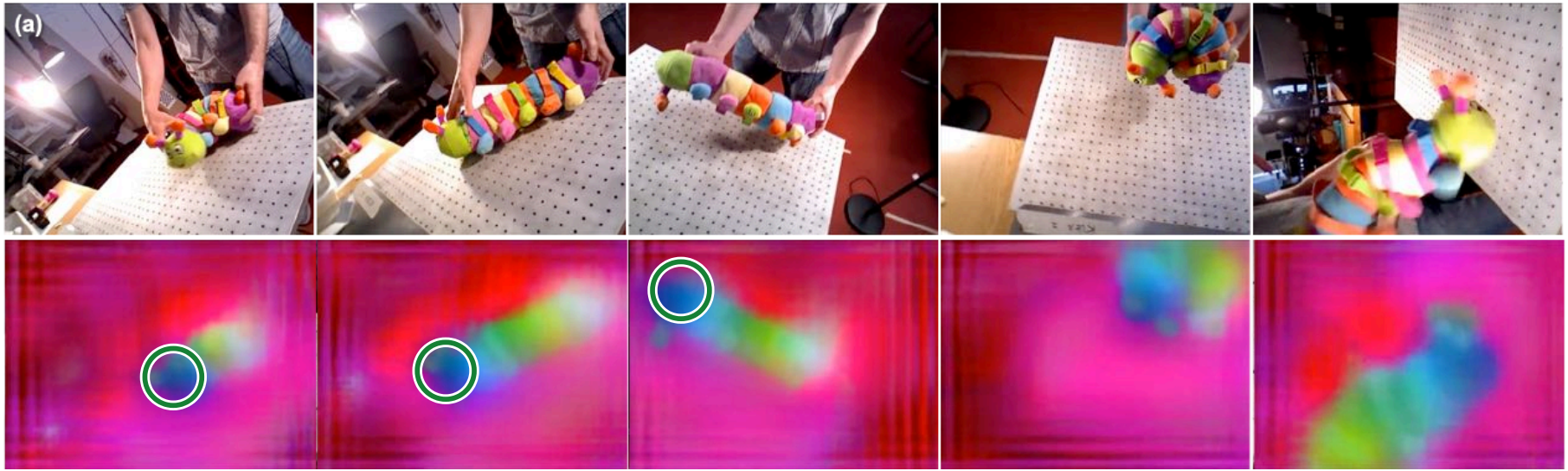


The same object changes configuration while corresponding regions retain similar feature colors.

Florence et al., Dense Object Nets, CoRL 2018 · Fig. 2(a), all five configurations · arXiv v2

Descriptor consistency across configurations

RGB observations above · dense descriptor colors below

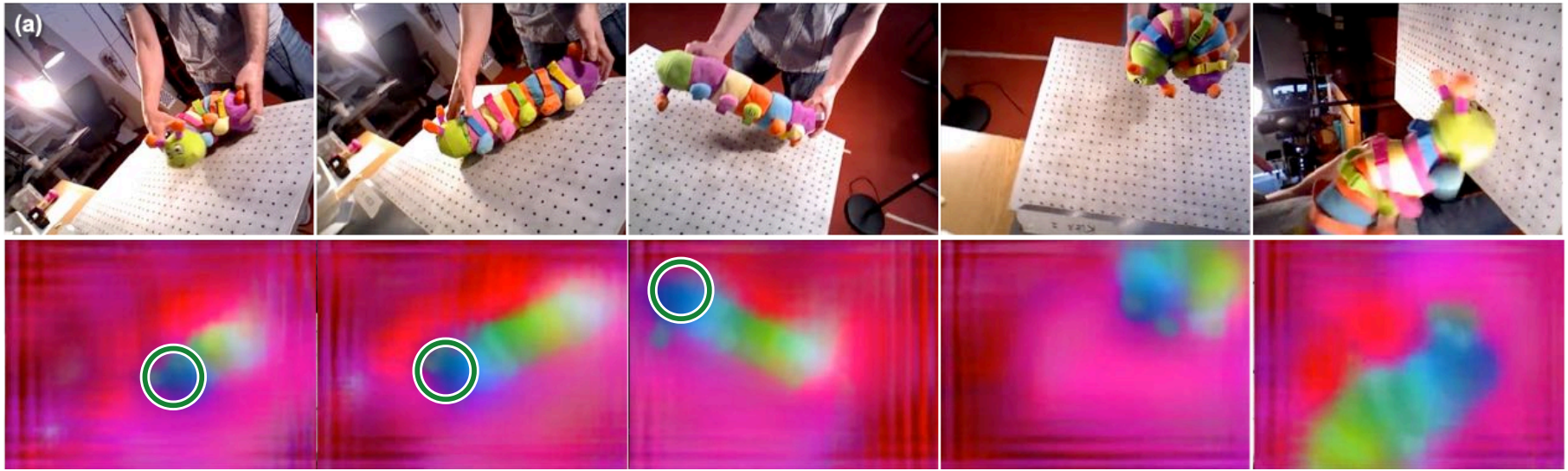


The same object changes configuration while corresponding regions retain similar feature colors.

Florence et al., Dense Object Nets, CoRL 2018 · Fig. 2(a), all five configurations · arXiv v2

Descriptor consistency across configurations

RGB observations above · dense descriptor colors below

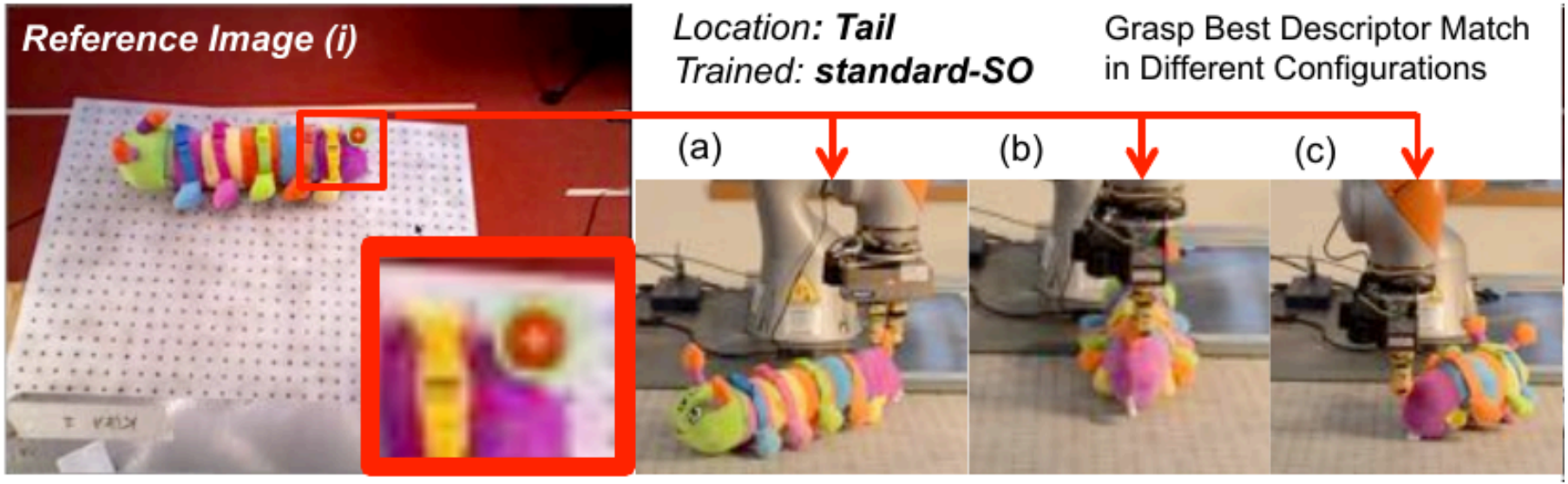


The same object changes configuration while corresponding regions retain similar feature colors.

> Feature colors encode descriptors, not manually assigned semantic part labels.

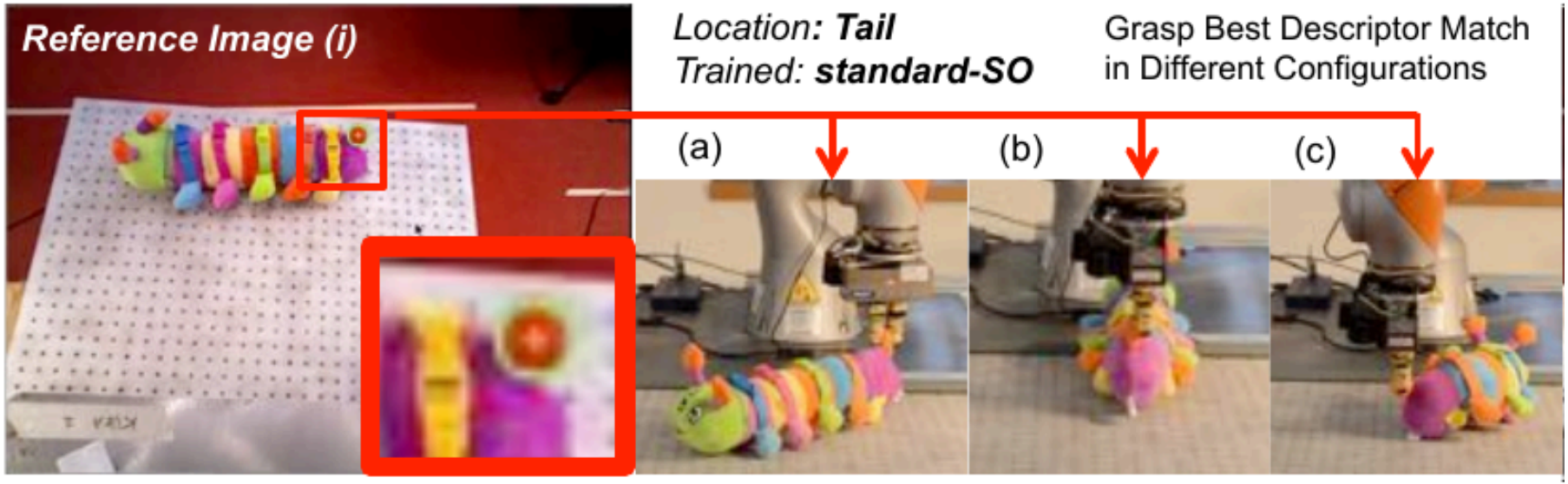
Florence et al., Dense Object Nets, CoRL 2018 · Fig. 2(a), all five configurations · arXiv v2

Finding a grasp point



Florence et al., Dense Object Nets, CoRL 2018 · Fig. 6(i), tail, standard-so · arXiv v2

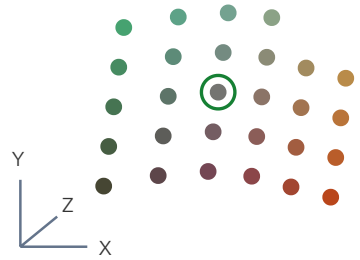
Finding a grasp point



Florence et al., Dense Object Nets, CoRL 2018 · Fig. 6(i), tail, standard-so · arXiv v2

Extension to 3D correspondence

Cloud A · frame a



28 points

Cloud B · frame b

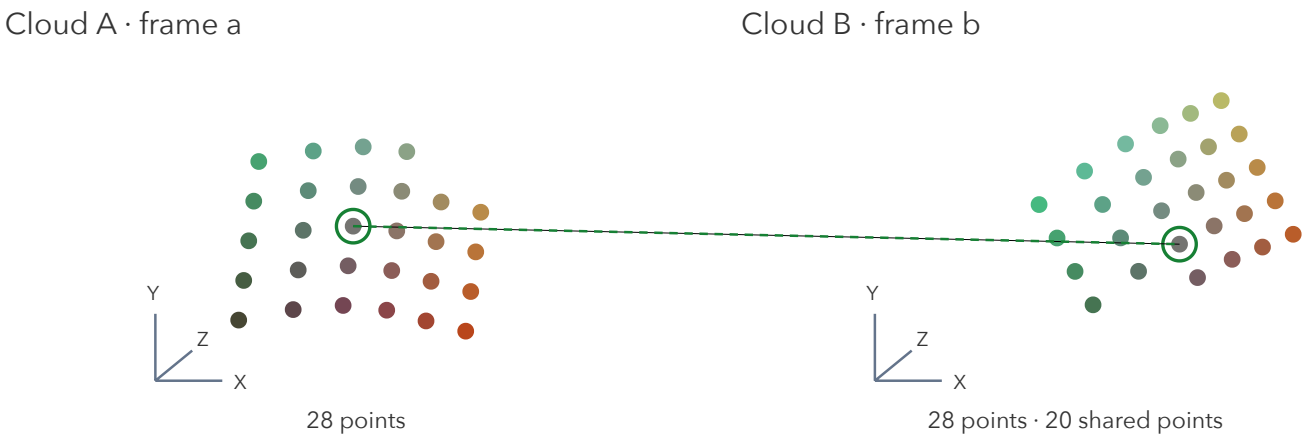


28 points · 20 shared points

Query p^a = [-0.18, -0.15, 0.49]

Overlapping clouds can describe some of the same physical surfaces.

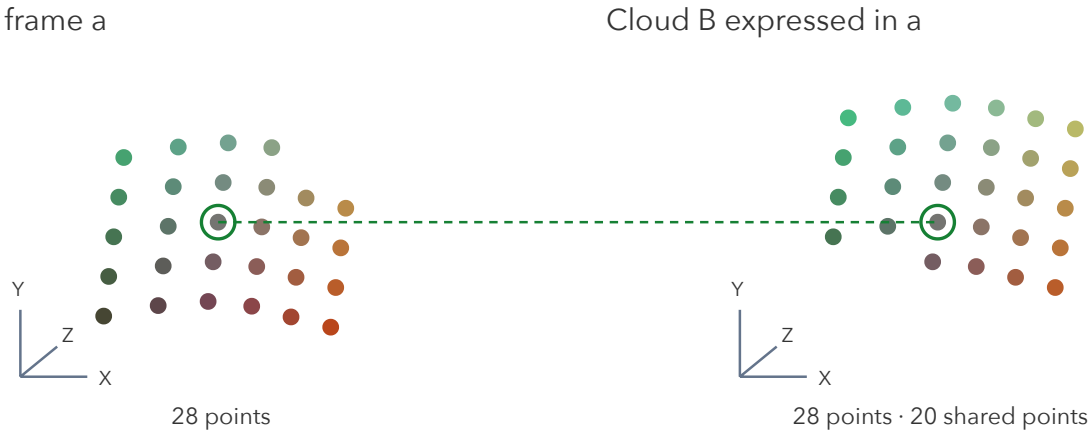
Extension to 3D correspondence



Query p^a = [-0.18, -0.15, 0.49] · matched p^b = [0.54, -0.48, 0.79]

Descriptor matches provide candidate point correspondences.

Extension to 3D correspondence



$$p^a = R_b^a p^b + t_b^a$$

Query p^a = [-0.18, -0.15, 0.49] · matched p^b = [0.54, -0.48, 0.79]

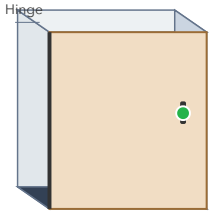
Several reliable, well-distributed correspondences can constrain a rigid registration.

SECTION 07

Learning action possibilities from experience

Observation, action, outcome, target

Earlier observation x_t



t = 0 · before the action

Commanded action a_t

Grasp handle, pull outward

Desired opening: ≥ 30°

Later sensor measurements o

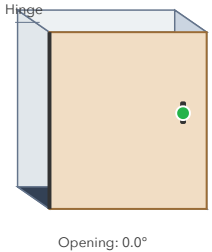
Contact detected · opening 55°

$$y_t = \Gamma(x_t, a_t, o_{t:t+\Delta})$$

The desired outcome is a task specification, not a measured training label.

Observation, action, outcome, target

Earlier observation x_t



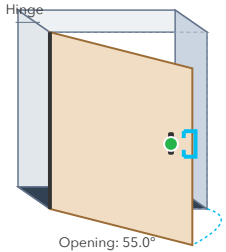
t = 0 · before the action

Commanded action a_t

Grasp handle, pull outward

Desired opening: ≥ 30°

Later sensor measurements o



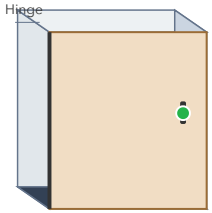
Contact detected · opening 55°

$$y_t = \Gamma(x_t, a_t, o_{t:t+\Delta})$$

Measurements after the action supply an automatic target.

Observation, action, outcome, target

Earlier observation x_t



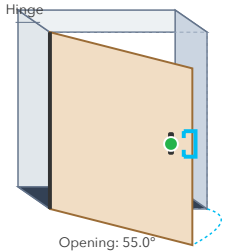
t = 0 · before the action

Commanded action a_t

Grasp handle, pull outward

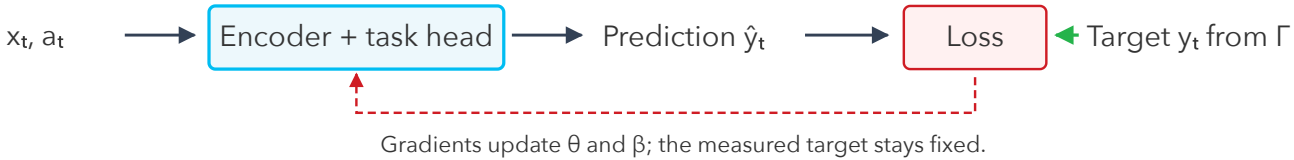
Desired opening: ≥ 30°

Later sensor measurements o



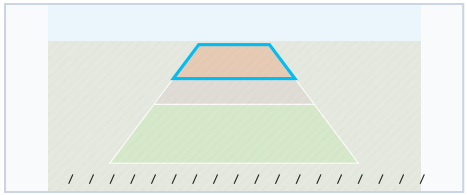
Contact detected · opening 55°

$$y_t = \Gamma(x_t, a_t, o_{t:t+\Delta})$$



Illustrative prediction 0.881 · measured target 1 · binary loss 0.1269

Traversability depends on the robot



Same selected mud patch · same 1.00 m/s command



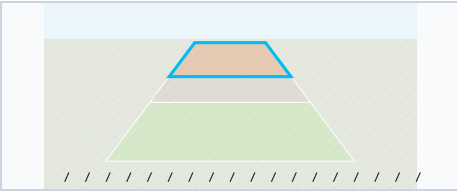
Narrow wheels / controller A



Broad tracks / controller B

Traversability depends on the platform, controller, action, and outcome rule.

Traversability depends on the robot



Same selected mud patch · same 1.00 m/s command



Narrow wheels / controller A

Measured 0.25 m/s

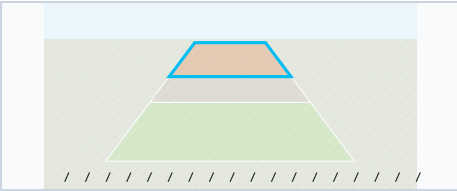


Broad tracks / controller B

Measured 0.65 m/s

Traversability depends on the platform, controller, action, and outcome rule.

Traversability depends on the robot



Same selected mud patch · same 1.00 m/s command



Narrow wheels / controller A

Measured 0.25 m/s

Loss: 0.75



Broad tracks / controller B

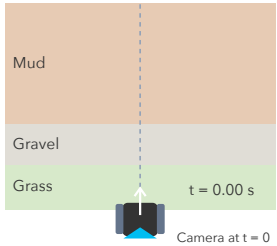
Measured 0.65 m/s

Loss: 0.35

Traversability depends on the platform, controller, action, and outcome rule.

Measuring traversal outcomes

Visited ground · top view



Earlier camera image I_0 · t = 0

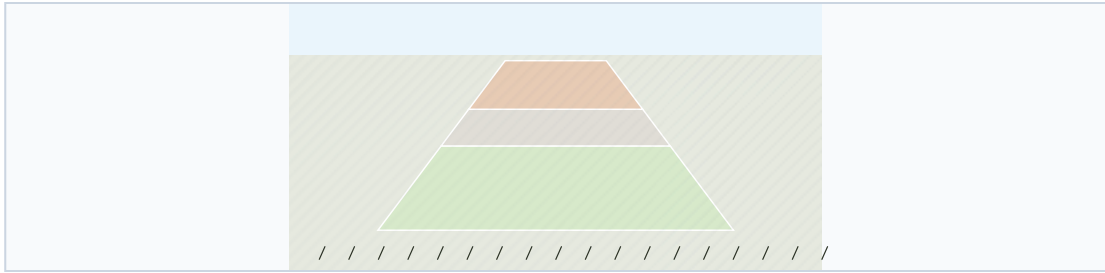


Image recorded → No outcome yet

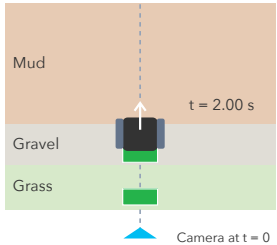
$$y = \Gamma(o) = \mathrm{clip}_{[0,1]}(1 - v_{\mathrm{meas}}/v_{\mathrm{cmd}})$$

Commanded 1.00 m/s · no later motion measurement yet

This run measures motion; the chosen proxy converts it into a cost.

Measuring traversal outcomes

Visited ground · top view



Earlier camera image I_0 · t = 0

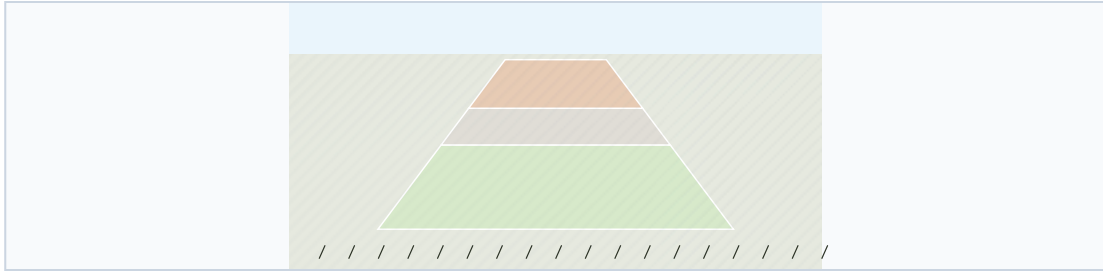


Image recorded → Outcome now available

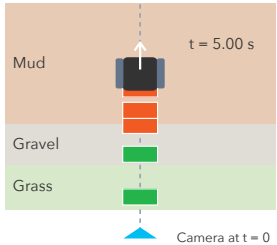
$$y = \Gamma(o) = \mathrm{clip}_{[0,1]}(1 - v_{\mathrm{meas}}/v_{\mathrm{cmd}})$$

Commanded 1.00 m/s · measured 0.80 m/s · loss y = 0.20

This run measures motion; the chosen proxy converts it into a cost.

Measuring traversal outcomes

Visited ground · top view



Earlier camera image I_0 · t = 0

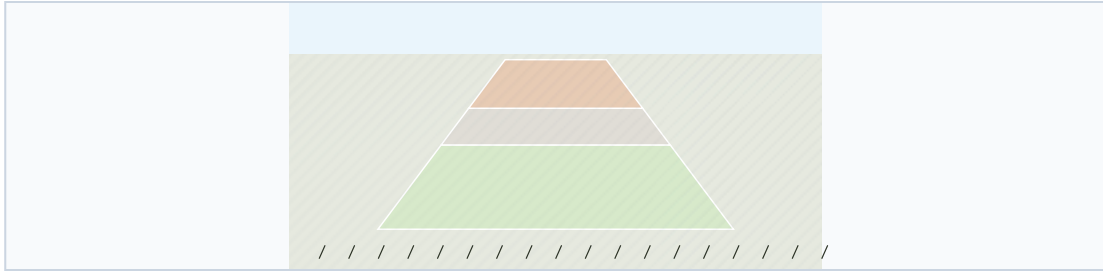


Image recorded → Outcome now available

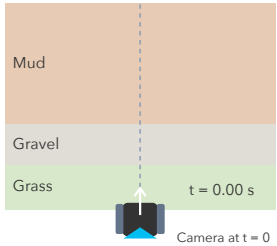
$$y = \Gamma(o) = \mathrm{clip}_{[0,1]}(1 - v_{\mathrm{meas}}/v_{\mathrm{cmd}})$$

Commanded 1.00 m/s · measured 0.60 m/s · loss y = 0.40

This run measures motion; the chosen proxy converts it into a cost.

Delayed supervision

Visited ground · top view



Earlier camera image I_0 · $t = 0$

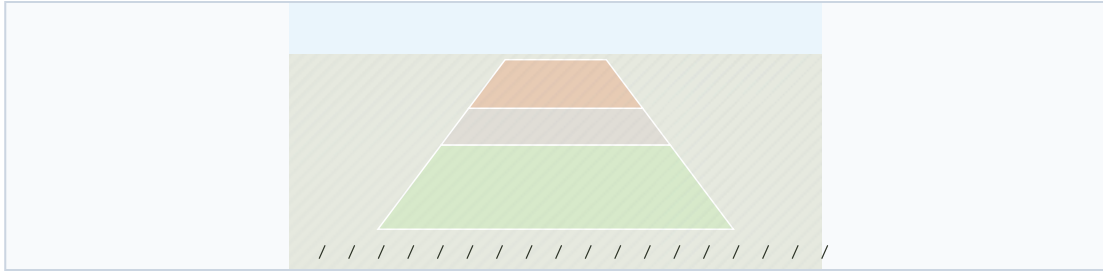


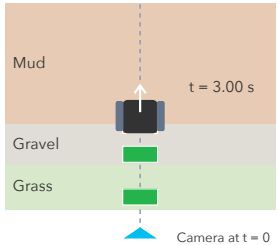
Image recorded → No outcome yet

Commanded 1.00 m/s · no later motion measurement yet

The earlier image is fixed. A target becomes available only after physical traversal.

Delayed supervision

Visited ground · top view



Earlier camera image I_0 · $t = 0$

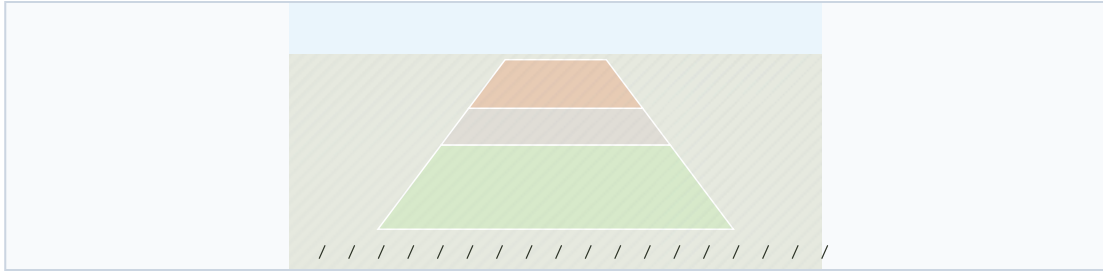


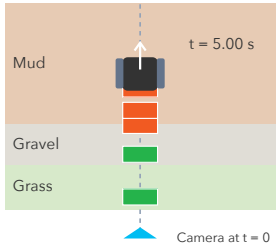
Image recorded → Outcome now available

Commanded 1.00 m/s · measured 0.35 m/s · loss $y = 0.65$

The earlier image is fixed. A target becomes available only after physical traversal.

Delayed supervision

Visited ground · top view



Earlier camera image I_0 · t = 0

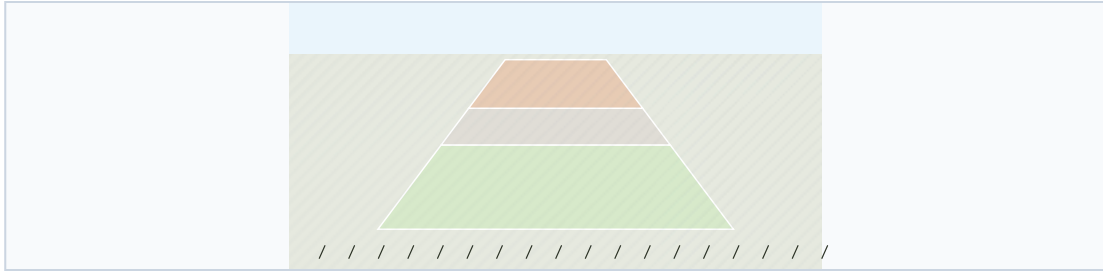


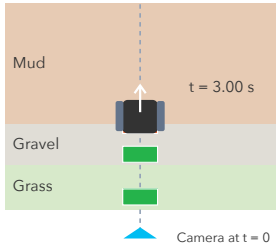
Image recorded → Outcome now available

Commanded 1.00 m/s · measured 0.60 m/s · loss y = 0.40

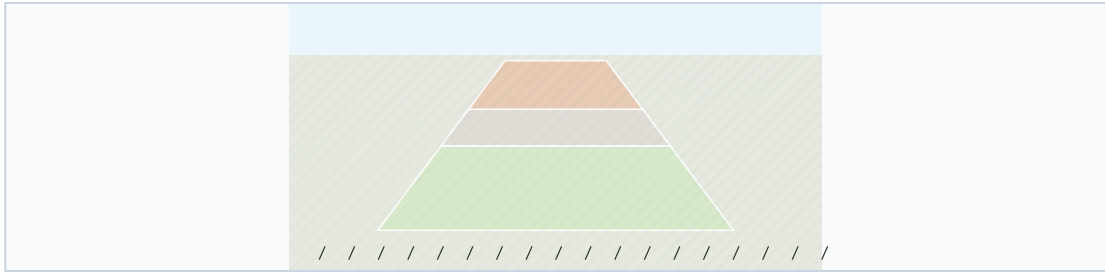
The earlier image is fixed. A target becomes available only after physical traversal.

Reprojecting experience

Visited ground · top view



Earlier camera image I_0 · t = 0



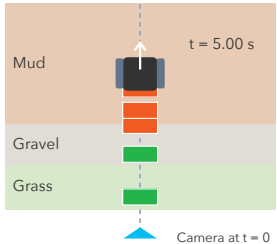
Observed lower cost Observed higher cost Unvisited

Commanded 1.00 m/s · measured 0.35 m/s · loss y = 0.65 · 0 projected targets · pose time offset 0.0 s

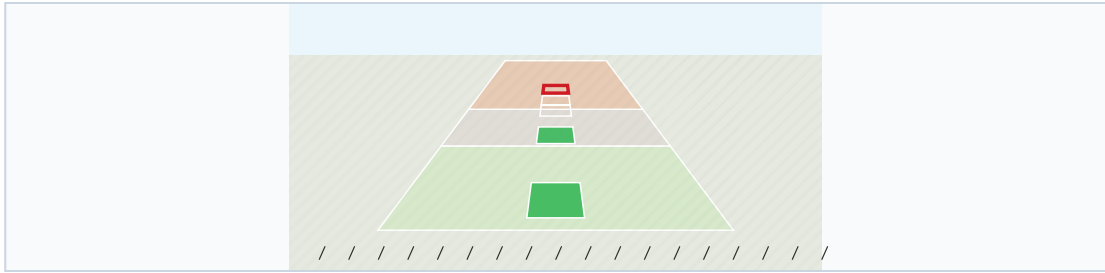
Use the measurement-time pose to reproject each visited footprint.

Reprojecting experience

Visited ground · top view



Earlier camera image I_0 · t = 0



Observed lower cost Observed higher cost Unvisited

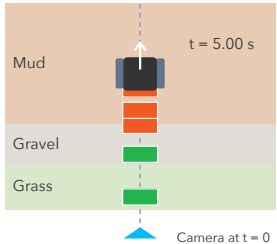
$$\mathbf{u}_0 = \pi\left(K_0\left(R_r^{c_0} p^r + t_r^{c_0}\right)\right)$$

Commanded 1.00 m/s · measured 0.60 m/s · loss y = 0.40 · 5 projected targets · pose time offset 0.0 s

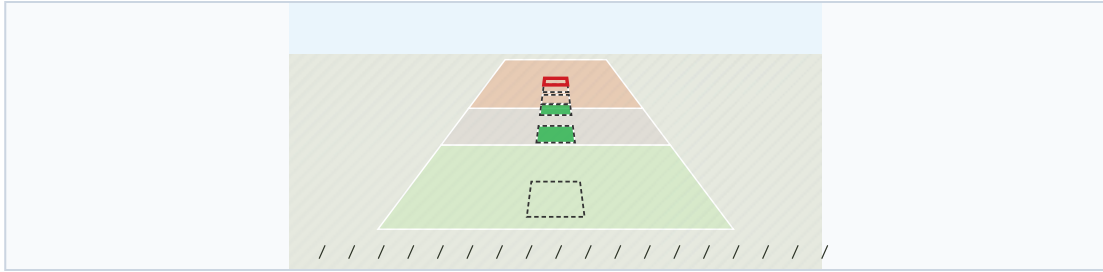
Use the measurement-time pose to reproject each visited footprint.

Reprojecting experience

Visited ground · top view



Earlier camera image I_0 · t = 0



Observed lower cost Observed higher cost Unvisited

Dashed outline: correctly associated footprint

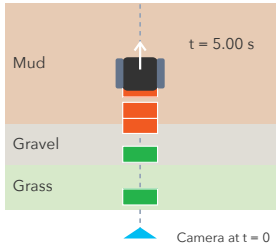
$$\mathbf{u}_0 = \pi\left(K_0\left(R_r^{c_0} p^r + t_r^{c_0}\right)\right)$$

Commanded 1.00 m/s · measured 0.60 m/s · loss y = 0.40 · 5 projected targets · pose time offset 1.0 s

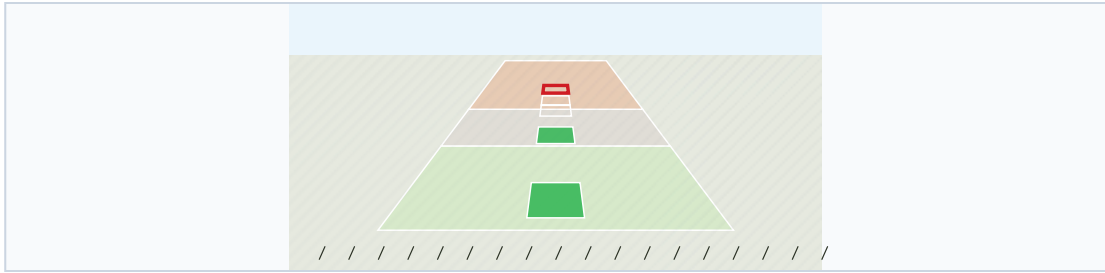
Wrong timestamp: the same measurement lands on a different visual region.

Reprojecting experience

Visited ground · top view



Earlier camera image I_0 · t = 0



Observed lower cost Observed higher cost Unvisited

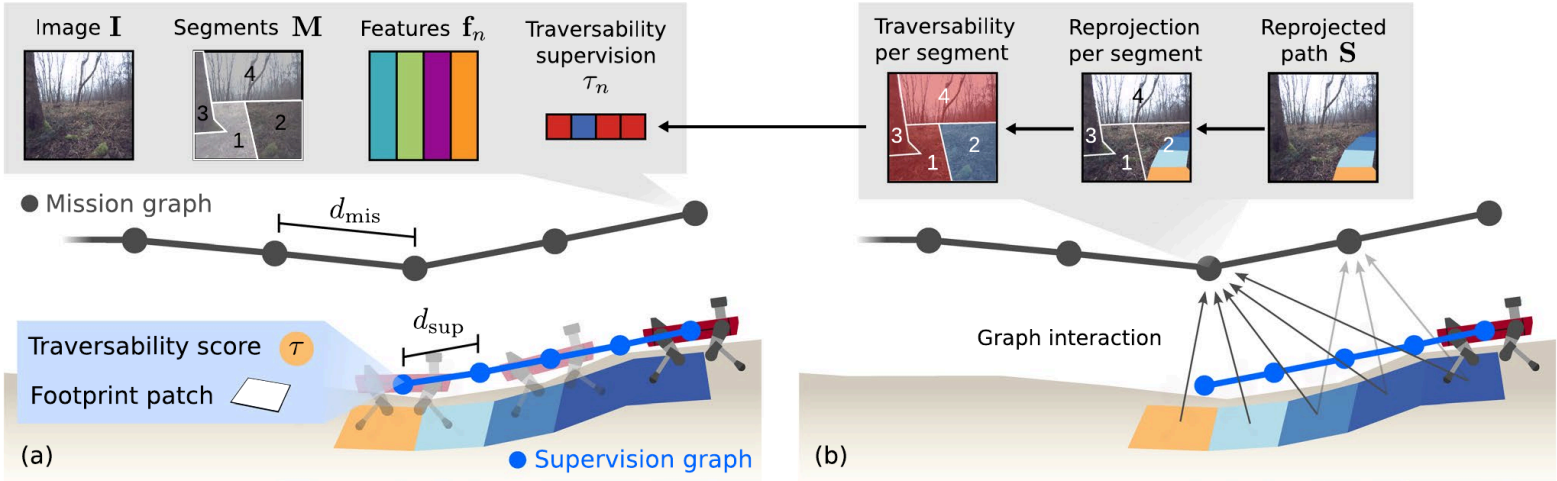
$$\mathbf{u}_0 = \pi\left(K_0\left(R_r^{c_0} p^r + t_r^{c_0}\right)\right)$$

Commanded 1.00 m/s · measured 0.60 m/s · loss y = 0.40 · 5 projected targets · pose time offset 0.0 s

Toy local estimate: 0.500 → measured target 0.40 · mean squared error 0.0690

Restore time alignment before using these measured targets.

Wild Visual Navigation



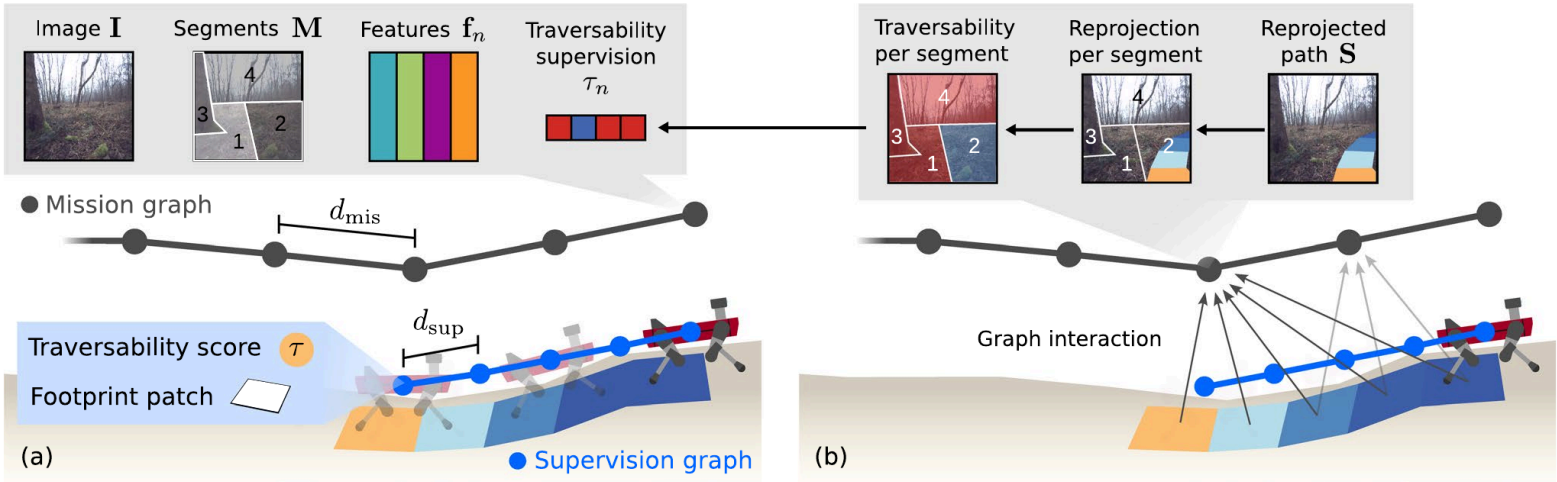
Visual input

RGB images and pretrained visual features

Experienced motion supplies targets for visual traversability estimation.

Mattamala et al., WVN, Autonomous Robots 2025 · Fig. 5(a,b), journal PDF page 6

Wild Visual Navigation



Visual input

RGB images and pretrained visual features

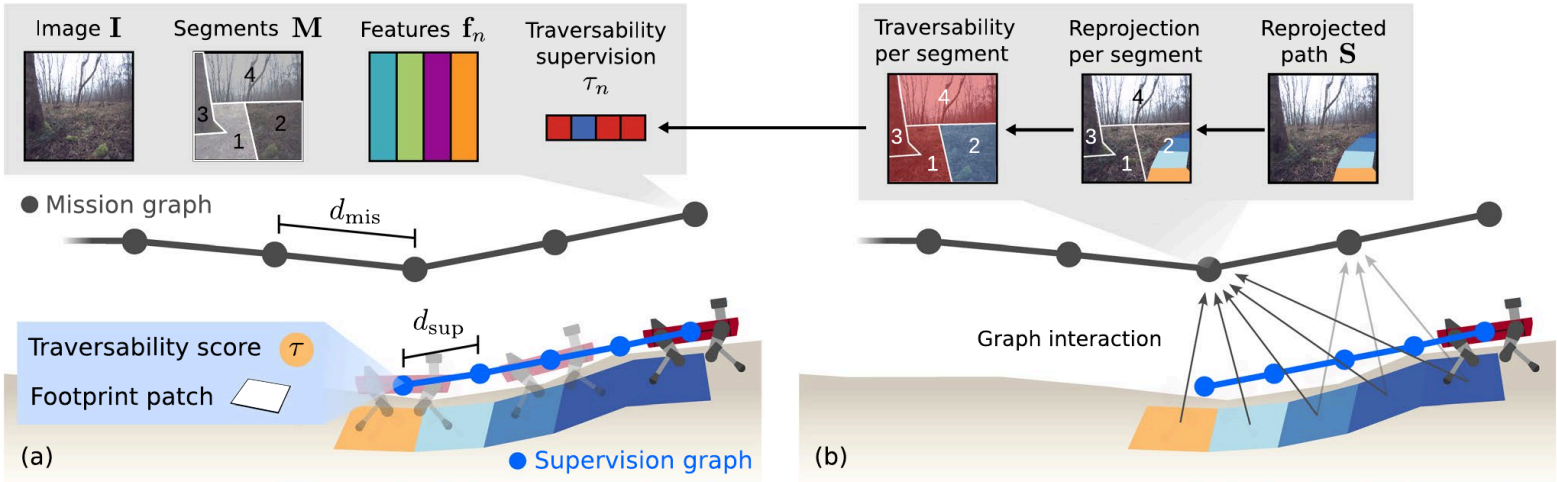
Measured signal

Commanded versus estimated planar velocity

Experienced motion supplies targets for visual traversability estimation.

Mattamala et al., WVN, Autonomous Robots 2025 · Fig. 5(a,b), journal PDF page 6

Wild Visual Navigation



Visual input

RGB images and pretrained visual features

Measured signal

Commanded versus estimated planar velocity

Accumulated supervision

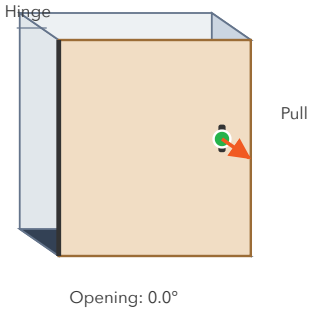
Footprint scores reprojected into stored images

Experienced motion supplies targets for visual traversability estimation.

Mattamala et al., WVN, Autonomous Robots 2025 · Fig. 5(a,b), journal PDF page 6

Manipulation affordances

Fixed initial observation x_t



Pull outward

Initial opening direction

Along the closed door

Little opening moment

Push inward

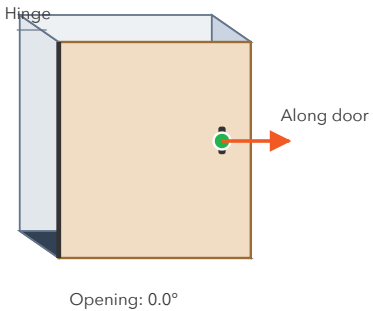
Closed stop blocks motion

Contact radius 0.85 m · direction F = [0.00, -1.00, 0.00] · fixed grasp orientation

The same handle supports different outcomes for different action directions.

Manipulation affordances

Fixed initial observation x_t



Pull outward

Initial opening direction

Along the closed door

Little opening moment

Push inward

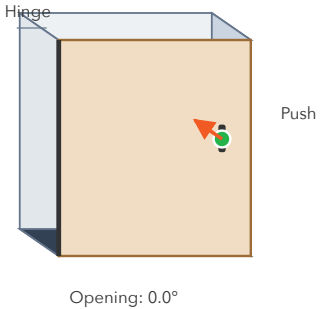
Closed stop blocks motion

Contact radius 0.85 m · direction F = [1.00, 0.00, 0.00] · fixed grasp orientation

The same handle supports different outcomes for different action directions.

Manipulation affordances

Fixed initial observation x_t



Pull outward

Initial opening direction

Along the closed door

Little opening moment

Push inward

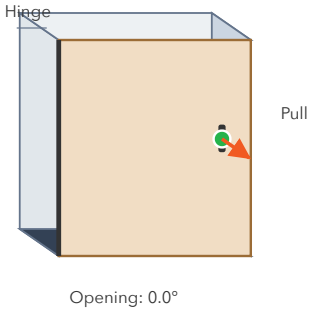
Closed stop blocks motion

Contact radius 0.85 m · direction F = [0.00, 1.00, 0.00] · fixed grasp orientation

The same handle supports different outcomes for different action directions.

Predicting an action's outcome

Fixed initial observation x_t



Pull outward

Initial opening direction

Along the closed door

Little opening moment

Push inward

Closed stop blocks motion

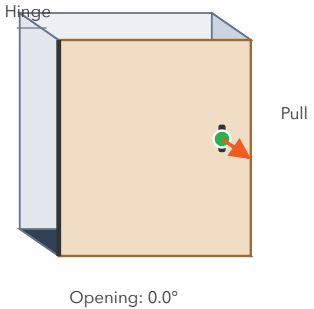
Contact radius 0.85 m · direction F = [0.00, -1.00, 0.00] · fixed grasp orientation

$$\hat{y}_t = h_\beta(f_\theta(x_t), a_t)$$

Predict from the observation and proposed action before seeing the outcome.

Predicting an action's outcome

Fixed initial observation x_t



Pull outward

Initial opening direction

Along the closed door

Little opening moment

Push inward

Closed stop blocks motion

Contact radius 0.85 m · direction F = [0.00, -1.00, 0.00] · fixed grasp orientation

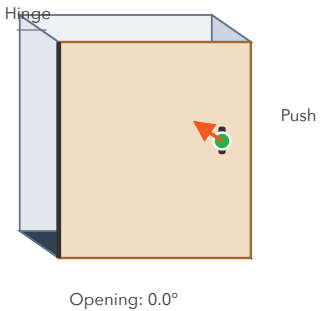
$$\hat{y}_t = h_\beta(f_\theta(x_t), a_t)$$

Predicted opening success: 0.881

Predict from the observation and proposed action before seeing the outcome.

Predicting an action's outcome

Fixed initial observation x_t



Pull outward

Initial opening direction

Along the closed door

Little opening moment

Push inward

Closed stop blocks motion

Contact radius 0.85 m · direction F = [0.00, 1.00, 0.00] · fixed grasp orientation

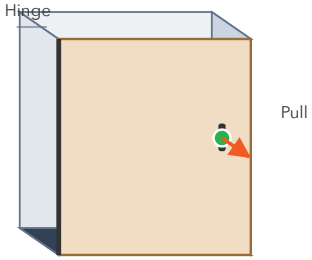
$$\hat{y}_t = h_\beta(f_\theta(x_t), a_t)$$

Predicted opening success: 0.018

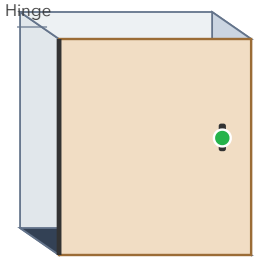
Predict from the observation and proposed action before seeing the outcome.

Automatically constructing an affordance target

Fixed initial observation x_t



Executed observation · not executed

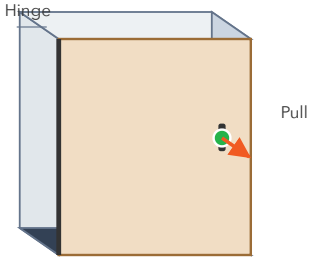


Contact radius 0.85 m · direction F = [0.00, -1.00, 0.00] · fixed grasp orientation

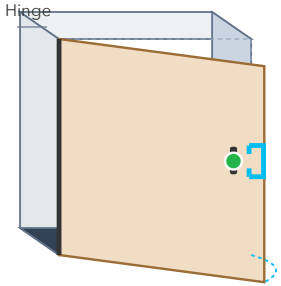
Task objective: open the door by at least 30°.

Automatically constructing an affordance target

Fixed initial observation x_t



Executed observation · in progress



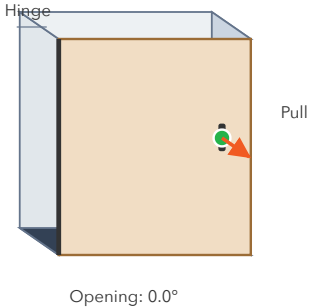
Contact radius 0.85 m · direction F = [0.00, -1.00, 0.00] · fixed grasp orientation

Predicted opening success: 0.881 · Target pending until execution finishes

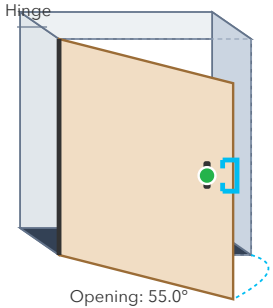
Task objective: open the door by at least 30°.

Automatically constructing an affordance target

Fixed initial observation x_t



Executed observation · complete



Contact radius 0.85 m · direction F = [0.00, -1.00, 0.00] · fixed grasp orientation

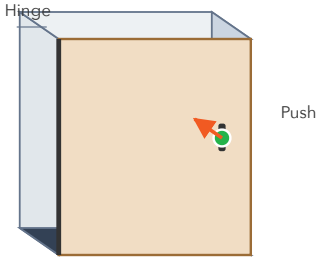
$$y_t = \Gamma(o) = \mathbf{1}[\text{measured opening} \geq 30^\circ]$$

Predicted opening success: 0.881 · Measured opening 55.0° → automatic target y = 1

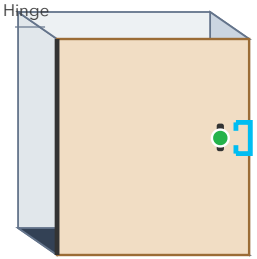
Task objective: open the door by at least 30°.

Automatically constructing an affordance target

Fixed initial observation x_t



Executed observation · complete



Contact radius 0.85 m · direction F = [0.00, 1.00, 0.00] · fixed grasp orientation

$$y_t = \Gamma(o) = \mathbf{1}[\text{measured opening} \geq 30^\circ]$$

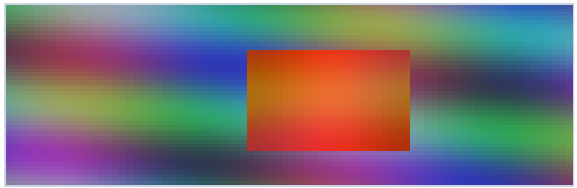
Predicted opening success: 0.018 · Measured opening 0.0° → automatic target y = 0

Task objective: open the door by at least 30°.

Summary: the learning principle

Geometric association

Recorded camera view



Depth + poses + visibility
→ corresponding pixels

Traversal experience

Command + measured motion
→ local cost targets

Cabinet interaction

Measured task outcome
→ success target

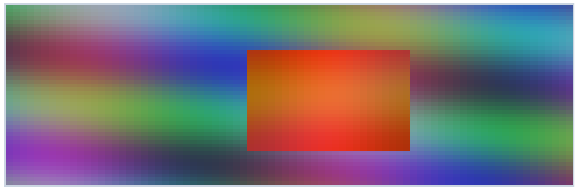
$$\hat{y}_t = h_\beta(f_\theta(x_t), a_t), \qquad y_t = \Gamma(x_t, a_t, o_{t:t+\Delta})$$

> Identify what supplies the target, which observations it applies to, and what the model must predict.

Summary: the learning principle

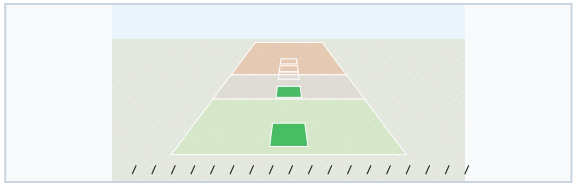
Geometric association

Recorded camera view



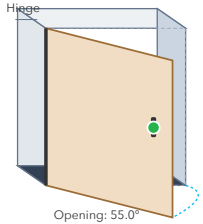
Depth + poses + visibility
→ corresponding pixels

Traversal experience



Command + measured motion
→ local cost targets

Cabinet interaction



Measured task outcome
→ success target

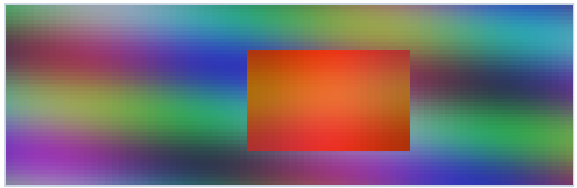
$$\hat{y}_t = h_\beta(f_\theta(x_t), a_t), \qquad y_t = \Gamma(x_t, a_t, o_{t:t+\Delta})$$

> Identify what supplies the target, which observations it applies to, and what the model must predict.

Summary: the learning principle

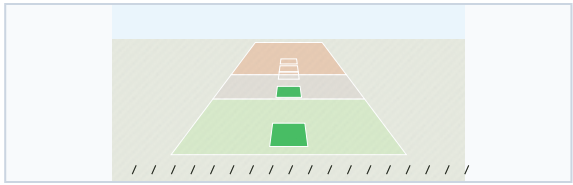
Geometric association

Recorded camera view



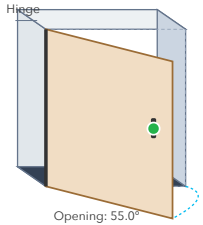
Depth + poses + visibility
→ corresponding pixels

Traversal experience



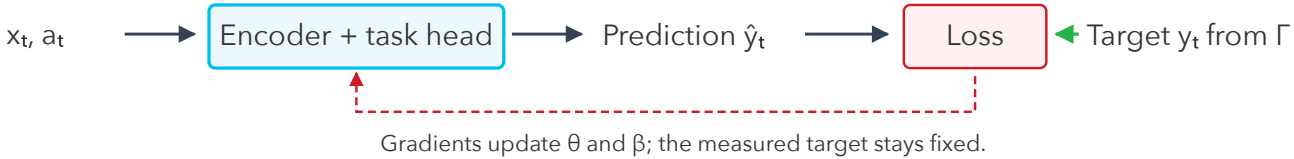
Command + measured motion
→ local cost targets

Cabinet interaction



Measured task outcome
→ success target

$$\hat{y}_t = h_\beta(f_\theta(x_t), a_t), \qquad y_t = \Gamma(x_t, a_t, o_{t:t+\Delta})$$



Identify what supplies the target, which observations it applies to, and what the model must predict.

SECTION 08

Model adaptation

Why adapt models in robotics?

A model trained in one context may lose performance when that context changes.

🌍 Diverse environments

Indoor, outdoor, underwater, aerial

What changes: lighting, visibility, obstacles, disturbances

🎯 Diverse tasks

Navigation, manipulation, inspection, exploration

What changes: objectives, accuracy, safety constraints

🤖 Diverse robots

Wheeled robots, quadrupeds, arms, drones

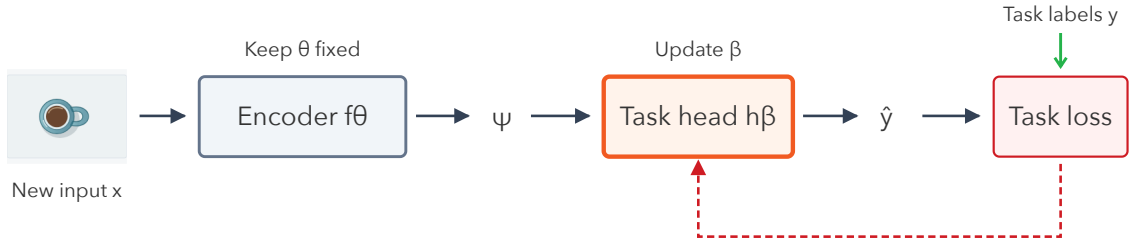
What changes: sensors, actuators, morphology, dynamics

Adapt models to transfer what they have learned to new conditions

Maintain performance while limiting the data and retraining required.

Adaptation

Freeze encoder · train task head



Dark: prediction · green: labels · dashed red: gradients

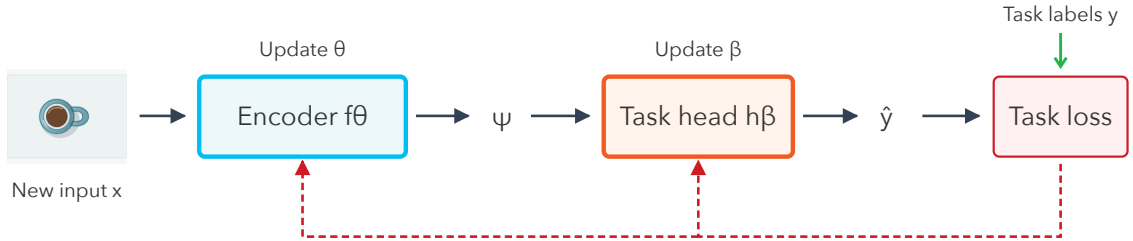
$$\theta = \text{fixed}, \quad \Theta = (\beta)$$

For a linear probe, hβ is linear. A frozen encoder can also feed a nonlinear head.

Chen et al., SimCLR, ICML 2020 · Appendix B.8

Adaptation

Fine-tune encoder + task head



Dark: prediction · green: labels · dashed red: gradients

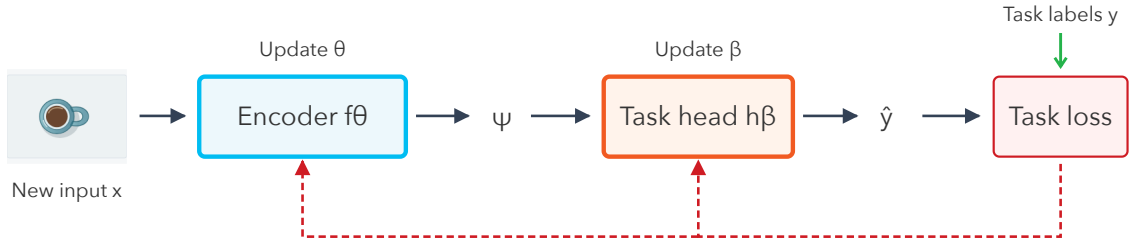
$$\Theta = (\theta, \beta), \quad \text{update encoder and head}$$

For a linear probe, hβ is linear. A frozen encoder can also feed a nonlinear head.

Chen et al., SimCLR, ICML 2020 · Appendix B.8

Adaptation

Fine-tune encoder + task head



Dark: prediction · green: labels · dashed red: gradients

$$\Theta = (\theta, \beta), \quad \text{update encoder and head}$$

Frozen features

Evaluate information accessible to the chosen head.

Fine-tuning

Adapt features and head to the target task.

For a linear probe, hβ is linear. A frozen encoder can also feed a nonlinear head.

Chen et al., SimCLR, ICML 2020 · Appendix B.8

LoRA: adapt a model with few parameters

Low-Rank Adaptation – learn a low-rank correction while keeping the original weights frozen.

$$W_{\text{adapted}} = \underbrace{W_0}_{\text{frozen weights}} + \underbrace{\Delta W}_{BA\,:\,\text{learned correction}}$$

⚙️ Lightweight adaptation

Only two small matrices, **A and B**, are trained.

Fewer parameters to optimize and less training memory.

🔒 Preserved model

The original weights **remain unchanged** during adaptation.

Disable the correction to **recover the initial model**.

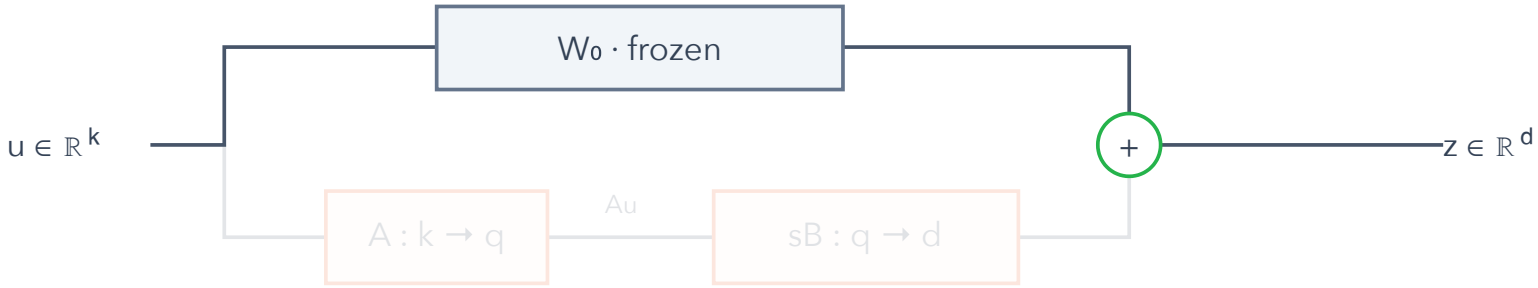
🔄 Modular adaptations

One base model can support **several corrections**.

Load the adaptation for the **domain, task, or robot**.

One shared base model + a small correction for each context

LoRA: Low-Rank Adaptation



$$z = W_0 u + sB(Au), \qquad W_0 \in \mathbb{R}^{d\times k},\ A \in \mathbb{R}^{q\times k},\ B \in \mathbb{R}^{d\times q},\ s = \alpha/q$$

A: k → q

Measure q learned combinations of the input coordinates.

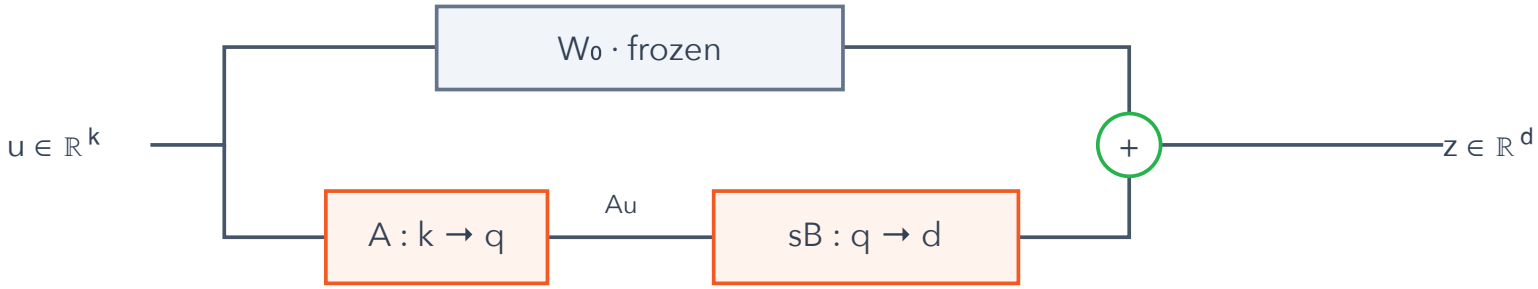
B: q → d

Use those q coefficients to form a correction in the output space.

Both branches receive the same u; add their outputs.

Hu et al. · LoRA · ICLR 2022 · §4

LoRA: Low-Rank Adaptation



$$z = W_0 u + sB(Au), \qquad W_0 \in \mathbb{R}^{d\times k},\ A \in \mathbb{R}^{q\times k},\ B \in \mathbb{R}^{d\times q},\ s = \alpha/q$$

A: k → q

Measure q learned combinations of the input coordinates.

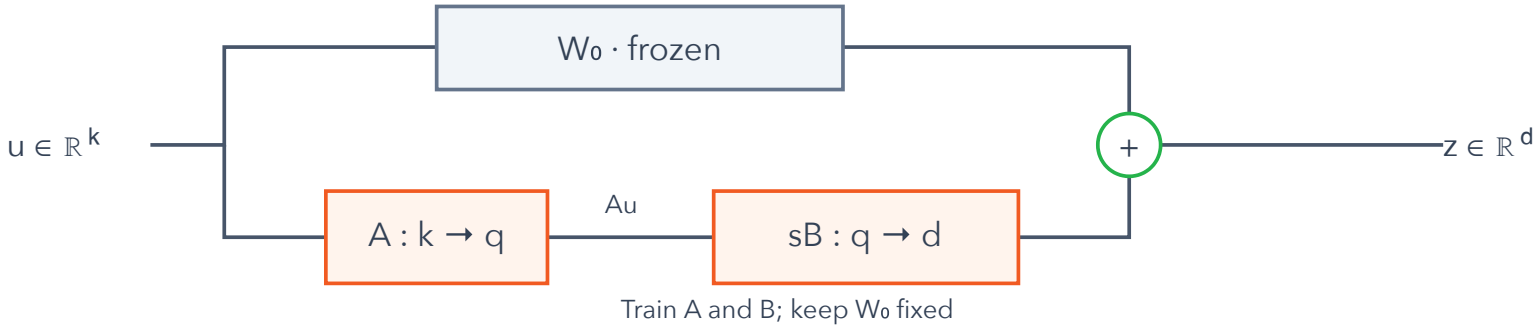
B: q → d

Use those q coefficients to form a correction in the output space.

Both branches receive the same u; add their outputs.

Hu et al. · LoRA · ICLR 2022 · §4

LoRA: Low-Rank Adaptation



$$z = W_0 u + sB(Au), \qquad W_0 \in \mathbb{R}^{d\times k},\ A \in \mathbb{R}^{q\times k},\ B \in \mathbb{R}^{d\times q},\ s = \alpha/q$$

A: k → q

Measure q learned combinations of the input coordinates.

B: q → d

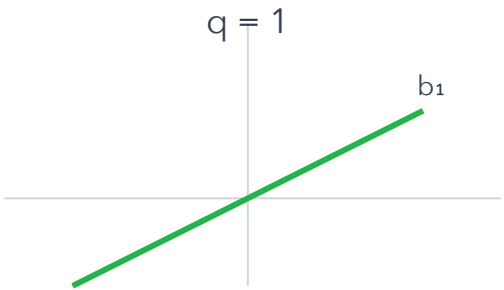
Use those q coefficients to form a correction in the output space.

> Both branches receive the same u; add their outputs.

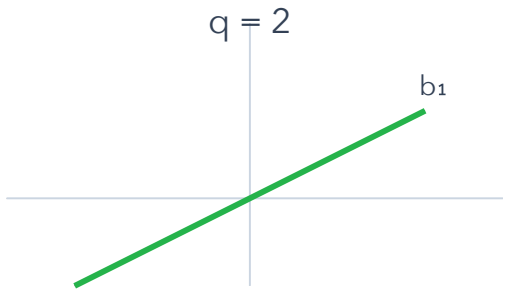
Hu et al. · LoRA · ICLR 2022 · §4

What does “low rank” constrain?

$$\Delta W = sBA = s\sum_{j=1}^{q} b_j a_j^\top, \qquad \Delta W u = s\sum_{j=1}^{q} b_j (a_j^\top u)$$



One correction direction



Up to two independent directions

Read a direction

Each row $a_j^\top$ extracts one scalar from u.

Write a direction

Each column b_j supplies one output direction.

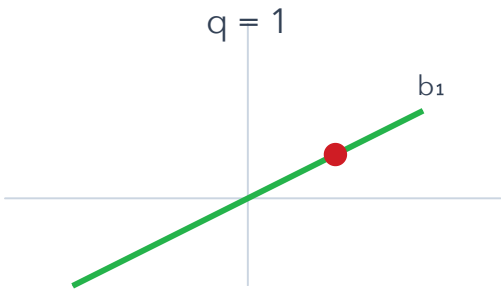
> rank(ΔW) ≤ q: every correction lies in span(b₁,…,bq). The directions themselves are learned.

The rank constraint applies to ΔW. The effective matrix W₀ + ΔW can remain full rank.

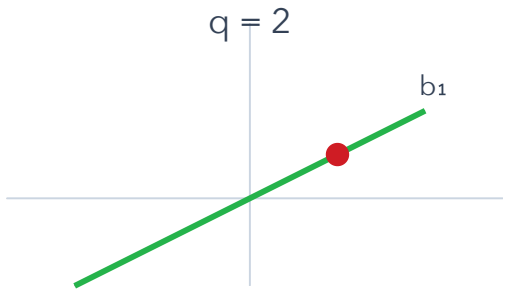
Hu et al. · LoRA · ICLR 2022 · §4

What does “low rank” constrain?

$$\Delta W = sBA = s\sum_{j=1}^{q} b_j a_j^\top, \qquad \Delta W u = s\sum_{j=1}^{q} b_j (a_j^\top u)$$



One correction direction



Up to two independent directions

Read a direction

Each row $a_j^\top$ extracts one scalar from u.

Write a direction

Each column b_j supplies one output direction.

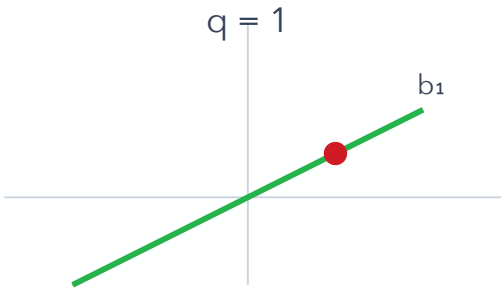
> rank(ΔW) ≤ q: every correction lies in span(b₁,…,bq). The directions themselves are learned.

The rank constraint applies to ΔW. The effective matrix W₀ + ΔW can remain full rank.

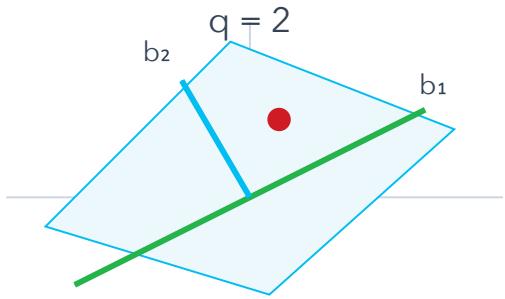
Hu et al. · LoRA · ICLR 2022 · §4

What does “low rank” constrain?

$$\Delta W = sBA = s\sum_{j=1}^{q} b_j a_j^\top, \qquad \Delta W u = s\sum_{j=1}^{q} b_j (a_j^\top u)$$



One correction direction



Up to two independent directions

Read a direction

Each row $a_j^\top$ extracts one scalar from u.

Write a direction

Each column b_j supplies one output direction.

rank(ΔW) ≤ q: every correction lies in span(b₁,…,bq). The directions themselves are learned.

The rank constraint applies to ΔW. The effective matrix W₀ + ΔW can remain full rank.

Hu et al. · LoRA · ICLR 2022 · §4

How many parameters do we train?

589,824

Full 768 × 768 matrix

12,288

A + B with q = 8

48×

Fewer trainable parameters

$$N_{\text{full}} = dk, \qquad N_{\text{LoRA}} = q(k+d), \qquad \frac{N_{\text{LoRA}}}{N_{\text{full}}} \approx 2.08\%$$

Low q

Less adapter storage and fewer optimizer states; more restricted corrections.

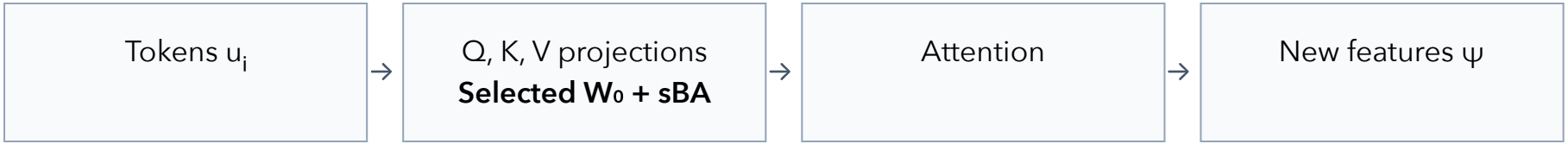
High q

More capacity and cost

W_0 remains resident. Activations and gradient propagation through the encoder still cost memory and compute.

Hu et al. · LoRA · ICLR 2022 · §4

Where does LoRA act in a Transformer?



$$Q_i = (W_{Q0} + sB_Q A_Q)u_i, \quad V_i = (W_{V0} + sB_V A_V)u_i, \quad K_i = W_{K0}u_i$$

Change Q

Modify which tokens the representation attends to.

Change V

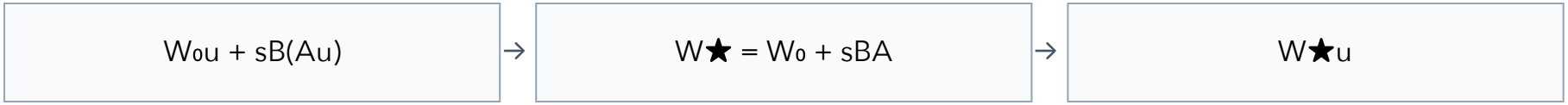
Modify the information carried by the attended tokens.

> The original paper studies Q/V adaptation. K, output projections and MLP matrices are also possible targets.

Choose layers, matrices, rank and scale on held-out data. Every adapted matrix has its own compatible factors; q is our rank notation.

Hu et al. · LoRA · ICLR 2022 · §4.2 · §7.1

At deployment, fold the correction into the weights



Before merging

Store one base model and compact task-specific adapters; switch adapters as needed.

After merging

One ordinary matrix multiplication. No separate LoRA branch for that fixed task.

$$(W_0 + sBA)u = W_0u + sB(Au)$$

This merges a correction into its own base weights. Combining unrelated models or several task adapters is a different problem.

Hu et al. · LoRA · ICLR 2022 · §4

Head only, LoRA, or full fine-tuning?

Head only

Fixed encoder. Cheap training; cache features if preprocessing is fixed. Limited by the available representation.

LoRA + head

Adapt internal features with few parameters. Still backpropagate through the encoder; choose rank and insertion sites.

Full fine-tuning

Most flexible updates. More gradients and optimizer states; greater storage and overfitting risk.

Hu et al. · LoRA · ICLR 2022 · §4

Head only, LoRA, or full fine-tuning?

Head only

Fixed encoder. Cheap training; cache features if preprocessing is fixed. Limited by the available representation.

LoRA + head

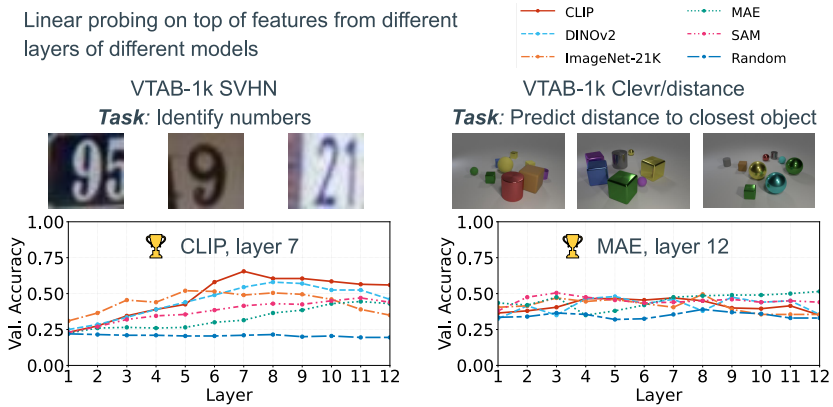
Adapt internal features with few parameters. Still backpropagate through the encoder; choose rank and insertion sites.

Full fine-tuning

Most flexible updates. More gradients and optimizer states; greater storage and overfitting risk.

Hu et al. · LoRA · ICLR 2022 · §4

Useful features may be hidden in different layers



Digits: CLIP, layer 7

Distance: MAE, layer 12

One frozen model

Its final layer need not be the best representation for the new task.

Several frozen models

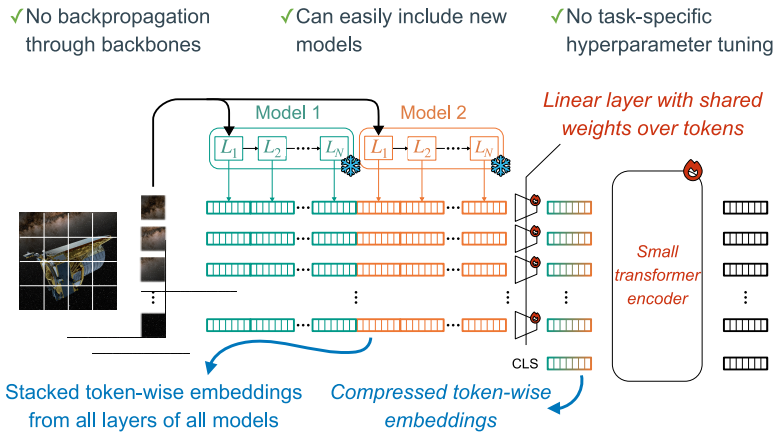
Different pretraining objectives can provide complementary information.

> ComBo learns to read intermediate features from one or several backbones.

Original project figure: validation linear-probe accuracy versus layer. Best model/layer differs between these tasks.

Ramtoula et al. · ComBo · NeurIPS 2025 · page projet / project figure

ComBo: combine features from multiple backbones



For a given task (e.g., segmentation or traversability), we train a small Transformer to combine features from different layers of several models. The result is a mixture of useful, task-specific features.

> Fusion happens in feature space. The original backbones remain separate and frozen.

Ramtoula et al. · ComBo · NeurIPS 2025 · fig. 3 · §3

Choose where the adaptation should happen

Head only

Read the final frozen features. Lowest training burden when they already suffice.

LoRA

Change internal computations of a backbone. Compact task updates and mergeable linear branches.

ComBo

Read and combine intermediate features from several models.

Choose where the adaptation should happen

Head only

Read the final frozen features. Lowest training burden when they already suffice.

LoRA

Change internal computations of a backbone. Compact task updates and mergeable linear branches.

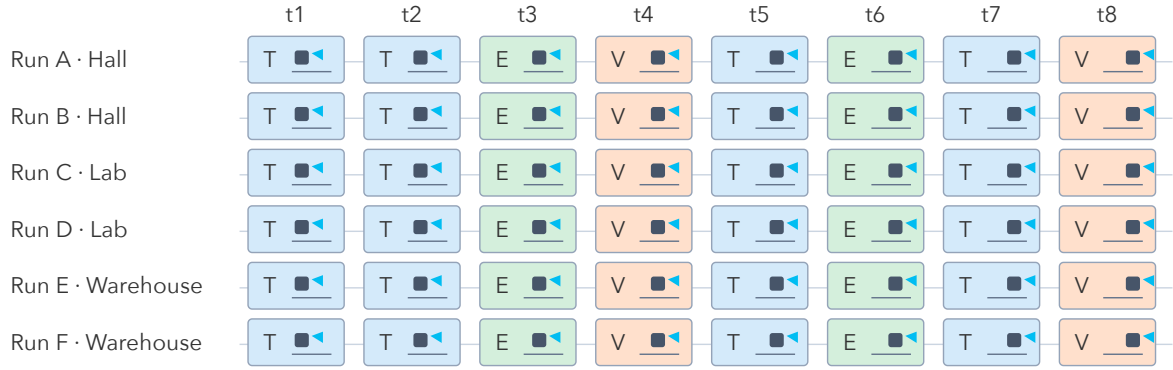
ComBo

Read and combine intermediate features from several models.

Splitting robot data correctly

T · Train V · Validation E · Test / evaluation

Time advances →



Train/test audit · adjacent pairs: **18** · shared trajectories: **6** · shared environments: **3**

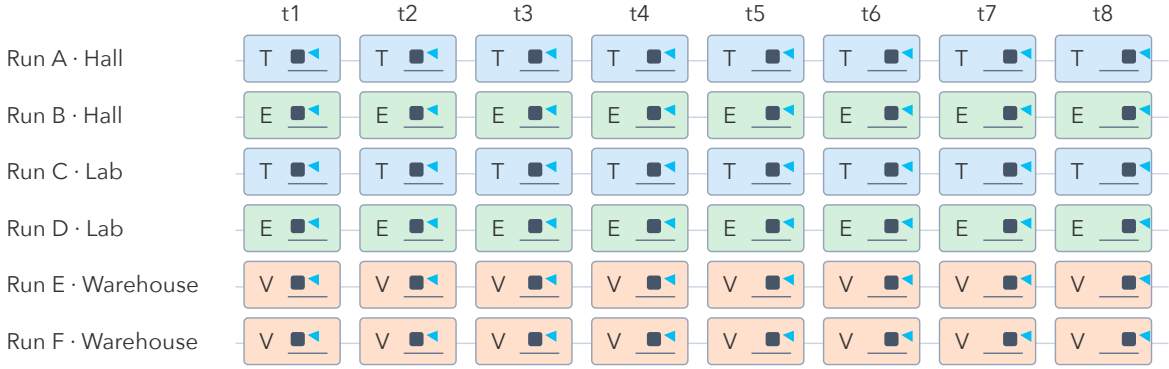
Adjacent frames from the same recording appear on both sides of the test boundary.

scikit-learn · grouped-data evaluation; robot grouping shown here is illustrative

Splitting robot data correctly

T · Train V · Validation E · Test / evaluation

Time advances →



Train/test audit · adjacent pairs: **0** · shared trajectories: **0** · shared environments: **2**

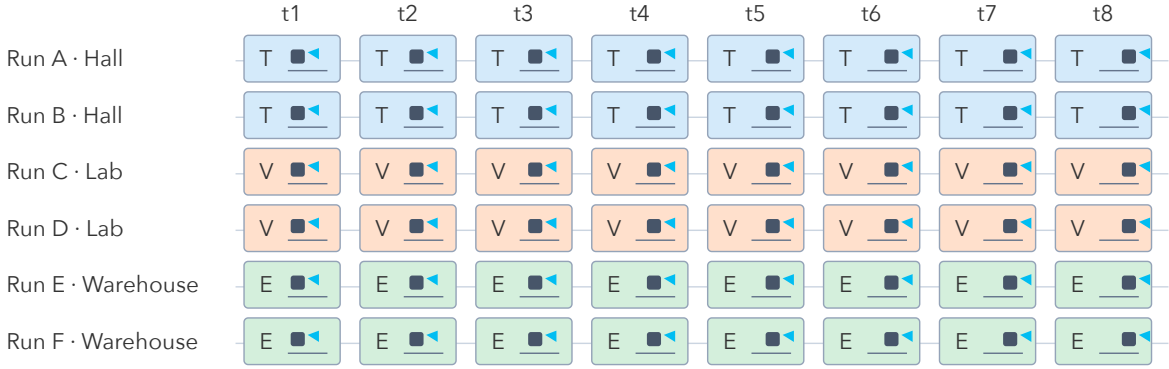
Whole runs are separated, but Hall and Lab still occur in both train and test.

scikit-learn · grouped-data evaluation; robot grouping shown here is illustrative

Splitting robot data correctly

T · Train V · Validation E · Test / evaluation

Time advances →



Train/test audit · adjacent pairs: **0** · shared trajectories: **0** · shared environments: **0**

Whole environments are separated in these metadata; verify the actual recordings too.

Choose the split for the claim: new run, recording session, object, or environment.

scikit-learn · grouped-data evaluation; robot grouping shown here is illustrative

Course synthesis

Related views	Hidden observations	Self-Distillation	Geometry	Later measurements
$\tilde{x}_1 \leftrightarrow \tilde{x}_2$	$x_k \rightarrow$ target x_k	$\mathrm{sg}(f_{\bar{\theta}}(x))$	Depth + poses	$o \rightarrow \Gamma \rightarrow y$
Augmentation / time	Withheld pixels	Target features	Depth / visible matches	Traversal / interaction

Alternative target sources, each with a defined construction rule and supported observations.



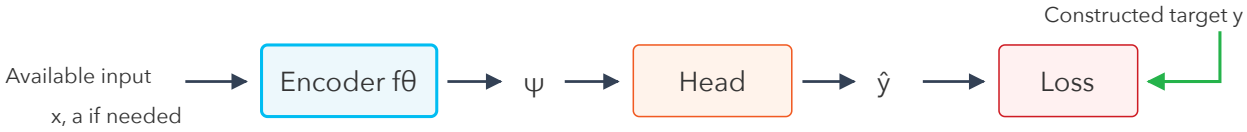
What the signal assumes View relations · representation · visibility · timing · measurement support

The automatic target supplies supervision; the objective shapes what the model preserves.

Course synthesis

Related views	Hidden observations	Self-Distillation	Geometry	Later measurements
$\tilde{x}_1 \leftrightarrow \tilde{x}_2$	$x_k \rightarrow$ target x_k	$\mathrm{sg}(f_{\bar{\theta}}(x))$	Depth + poses	o → Γ → y
Augmentation / time	Withheld pixels	Target features	Depth / visible matches	Traversal / interaction

Alternative target sources, each with a defined construction rule and supported observations.



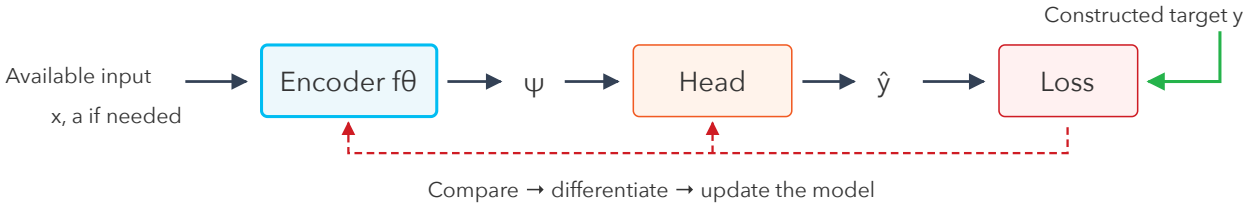
What the signal assumes View relations · representation · visibility · timing · measurement support

The automatic target supplies supervision; the objective shapes what the model preserves.

Course synthesis

Related views	Hidden observations	Self-Distillation	Geometry	Later measurements
$\tilde{x}_1 \leftrightarrow \tilde{x}_2$	$x_k \rightarrow$ target x_k	$\mathrm{sg}(f_{\bar{\theta}}(x))$	Depth + poses	o → Γ → y
Augmentation / time	Withheld pixels	Target features	Depth / visible matches	Traversal / interaction

Alternative target sources, each with a defined construction rule and supported observations.



What the signal assumes View relations · representation · visibility · timing · measurement support

Takeaway: the same representation can be reused for several tasks.

COURSE SOURCES

References

References: representation learning

SimCLR A Simple Framework for Contrastive Learning of Visual Representations
T. Chen, S. Kornblith, M. Norouzi, G. Hinton. **ICML 2020.**

DINO Emerging Properties in Self-Supervised Vision Transformers
M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, A. Joulin. **ICCV 2021.**

VICReg VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning
A. Bardes, J. Ponce, Y. LeCun. **ICLR 2022.**

MAE Masked Autoencoders Are Scalable Vision Learners
K. He, X. Chen, S. Xie, Y. Li, P. Dollár, R. Girshick. **CVPR 2022.**

I-JEPA Self-Supervised Learning From Images With a Joint-Embedding Predictive Architecture
M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, N. Ballas. **CVPR 2023.**
Official training code: src/train.py

Other sources cited in the course

FixMatch FixMatch: Simplifying Semi-Supervised Learning with Consistency and Confidence
K. Sohn, D. Berthelot, C.-L. Li, Z. Zhang, N. Carlini, E. D. Cubuk, A. Kurakin, H. Zhang, C. Raffel. **NeurIPS 2020.**

NetVLAD NetVLAD: CNN Architecture for Weakly Supervised Place Recognition
R. Arandjelović, P. Gronat, A. Torii, T. Pajdla, J. Sivic. **CVPR 2016.**

References: geometry and correspondence

Monodepth2

Digging Into Self-Supervised Monocular Depth Estimation

C. Godard, O. Mac Aodha, M. Firman, G. Brostow. **ICCV 2019.**

Official depth evaluator: evaluate_depth.py · commit b017a81

TCN

Time-Contrastive Networks: Self-Supervised Learning from Video

P. Sermanet, C. Lynch, Y. Chebotar, J. Hsu, E. Jang, S. Schaal, S. Levine. **ICRA 2018.**

Dense Object Nets

Dense Object Nets: Learning Dense Visual Object Descriptors By and For Robotic Manipulation

P. R. Florence, L. Manuelli, R. Tedrake. **CoRL 2018.**

References: interaction and evaluation

WVN

Wild visual navigation: fast traversability learning via pre-trained models and online self-supervision

M. Mattamala, J. Frey, P. Libera, N. Chebrolu, G. Martius, C. Cadena, M. Hutter, M. Fallon.
Autonomous Robots 49, article 19, 2025.

ActAIM

Self-Supervised Learning of Action Affordances as Interaction Modes

L. Wang, N. Dvornik, R. Dubeau, M. Mittal, A. Garg. **ICRA 2023.**

Evaluation source

Evaluation

Cross-validation: evaluating estimator performance — grouped-data iterators

scikit-learn developers. **Official documentation · accessed 7 October 2026.**

References: adaptation and feature fusion

LoRA

LoRA: Low-Rank Adaptation of Large Language Models

E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. **ICLR 2022.**

ComBo

Fantastic Features and Where to Find Them: A Probing Method to combine Features from Multiple Foundation Models

B. Ramtoula, P.-Y. Lajoie, P. Newman, D. De Martini. **NeurIPS 2025.**

Project and original figures